

**O‘ZBEKISTON RESPUBLIKASI RAQAMLI TEXNOLOGIYALAR
VAZIRLIGI
MUHAMMAD AL-XORAZMIY NOMIDAGI
TOSHKENT AXBOROT TEXNOLOGIYALARI UNIVERSITETI**

Himoyaga ruxsat
“Tizimli va amaliy dasturlash”
kafedrası mudiri
_____ K.F. Kerimov
«_____» _____ 2026-yil

BITIRUV MALAKAVIY ISHI

Mavzu: “Milliy chat” dasturini ishlab chiqish
(Telegramga o‘xshash milliy messenger)

Bitiruvchi	_____	S.O. Sohinazarov
	(imzo)	
Ilmiy rahbar	_____	_____
	(imzo)	
Taqrizchi	_____	_____
	(imzo)	
HFX bo‘yicha maslahatchi	_____	S. Abdullayeva
	(imzo)	

Toshkent – 2026

**O‘ZBEKISTON RESPUBLIKASI RAQAMLI TEXNOLOGIYALAR
VAZIRLIGI
MUHAMMAD AL-XORAZMIY NOMIDAGI
TOSHKENT AXBOROT TEXNOLOGIYALARI UNIVERSITETI**

Fakultet: Dasturiy injiniring

Kafedra: Tizimli va amaliy dasturlash

Ta’lim yo‘nalishi: 60610600 – Dasturiy injiniring

TASDIQLAYMAN

“TAD” kafedrası mudiri

_____ K.F. Kerimov

« ____ » _____ 2026-yil

Sohinazarov Sardor O‘rinboy o‘g‘lining

bitiruv malakaviy ishiga

T O P S H I R I Q

1. BMI mavzusi: “Milliy chat” dasturini ishlab chiqish.
2. BMI tasdiqlangan buyruq: « ____ » _____ 2026-yil № _____.
3. BMIni himoyaga topshirish muddati: _____.
4. BMIning boshlang‘ich ma’lumotlari: mavjud messengerlar (Telegram, WhatsApp, Signal) tahlili, real vaqtda xabar almashish texnologiyalari, .NET 8, ASP.NET Core, PostgreSQL, SignalR va Angular texnologiyalariga oid adabiyotlar hamda amaliyot materiallari.
5. Hisob-tushuntirish yozuvining mazmuni: sohaning tahlili va masalaning qo‘yilishi, tizim arxitekturası va ma’lumotlar bazasini loyihalash, server va klient qismlarini ishlab chiqish, xavfsizlik choralari, foydalanish yo‘riqnomasi, hayot faoliyati xavfsizligi, xulosa.
6. Grafik materiallar ro‘yxati: tizim arxitekturası, ER-diagramma, UML (use-case, sequence) diagrammalari, dastur interfeysi skrinshotlari va PowerPoint taqdimoti.
7. Topshiriq berilgan sana: _____.

Rahbar:

(imzo)

Topshiriqni oldim:

(imzo)

8. BMIning alohida bo‘limlari bo‘yicha maslahatchilar:

№	Bo‘lim nomi	Maslahatchi	Topshiriq berdi	Topshiriq oldi
1	Asosiy qism	_____		
2	Hayot faoliyati xavfsizligi	S. Abdullayeva		

9. BMIning bajarish bo‘yicha kalendar grafik:

№	Malakaviy ish bo‘limlari	Bajarish muddati	Rahbar imzosi
1	Kirish, masalaning qo‘yilishi		
2	Soha tahlili, mavjud messenjerlarni o‘rganish		
3	Tizimni loyihalash (arxitektura, ma’lumotlar bazasi, UML)		
4	Dasturning server va klient qismlarini ishlab chiqish		
5	Hayot faoliyati xavfsizligi		
6	Xulosa, adabiyotlar ro‘yxati, ilovalar		
7	Taqdimot slaydlarini tayyorlash		
8	Dastlabki himoya		

Bitiruv malakaviy ish rahbari:

(imzo)

Bitiruvchi:

(imzo)

ANNOTATSIYA

Ushbu bitiruv malakaviy ishi “Milliy chat” nomli, Telegramga o‘xshash milliy messenger dasturini ishlab chiqishga bag‘ishlangan. Ishda zamonaviy messengerlar va real vaqtda aloqa texnologiyalari tahlil qilingan, tizimning arxitekturasi, ma’lumotlar bazasi va UML modellari loyihalangan. Dasturning server qismi .NET 8 platformasida ASP.NET Core, Entity Framework Core, PostgreSQL va SignalR texnologiyalari asosida Clean Architecture tamoyillari bo‘yicha, klient qismi esa Angular freymvorkida ishlab chiqilgan. Tizim elektron pochta orqali bir martalik kod va Google hisobi yordamida autentifikatsiya, shaxsiy va guruh chatlari, real vaqtda xabar almashish, “yozmoqda” va o‘qilganlik belgilari, rasm, video va fayllar yuborish, hikoyalar (stories) hamda onlayn holatni kuzatish imkoniyatlarini taqdim etadi. Ilova Docker konteyneri ko‘rinishida bulutli serverga joylashtirilgan.

АННОТАЦИЯ

Выпускная квалификационная работа посвящена разработке национального мессенджера «Milliy chat», аналогичного Telegram. В работе проанализированы современные мессенджеры и технологии обмена данными в реальном времени, спроектированы архитектура системы, база данных и UML-модели. Серверная часть разработана на платформе .NET 8 с использованием ASP.NET Core, Entity Framework Core, PostgreSQL и SignalR в соответствии с принципами Clean Architecture, клиентская часть — на фреймворке Angular. Система поддерживает аутентификацию по одноразовому коду, отправляемому на электронную почту, и через аккаунт Google, личные и групповые чаты, обмен сообщениями в реальном времени, индикаторы набора текста и прочтения, отправку изображений, видео и файлов, истории (stories), а также отслеживание статуса «в сети». Приложение развёрнуто в облаке в виде Docker-контейнера.

ANNOTATION

This graduation qualification work is devoted to the development of “Milliy chat”, a national messenger similar to Telegram. The work analyses modern messengers and real-time communication technologies and designs the system architecture, the database and the UML models. The server side is built on .NET 8 with ASP.NET Core, Entity Framework Core, PostgreSQL and SignalR following Clean Architecture principles, and the client side is built with the Angular framework. The system provides sign-in with a one-time code sent by e-mail or with a Google account, private and group chats, real-time messaging, typing and read indicators, sending images, videos and files, stories, and online presence tracking. The application is deployed to the cloud as a Docker container.

MUNDARIJA

KIRISH	8
I-BOB. ZAMONAVIY MESSENERLAR VA MILLIY MESSENER YARATISH ZARURIYATI.....	12
1.1. Messenjerlarning rivojlanishi va hozirgi holati	12
1.2. Mavjud messenjerlar tahlili.....	14
1.3. Axborot xavfsizligi va ma'lumotlar lokalizatsiyasi.....	17
1.4. Real vaqtda aloqa texnologiyalari.....	18
1.5. Masalaning qo'yilishi	20
1-bob bo'yicha xulosa	22
II-BOB. "MILLIY CHAT" TIZIMINI LOYIHALASH	24
2.1. Texnologiyalarni tanlash va asoslash.....	24
2.2. Tizim arxitekturasini.....	26
2.3. Ma'lumotlar bazasini loyihalash	31
2.4. Tizimning UML modellari.....	37
2.5. Xavfsizlikni loyihalash	42
2-bob bo'yicha xulosa	45
III-BOB. "MILLIY CHAT" DASTURINI ISHLAB CHIQISH.....	47
3.1. Server qismini ishlab chiqish	47
3.2. Real vaqtda xabar almashishni amalga oshirish	51
3.3. Media fayllar bilan ishlash.....	53
3.4. Guruh chatlari va hikoyalar	55
3.5. Klient qismini ishlab chiqish	57
3.6. Joylashtirish va testlash	60
3.7. Foydalanish yo'riqnomasi.....	64

3-bob bo'yicha xulosa	69
IV-BOB. HAYOT FAOLİYATI XAVFSIZLIGI	71
4.1. Dasturchining ergonomik ish joyini tashkil etish	71
4.2. Evakuatsiya tadbirlarini rejalashtirish hamda avariya va yong'inlar sodir bo'lganda xodimlarning harakatlari	79
4-bob bo'yicha xulosa	88
XULOSA	90
FOYDALANILGAN ADABIYOTLAR	92
ILOVALAR	95
1-ilova. SignalR habi (ChatHub)	95
2-ilova. Guruh chatlari servisi (GroupService)	97
3-ilova. Bir martalik kodlarni xeshlash (PBKDF2)	102
4-ilova. Klientdagi real vaqt ulanishi (ChatRealtimeService)	102
5-ilova. Guruh chatlari uchun unit testlar (parcha)	104

KIRISH

Bugungi kunda axborot-kommunikatsiya texnologiyalari jamiyat hayotining barcha sohalariga chuqur kirib borgan. Odamlar o'rtasidagi muloqotning katta qismi telefon qo'ng'iroqlari va elektron pochtdan tezkor xabar almashish dasturlariga, ya'ni messengerlarga ko'chdi. Messengerlar endi faqat do'stlar va yaqinlar bilan suhbatlashish vositasi emas: ularda o'quv guruhlar va ish jamoalari tashkil etiladi, yangiliklar tarqatiladi, davlat va tijorat xizmatlari ko'rsatiladi. Shu sababli messenger bugungi kunda muhim axborot infratuzilmasining bir qismiga aylangan.

O'zbekistonda ham messengerlardan foydalanish keng tarqalgan. Aholining asosiy qismi xorijiy kompaniyalar tomonidan ishlab chiqilgan va boshqariladigan Telegram, WhatsApp kabi ilovalardan foydalanadi. Bu ilovalar qulay bo'lsa-da, fuqarolarning yozishmalari, aloqalar ro'yxati va boshqa shaxsga doir ma'lumotlari mamlakat hududidan tashqarida joylashgan serverlarda saqlanadi. Ushbu serverlar ustidan milliy nazorat yo'q, xizmat ko'rsatish shartlari esa istalgan vaqtda bir tomonlama o'zgartirilishi mumkin.

Mavzuning dolzarbligi.

O'zbekiston Respublikasi Prezidentining 2020-yil 5-oktabrdagi PF-6079-son Farmoni bilan tasdiqlangan "Raqamli O'zbekiston – 2030" strategiyasi raqamli infratuzilmani rivojlantirish, mahalliy dasturiy mahsulotlar ishlab chiqarishni kengaytirish va axborot xavfsizligini ta'minlashni ustuvor vazifalar sifatida belgilaydi. "Shaxsga doir ma'lumotlar to'g'risida"gi Qonunga 2021-yilda kiritilgan o'zgartishlarga ko'ra esa O'zbekiston fuqarolarining shaxsga doir ma'lumotlari respublika hududida joylashgan texnik vositalarda to'planishi va saqlanishi lozim.

Bu talablar milliy messenger yaratish masalasini dolzarb qiladi. Milliy messengerda foydalanuvchi ma'lumotlari mamlakat ichidagi serverlarda saqlanadi, interfeys o'zbek tilida bo'ladi, dastur kodi va arxitekturasi esa mahalliy mutaxassislar nazoratida rivojlantiriladi. Bundan tashqari, bunday loyihani ishlab chiqish mahalliy dasturchilarning yuqori yuklamali, real vaqtda ishlaydigan tizimlarni yaratish tajribasini oshiradi.

Messenjer yaratish texnik jihatdan ham murakkab va qiziqarli masala. Xabarlar bir necha millisekund ichida yetkazilishi, foydalanuvchining onlayn holati va “yozmoqda” belgisi darhol ko‘rinishi, katta hajmdagi yozishmalar tarixi tez yuklanishi, media fayllar xavfsiz saqlanishi, autentifikatsiya esa ishonchli bo‘lishi kerak. Bu masalalarni hal etish zamonaviy dasturiy injiniring usullarini, jumladan qatlamli arxitektura, real vaqtda aloqa protokollari va ma’lumotlar bazasini optimallashtirishni amalda qo‘llashni talab qiladi.

Bitiruv malakaviy ishining maqsadi

Ishning maqsadi — Telegramga o‘xshash, foydalanuvchi ma’lumotlarini o‘z serverida saqlaydigan, shaxsiy va guruh yozishmalarini real vaqtda ta’minlaydigan “Milliy chat” veb-messenjerini loyihalash va ishlab chiqishdir.

Qo‘yilgan maqsadga erishish uchun quyidagi vazifalar belgilandi:

- mavjud messenjerlarni (Telegram, WhatsApp, Signal, Viber va boshqalar) funksional imkoniyatlari, xavfsizligi va arxitekturasini bo‘yicha tahlil qilish;
- real vaqtda ma’lumot almashish texnologiyalarini (long polling, Server-Sent Events, WebSocket, SignalR) o‘rganish va tizim uchun eng maqbulini tanlash;
- tizimga qo‘yiladigan funksional va nofunksional talablarni aniqlash;
- Clean Architecture tamoyillari asosida tizim arxitekturasini, PostgreSQL ma’lumotlar bazasi sxemasini va UML modellarini loyihalash;
- elektron pochtaga yuboriladigan bir martalik kod, Google hisobi, JWT va refresh token asosida xavfsiz autentifikatsiya va sessiyalarni boshqarish tizimini ishlab chiqish;
- shaxsiy va guruh chatlari, xabarlarni tahrirlash va o‘chirish, qidirish, media fayllar yuborish, hikoyalar va onlayn holat funksiyalarini .NET 8 platformasida amalga oshirish;
- Angular freymvorkida moslashuvchan (responsive), yorug‘ va qorong‘i mavzuni qo‘llab-quvvatlaydigan foydalanuvchi interfeysini yaratish;
- dasturni Docker konteyneri ko‘rinishida bulutli serverga joylashtirish va sinovdan o‘tkazish;

- dasturchining ish joyini ergonomik talablar asosida tashkil etish, evakuatsiya tadbirlarini rejalashtirish va favqulodda vaziyatlarda xodimlarning harakatlar tartibini ishlab chiqish.

Tadqiqot obyekti va predmeti.

Tadqiqot obyekti — foydalanuvchilar o‘rtasida matnli va multimedia xabarlar almashishni ta‘minlaydigan onlayn muloqot tizimlari. Tadqiqot predmeti — real vaqtda ishlaydigan messenjerning arxitekturasini, ma‘lumotlar modeli, xabarlarni yetkazish mexanizmlari hamda ularni .NET va Angular texnologiyalari yordamida amalga oshirish usullari.

Tadqiqot usullari.

Ishda qiyosiy tahlil, tizimli yondashuv, obyektga yo‘naltirilgan loyihalash, UML yordamida modellashtirish, relatsion ma‘lumotlar bazasini normallashtirish hamda dasturiy ta‘minotni bosqichma-bosqich (iterativ) ishlab chiqish va sinash usullaridan foydalanildi. Dastur kodi Git versiyalar nazorati tizimida yuritildi. Har bir funksiya alohida tarmoqda (branch) ishlab chiqilib, tekshirilgandan so‘ng asosiy tarmoqqa birlashtirildi.

Ishning amaliy ahamiyati.

Ishlab chiqilgan “Milliy chat” tizimi to‘liq ishlaydigan veb-ilova bo‘lib, uni tashkilotlar, ta‘lim muassasalari yoki davlat idoralari o‘z serverlarida joylashtirib, ichki muloqot vositasi sifatida qo‘llashi mumkin. Tizim modulli tuzilgan. Shu sababli unga kanallar, ovozli va video qo‘ng‘iroqlar, stikerlar, botlar kabi yangi imkoniyatlarni qo‘shish oson. Ma‘lumotlar bazasi sxemasida bu imkoniyatlar uchun zarur jadvallar oldindan loyihalangan.

Bitiruv malakaviy ishining tuzilishi.

Ish kirish, to‘rt bob, xulosa, foydalanilgan adabiyotlar ro‘yxati va ilovalardan iborat. Birinchi bobda messenjerlarning rivojlanishi, mavjud yechimlar va real vaqtda aloqa texnologiyalari tahlil qilinib, masala qo‘yilgan. Ikkinchi bobda texnologiyalar tanlangan, tizim arxitekturasini, ma‘lumotlar bazasi, UML modellari va xavfsizlik mexanizmlari loyihalangan. Uchinchi bobda dasturning server va

klient qismlarini ishlab chiqish, joylashtirish va sinash jarayonlari hamda foydalanish yo'riqnomasi keltirilgan. To'rtinchi bobda hayot faoliyati xavfsizligi doirasida dasturchining ergonomik ish joyini tashkil etish hamda evakuatsiya tadbirlarini rejalashtirish va avariya, yong'inlar sodir bo'lganda xodimlarning harakatlari ko'rib chiqilgan. Ish 32 ta rasm, 17 ta jadval, 6 ta kod listingi va 5 ta ilovani o'z ichiga oladi.

I-BOB. ZAMONAVIY MESSENERLAR VA MILLIY MESSENER YARATISH ZARURIYATI

1.1. Messenjerlarning rivojlanishi va hozirgi holati

Messenjer (ingl. instant messenger — tezkor xabar almashish dasturi) — foydalanuvchilar oʻrtasida matnli, ovozli, video va boshqa turdagi xabarlarni Internet orqali deyarli bir zumda yetkazib beradigan dasturiy taʼminot. Elektron pochtdan farqli ravishda messenjerda muloqot suhbat (dialog) koʻrinishida kechadi. Xabar qabul qiluvchiga darhol yetkaziladi, joʻnatuvchi esa suhbatdoshning onlayn ekanligi, xabarni oʻqigani yoki javob yozayotgani haqida maʼlumot oladi.

Tezkor xabar almashishning ildizlari 1970-yillardagi koʻp foydalanuvchili operatsion tizimlarga borib taqaladi. Oʻsha davrda bitta katta kompyuterga ulangan terminallar foydalanuvchilari bir-biriga qisqa xabar yubora olardi. 1988-yilda finlyandiyalik dasturchi Yarkko Oykariinen IRC (Internet Relay Chat) protokolini yaratdi. IRC Internetdagi birinchi ommaviy chat tizimi boʻlib, unda foydalanuvchilar mavzuli kanallarga birlashib suhbatlashgan.

Shaxsiy kompyuterlar va Internetning ommalashuvi bilan 1990-yillarning ikkinchi yarmida birinchi ommaviy messenjerlar paydo boʻldi. 1996-yilda isroillik Mirabilis kompaniyasi ICQ (“I seek you” — “seni izlayapman”) dasturini chiqardi. ICQ har bir foydalanuvchiga raqamli identifikator berdi, kontaktlar roʻyxati va onlayn holat tushunchalarini ommalashtirdi. Keyingi yillarda AOL Instant Messenger (1997), Yahoo! Messenger (1998) va MSN Messenger (1999) kabi raqobatchilar paydo boʻldi. 1999-yilda ochiq Jabber loyihasi boshlandi. Uning asosida keyinchalik IETF tomonidan standartlashtirilgan XMPP (Extensible Messaging and Presence Protocol) protokoli ishlab chiqildi. 2003-yilda chiqarilgan Skype esa matnli chatni ovozli va video qoʻngʻiroqlar bilan birlashtirdi.

Messenjerlar rivojidagi keyingi burilish nuqtasi smartfonlar bilan bogʻliq. 2007–2008-yillarda iPhone va mobil ilovalar doʻkonlarining paydo boʻlishi doimiy Internetga ulangan qurilmani har bir foydalanuvchining choʻntagiga olib kirdi. 2009-yilda Yan Kum va Brayan Ekton tomonidan asos solingan WhatsApp

foydalanuvchini login va parol o‘rniga telefon raqami orqali identifikatsiya qildi va kontaktlarni telefon kitobidan avtomatik oldi. Bu yondashuv messenjerni SMS xabarlarining bepul muqobiliga aylantirdi. 2014-yilda WhatsApp Facebook (hozirgi Meta) kompaniyasi tomonidan sotib olindi. Shu davrda Viber (2010), WeChat (2011), LINE (2011) va KakaoTalk (2010) kabi ilovalar ham paydo bo‘ldi.

2013-yilda Pavel va Nikolay Durovlar Telegram messenjerini ishga tushirdi. Telegram “bulutli” yondashuvni tanladi: yozishmalar serverda saqlanadi va foydalanuvchining barcha qurilmalarida bir vaqtda mavjud bo‘ladi. Telegram minglab a‘zoli guruhlar, cheksiz obunachili kanallar, botlar platformasi va katta hajmdagi fayllarni yuborish imkoniyati bilan tezda ommalashdi. 2014–2015-yillarda maxfiylikka urg‘u bergan Signal messenjeri chiqdi. Uning Signal Protocol deb nomlangan uchdan-uchgacha (end-to-end) shifrlash protokoli keyinchalik WhatsApp va boshqa ilovalarda ham qo‘llanila boshlandi.

Hozirgi bosqichda messenjerlar quyidagi yo‘nalishlarda rivojlanmoqda:

- **xavfsizlik va maxfiylik** — uchdan-uchgacha shifrlash, ikki bosqichli autentifikatsiya, faol sessiyalarni boshqarish, o‘z-o‘zidan o‘chadigan xabarlar;
- **ko‘p qurilmalilik** — bitta hisobdan telefon, kompyuter va brauzerda bir vaqtda foydalanish;
- **ommaviy kommunikatsiya** — katta guruhlar, kanallar, hamjamiyatlar;
- **qisqa muddatli kontent** — 24 soatdan keyin o‘chadigan hikoyalar (stories);
- **platformaga aylanish** — botlar, mini-ilovalar, to‘lov tizimlari. Bunday “super-ilova”ning yorqin namunasi xitoyning WeChat ilovasidir;
- **media imkoniyatlar** — ovozli va video xabarlar, guruhli video qo‘ng‘iroqlar, katta fayllar almashish.

Messenjerlar auditoriyasi juda katta. Meta kompaniyasi 2020-yilda WhatsApp foydalanuvchilari soni 2 milliarddan oshganini, Telegram asoschisi esa 2025-yilda Telegramning oylik faol foydalanuvchilari 1 milliardga yetganini e‘lon

qildi. Bu raqamlar messenjerlar bugungi kunda elektron pochta va ijtimoiy tarmoqlar bilan bir qatorda asosiy aloqa kanaliga aylanganini ko'rsatadi.

Mamlakatlar kesimida qaralganda, ayrim davlatlarda xalqaro ilovalar emas, mahalliy messenjerlar ustunlik qiladi. Janubiy Koreyada KakaoTalk, Yaponiyada LINE, Vyetnamda Zalo, Xitoyda esa WeChat eng ommabop messenjer hisoblanadi. Bu misollar milliy messenjer xalqaro raqobatchilar bilan muvaffaqiyatli raqobatlasha olishini ko'rsatadi. Buning uchun u mahalliy foydalanuvchilar ehtiyojlarini yaxshi hisobga olishi va sifatli texnik yechimga ega bo'lishi kerak.

1.2. Mavjud messenjerlar tahlili

Ishlab chiqiladigan tizimga qo'yiladigan talablarni aniqlash uchun eng ko'p tarqalgan messenjerlarning funksional imkoniyatlari, xavfsizlik modeli va arxitekturasi tahlil qilindi. Tahlil uchun O'zbekistonda keng qo'llaniladigan Telegram va WhatsApp, maxfiylik bo'yicha etalon hisoblangan Signal, shuningdek Viber va WeChat tanlandi.

Telegram.

Telegram — BAAda ro'yxatdan o'tgan kompaniya tomonidan ishlab chiqilayotgan bulutli messenjer. U o'zining MTPROTO 2.0 protokolidan foydalanadi. Oddiy (bulutli) chatlarda xabarlar mijoz va server o'rtasida shifrlanadi va serverda saqlanadi. Shu tufayli foydalanuvchi har qanday qurilmadan o'z yozishmalariga kira oladi. Uchdan-uchgacha shifrlash faqat alohida yaratiladigan "maxfiy chat"larda (secret chats) qo'llanadi. Telegram 200 000 a'zogacha bo'lgan guruhlarini, cheksiz obunachili kanallarni, Bot API orqali botlarni va 2 GB gacha bo'lgan fayllarni yuborishni qo'llab-quvvatlaydi. Klient ilovalarining manba kodi ochiq, server qismi esa yopiq.

Telegramning kuchli tomonlari — tezligi, bulutli sinxronizatsiya, boy funktsionallik va ochiq API. Kamchiliklari — standart chatlarning uchdan-uchgacha shifrlanmagani, serverlarning xorijda joylashgani va server kodining yopiqligi. Foydalanuvchi ma'lumotlari qaysi yurisdiksiyada saqlanishi va ularga kim kira olishini tashqaridan nazorat qilib bo'lmaydi.

WhatsApp.

WhatsApp Meta kompaniyasiga tegishli. Foydalanuvchi telefon raqami orqali ro'yxatdan o'tadi. 2016-yildan boshlab barcha shaxsiy va guruh yozishmalari Signal Protocol asosida uchdan-uchgacha shifrlanadi. Xabarlar serverda saqlanmaydi, faqat yetkazilgunga qadar vaqtincha turadi. WhatsApp guruhlarida 1024 a'zoga bo'lishi mumkin. Bundan tashqari hamjamiyatlar, kanallar va "Status" (hikoyalar) funksiyalari mavjud. Kamchiliklari: kodi yopiq, xabar mazmuni shifrlangan bo'lsa-da, metama'lumotlar (kim, kim bilan, qachon yozishgani) kompaniya ixtiyorida qoladi, biznes-modeli esa Meta ekotizimi bilan bog'liq.

Signal.

Signal notijorat Signal Foundation tashkiloti tomonidan ishlab chiqiladi. Uning klient va server kodlari to'liq ochiq. Barcha yozishmalar sukut bo'yicha uchdan-uchgacha shifrlanadi, serverda saqlanadigan metama'lumotlar esa minimal darajaga keltirilgan. "Sealed sender" texnologiyasi hatto serverdan ham xabar jo'natuvchisini yashiradi. Signal maxfiylik bo'yicha etalon hisoblanadi, lekin funksional jihatdan Telegramdan kambag'alroq: kanallar, botlar va katta ommaviy guruhlar yo'q.

Viber va WeChat.

Viber yapon Rakuten kompaniyasiga tegishli bo'lib, shaxsiy va guruh chatlarida uchdan-uchgacha shifrlashni qo'llaydi. U ayniqsa ovozli qo'ng'iroqlar uchun Sharqiy Yevropa va MDH mamlakatlarida ommalashgan. Xitoyning Tencent kompaniyasiga tegishli WeChat esa messenjer, ijtimoiy tarmoq, to'lov tizimi va mini-ilovalar platformasini birlashtirgan "super-ilova" hisoblanadi. U uchdan-uchgacha shifrlashni qo'llamaydi va Xitoy qonunchiligiga to'liq bo'ysunadi.

Ko'rib chiqilgan messenjerlarning asosiy xususiyatlari va ishlab chiqilayotgan "Milliy chat" tizimi bilan taqqoslanishi 1.1-jadvalda keltirilgan.

Messenjerlarning qiyosiy tahlili

Mezon	Telegram	WhatsApp	Signal	WeChat	Milliy chat
Egasi (davlat)	Telegram (BAA)	Meta (AQSh)	Signal Foundation (AQSh)	Tencent (Xitoy)	Mahalliy ishlab chiquvchi
Ro'yxatdan o'tish	Telefon raqami	Telefon raqami	Telefon raqami	Telefon raqami	E-pochta kodi, Google
E2E shifrlash	Faqat maxfiy chatlarda	Sukut bo'yicha	Sukut bo'yicha	Yo'q	Rejalashtirilgan (TLS mavjud)
Yozishmalar saqlanishi	Bulutda (serverda)	Qurilmada	Qurilmada	Serverda	O'z serverida (bulutli)
Guruh hajmi	200 000	1024	1000	500	200
Kanallar, botlar	Bor	Kanallar bor	Yo'q	Bor	Kelajakda (sxemasi tayyor)
Hikoyalar	Bor	Bor (Status)	Bor	Bor	Bor
Server kodi	Yopiq	Yopiq	Ochiq	Yopiq	Mahalliy nazoratda
Ma'lumotlar joylashuvi	Xorijda	Xorijda	Xorijda	Xitoyda	Istalgan serverda, jumladan O'zbekistonda

Tahlil natijalari shuni ko'rsatadiki, mavjud messenjerlar funksionallik va qulaylik bo'yicha yuqori darajada. Ammo ularning barchasi xorijiy kompaniyalar tomonidan boshqariladi va ma'lumotlarni mamlakatdan tashqarida saqlaydi. Telegram funksionallik, bulutli sinxronizatsiya va foydalanish qulayligi bo'yicha eng yaxshi muvozanatni taklif etadi. Shu sababli "Milliy chat" tizimi uchun foydalanuvchi tajribasi (UX) bo'yicha asosiy namuna sifatida Telegram tanlandi.

Shu bilan birga, Telegramning ayrim yechimlari bitiruv malakaviy ishi doirasida soddalashtirildi yoki o'zgartirildi. Masalan, ro'yxatdan o'tish SMS o'rniga elektron pochta orqali amalga oshiriladi, chunki SMS-shlyuzlar pullik va operatorlar bilan shartnoma talab qiladi. Google hisobi orqali kirish esa foydalanuvchiga qo'shimcha qulaylik beradi.

1.3. Axborot xavfsizligi va ma'lumotlar lokalizatsiyasi

Messenjer foydalanuvchi haqidagi eng shaxsiy ma'lumotlarni o'z ichiga oladi. Bular yozishmalar mazmuni, yuborilgan fotosuratlar va hujjatlar, kontaktlar ro'yxati, qaysi vaqtda kim bilan muloqot qilingani, IP-manzillar va qurilmalar haqidagi ma'lumotlar. Bu ma'lumotlarning to'planishi ularni saqlovchi tashkilot uchun katta mas'uliyat va jiddiy xavf manbaidir.

Xorijiy messenjerlardan foydalanishda quyidagi xavflar mavjud:

- **yurisdiksiya xavfi** — ma'lumotlar boshqa davlat qonunlariga bo'ysunadi va ularni o'sha davlat organlari talab qilishi mumkin;
- **metama'lumotlar xavfi** — xabar mazmuni shifrlangan bo'lsa ham, muloqot grafigi (kim kim bilan qancha yozishgani) xizmat egasida qoladi va tahlil qilinishi mumkin;
- **xizmatning uzilishi xavfi** — xizmat ko'rsatish shartlari o'zgarishi, hisob bloklanishi yoki xizmatning mintaqada cheklanishi foydalanuvchini aloqa vositasidan bir zumda mahrum qiladi;
- **nazorat qilib bo'lmazlik** — yopiq server kodi ma'lumotlar qanday qayta ishlanishini mustaqil tekshirishga imkon bermaydi;
- **axborot xurujlari** — firibgarlik, fishing va zararli fayllar tarqatish ko'pincha ommaviy messenjerlar orqali amalga oshiriladi, ularga qarshi kurash esa xorijiy platforma siyosatiga bog'liq.

O'zbekiston qonunchiligi bu masalalarni tartibga soladi. 2019-yil 2-iyulda qabul qilingan "Shaxsga doir ma'lumotlar to'g'risida"gi O'RQ-547-son Qonun shaxsga doir ma'lumotlarni yig'ish, saqlash, qayta ishlash va himoya qilish tartibini belgilaydi. Qonunga 2021-yilda kiritilgan qo'shimchalarga muvofiq, O'zbekiston fuqarolarining shaxsga doir ma'lumotlarini Internet orqali qayta ishlovchi mulkdorlar va operatorlar ularni respublika hududida joylashgan va davlat ro'yxatidan o'tgan ma'lumotlar bazalarida saqlashi shart. "Raqamli O'zbekiston – 2030" strategiyasi esa milliy dasturiy mahsulotlar va xizmatlarni rivojlantirishni ustuvor yo'nalish sifatida belgilaydi.

Milliy messenjer bu talablarga tabiiy ravishda javob beradi. Uning serverlari mamlakat ichidagi data-markazda joylashtirilishi mumkin. Dastur kodi mahalliy mutaxassislar nazoratida bo‘ladi. Xavfsizlik siyosatini esa milliy qonunchilik va ehtiyojlarga moslashtirish mumkin. Masalan, “Milliy chat” tizimida yuklanayotgan fayllar ClamAV antivirusi orqali tekshiriladi. Foydalanuvchi o‘z hisobiga ulangan barcha qurilmalarni ko‘rishi va begona qurilmani masofadan chiqarib yuborishi mumkin. Bir martalik kodlar va refresh tokenlar ma’lumotlar bazasida ochiq holda emas, faqat xesh ko‘rinishida saqlanadi.

Axborot xavfsizligining asosiy uch talabi — maxfiylik, yaxlitlik va foydalanish imkoniyati (CIA triadasi) — “Milliy chat” loyihasida quyidagicha ta’minlanadi. Maxfiylik barcha trafikni HTTPS/TLS orqali shifrlash, qisqa muddatli JWT tokenlar va chatga kirish huquqini har bir so‘rovda tekshirish orqali ta’minlanadi. Yaxlitlik ma’lumotlar bazasi darajasidagi cheklovlar, tranzaksiyalar va kiruvchi ma’lumotlarni validatsiya qilish orqali, foydalanish imkoniyati esa holatsiz (stateless) server, konteynerlash va avtomatik qayta ulanadigan real vaqt aloqasi orqali ta’minlanadi.

1.4. Real vaqtda aloqa texnologiyalari

Messenjerning asosiy texnik vazifasi — yangi xabarni qabul qiluvchiga imkon qadar tez yetkazish. Klassik HTTP protokoli “so‘rov–javob” modeliga asoslangan: faqat mijoz so‘rov yuborishi va server unga javob qaytarishi mumkin. Server o‘z tashabbusi bilan mijozga ma’lumot yubora olmaydi. Shu sababli veb-ilovalarda serverdan mijozga ma’lumot “itarish” (push) uchun bir necha texnologiya ishlab chiqilgan.

Qisqa so‘rovlar (short polling).

Eng oddiy usulda mijoz serverga har bir necha soniyada “yangi xabar bormi?” degan so‘rov yuboradi. Usulni amalga oshirish oson, lekin u samarasiz: so‘rovlarning aksariyati bo‘sh javob qaytaradi, server va tarmoq ortiqcha yuklanadi, xabarlar esa so‘rovlar orasidagi interval qadar kechikib yetkaziladi.

Uzun so‘rovlar (long polling).

Bu usulda server so‘rovga darhol javob bermaydi. U yangi ma’lumot paydo bo‘lguncha yoki belgilangan vaqt tugaguncha ulanishni ochiq ushlab turadi. Javob kelgach, mijoz darhol yangi so‘rov yuboradi. Kechikish sezilarli kamayadi, lekin har bir xabar uchun yangi HTTP so‘rovi va sarlavhalar yuborish kerak bo‘ladi. Server esa ko‘p sonli ochiq so‘rovlarni ushlab turishga majbur bo‘ladi.

Server-Sent Events (SSE).

SSE — HTML standartiga kiritilgan texnologiya. Unda mijoz bitta HTTP ulanishini ochadi, server esa shu ulanish orqali text/event-stream formatida hodisalarni uzluksiz yuboradi. Brauzerdagi EventSource obyekti ulanish uzilganda avtomatik qayta ulanadi. SSE oddiy va samarali, lekin u bir yo‘nalishli: mijozdan serverga ma’lumot yuborish uchun alohida HTTP so‘rovlari kerak.

WebSocket.

WebSocket protokoli 2011-yilda IETF tomonidan RFC 6455 standarti sifatida qabul qilingan. Ulanish oddiy HTTP so‘rovi bilan boshlanadi (Upgrade: websocket sarlavhasi). Server 101 Switching Protocols javobini qaytargach, shu TCP ulanish ikki tomonlama (full-duplex) kanalga aylanadi. Shundan so‘ng mijoz ham, server ham istalgan vaqtda kichik hajmli freymmlar ko‘rinishida xabar yuborishi mumkin. WebSocket minimal kechikish va minimal qo‘shimcha trafikni ta’minlaydi, shuning uchun u zamonaviy veb-messenjerlar uchun asosiy transport hisoblanadi.

SignalR.

ASP.NET Core SignalR — Microsoft tomonidan ishlab chiqilgan, real vaqtda ishlaydigan veb-ilovalar yaratish uchun mo‘ljallangan kutubxona. SignalR transport darajasini abstraksiya qiladi. U avval WebSocket orqali ulanishga harakat qiladi. Agar brauzer yoki tarmoq infratuzilmasi WebSocketni qo‘llab-quvvatlamasa, avtomatik ravishda Server-Sent Events yoki long polling usuliga o‘tadi. Dasturchi transport tafsilotlari bilan emas, “hab” (Hub) deb ataluvchi yuqori darajadagi abstraksiya bilan ishlaydi.

Habda server metodlari e'lon qilinadi va ularni mijoz masofadan chaqira oladi (RPC). Server esa mijozlardagi funksiyalarni chaqiradi. SignalR ulanishlarni guruhlariga birlashtirish, ma'lum foydalanuvchiga yoki guruhga xabar yuborish, JWT orqali autentifikatsiya va avtomatik qayta ulanish kabi messenjer uchun zarur imkoniyatlarni tayyor holda beradi. Tizimni bir nechta serverga kengaytirish uchun esa Redis backplane yoki Azure SignalR Service qo'llaniladi.

Messenjerlarda boshqa protokollar ham qo'llanadi. Masalan, XMPP ochiq standart sifatida ko'plab korporativ chatlarda ishlatiladi. MQTT esa kam resursli qurilmalar va mobil tarmoqlar uchun mo'ljallangan yengil publish/subscribe protokoli. Ko'rib chiqilgan texnologiyalar 1.2-jadvalda taqqoslangan.

1.2-jadval

Real vaqtda aloqa texnologiyalarining taqqoslanishi

Texnologiya	Yo'nalish	Kechikish	Server yuklamasi	Brauzerda qo'llab-quvvatlash
Short polling	Mijoz → server	Yuqori (interval)	Yuqori	Barcha brauzerlar
Long polling	Mijoz → server	O'rtacha	O'rtacha	Barcha brauzerlar
Server-Sent Events	Server → mijoz	Past	Past	Zamonaviy brauzerlar
WebSocket	Ikki tomonlama	Juda past	Past	Zamonaviy brauzerlar
SignalR	Ikki tomonlama (WebSocket, SSE yoki long polling)	Juda past	Past	Barcha brauzerlar (fallback hisobiga)

Tahlil asosida “Milliy chat” tizimida real vaqtda aloqa uchun SignalR tanlandi. U WebSocketning samaradorligini saqlagan holda eski yoki cheklangan tarmoqlarda ham ishlashni kafolatlaydi. Server tomoni .NET platformasida yozilgani uchun SignalR tabiiy integratsiyani ta'minlaydi, brauzer uchun esa rasmiy @microsoft/signalr kutubxonasi mavjud.

1.5. Masalaning qo'yilishi

Yuqoridagi tahlil natijalariga ko'ra, Telegram namunasi asosida veb-brauzerda ishlaydigan, foydalanuvchi ma'lumotlarini o'z serverida saqlaydigan

“Milliy chat” messenjerini ishlab chiqish talab etiladi. Tizimda uch turdagi foydalanuvchi (aktor) mavjud: tizimga hali kirmagan **mehmon**, ro‘yxatdan o‘tgan **foydalanuvchi** va guruhni boshqaruvchi **guruh administratori (egasi)**.

Funksional talablar. Tizim quyidagi imkoniyatlarni ta’minlashi kerak:

1. elektron pochtaga yuboriladigan 6 xonali bir martalik kod yoki Google hisobi orqali ro‘yxatdan o‘tish va tizimga kirish;
2. profilni (ism, familiya, username, bio) tahrirlash va profil rasmini yuklash;
3. faol sessiyalar (qurilmalar) ro‘yxatini ko‘rish, alohida sessiyani yoki boshqa barcha sessiyalarni yakunlash;
4. foydalanuvchilarni ism yoki username bo‘yicha qidirish va kontaktlar ro‘yxatini yuritish;
5. shaxsiy (ikki kishilik) chat yaratish, chatlar ro‘yxatini oxirgi xabar va o‘qilmagan xabarlar soni bilan ko‘rish;
6. guruh chatlarini yaratish, a’zolari qo‘shish va chiqarish, administratorlar tayinlash, guruh nomi, tavsifi va rasmini o‘zgartirish, guruhdan chiqish;
7. matnli xabar yuborish, xabarga javob berish (reply), o‘z xabarini tahrirlash va o‘chirish, chat tarixini tozalash;
8. rasm, video va ixtiyoriy fayllarni yuborish, chatdagi media fayllarni turlari bo‘yicha ko‘rish;
9. chat ichida xabarlarni matn bo‘yicha qidirish va topilgan xabar kontekstiga o‘tish;
10. yangi xabarlarni, “yozmoqda” holatini, o‘qilganlik belgilarini va suhbatdoshning onlayn holatini sahifani yangilamasdan real vaqtda ko‘rsatish;
11. 24 soat davomida ko‘rinadigan hikoyalar (stories) joylash, ko‘rish va ularni kim ko‘rganini bilish;
12. yorug‘ va qorong‘i mavzular hamda mobil qurilmalarga moslashuvchan interfeys.

Nofunksional talablar:

- **unumdorlik** — xabarlar va chatlar ro'yxati sahifalab (cursor pagination) yuklanishi, real vaqt xabarlari 1 soniyadan kam kechikish bilan yetkazilishi;
- **xavfsizlik** — HTTPS, qisqa muddatli access token va xavfsiz cookie'dagi refresh token, kodlar va tokenlarni xesh ko'rinishida saqlash, har bir so'rovda chatga kirish huquqini tekshirish, yuklanayotgan fayllarni antivirus orqali tekshirish;
- **ishonchlilik** — xatolarni markazlashgan holda qayta ishlash, real vaqt ulanishi uzilganda avtomatik qayta ulanish;
- **kengaytiriluvchanlik** — holatsiz API, onlayn holatni Redis orqali saqlash imkoniyati, yangi funksiyalarni boshqa qismlarga ta'sir qilmasdan qo'shish mumkinligi;
- **qo'llab-quvvatlanuvchanlik** — Clean Architecture qatlamlari, yagona kod uslubi, versiyalar nazorati;
- **ko'chiriluvchanlik** — tizim Docker konteynerida ishga tushirilishi va istalgan serverga joylashtirilishi;
- **foydalanish qulayligi** — o'zbek tilidagi interfeys, Telegram foydalanuvchilariga tanish bo'lgan joylashuv va boshqaruv elementlari.

Masalani yechish uchun zamonaviy texnologiyalarni tanlash, tizim arxitekturasini va ma'lumotlar bazasini loyihalash, so'ngra server va klient qismlarini ishlab chiqish, joylashtirish va sinash talab etiladi. Bu vazifalar keyingi boblarda ketma-ket yoritiladi.

1-bob bo'yicha xulosa

Birinchi bobda messenjerlarning IRC va ICQ dan boshlab bugungi Telegram, WhatsApp va Signal kabi ilovalargacha bo'lgan rivojlanish yo'li ko'rib chiqildi. Mavjud messenjerlar funktsionallik, xavfsizlik modeli va ma'lumotlarni saqlash joyi bo'yicha taqqoslandi. Ularning barchasi xorijiy kompaniyalar tomonidan boshqarilishi va foydalanuvchi ma'lumotlarini mamlakatdan tashqarida saqlashi

aniqlandi. Bu holat “Shaxsga doir ma’lumotlar to’g’risida”gi Qonunning lokalizatsiya talablari va “Raqamli O‘zbekiston – 2030” strategiyasi maqsadlari nuqtai nazaridan milliy messenjer yaratish zaruriyatini asoslaydi.

Real vaqtda aloqa texnologiyalari tahlili asosida WebSocket va unga zaxira transportlarni avtomatik tanlaydigan SignalR kutubxonasi eng maqbul yechim sifatida tanlandi. Bob yakunida tizimning aktorlari, funksional va nofunksional talablari aniqlanib, masala qo’yildi.

II-BOB. “MILLIY CHAT” TIZIMINI LOYIHALASH

2.1. Texnologiyalarni tanlash va asoslash

Birinchi bobda aniqlangan talablar texnologiyalarni tanlashda quyidagi mezonlarni belgilab berdi: real vaqtda ikki tomonlama aloqani qo‘llab-quvvatlash; yuqori unumdorlik va ko‘p sonli bir vaqtdagi ulanishlarga chidamlilik; xavfsizlik vositalarining (autentifikatsiya, kriptografiya) tayyor va ishonchli bo‘lishi; ekotizimning yetukligi va uzoq muddatli qo‘llab-quvvatlanishi; ochiq kodli va bepul litsenziya; konteynerlarda joylashtirish qulayligi.

Server qismi.

Server qismi uchun Microsoft kompaniyasining **.NET 8** platformasi va **ASP.NET Core 8** veb-freymvorki tanlandi. .NET 8 uzoq muddatli qo‘llab-quvvatlanadigan (LTS) versiya bo‘lib, kross-platformali, ochiq kodli va Linux konteynerlarida ishlaydi. ASP.NET Core mustaqil sinovlarda eng tezkor veb-freymvorklar qatorida turadi, unda bog‘liqliklarni kiritish (Dependency Injection), konfiguratsiya, loglash va middleware konveyeri tayyor holda mavjud. C# tilining qat’iy tiplanishi, async/await modeli va LINQ so‘rovlari murakkab biznes-mantiqni ishonchli yozish imkonini beradi.

Muqobil sifatida Node.js (Express, NestJS) va Java (Spring Boot) ko‘rib chiqildi. Node.js real vaqt ilovalari uchun qulay, lekin dinamik tiplash katta loyihada xatolar ehtimolini oshiradi. Spring Boot esa kuchli, ammo ko‘proq resurs talab qiladi va konfiguratsiyasi murakkabroq. .NET tanlovining hal qiluvchi sababi — **SignalR** kutubxonasi platformaning tarkibiy qismi ekanligi: WebSocket, SSE va long polling transportlari, hablar, guruhlar va JWT autentifikatsiyasi qo‘shimcha kutubxonalarsiz ishlaydi.

Ma’lumotlar bazasi sifatida **PostgreSQL** tanlandi. U ochiq kodli, ACID tranzaksiyalarni to‘liq qo‘llab-quvvatlaydigan, sxemalar, izohlar (comment), qisman va murakkab indekslar, matnni registrga bog‘liq bo‘lmagan holda qidirish (ILIKE) kabi imkoniyatlarga ega obyekt-relatsion MBBT. Ma’lumotlar bazasi bilan ishlash **Entity Framework Core 8** ORM va Npgsql provayderi orqali amalga

oshriladi. EF Core LINQ so'rovlarini SQLga tarjima qiladi, sxema o'zgarishlarini migratsiyalar ko'rinishida boshqaradi va obyektlarni kuzatish (change tracking) mexanizmini taqdim etadi.

Klient qismi.

Foydalanuvchi interfeysi Google kompaniyasining **Angular 22** freymvorkida ishlab chiqildi. Angular to'liq freymvork bo'lib, marshrutlash, formalar, HTTP klient, bog'liqliklarni kiritish va sinov vositalarini bitta ekotizimda beradi. Loyihada Angularning zamonaviy imkoniyatlari qo'llanildi: mustaqil (standalone) komponentlar, holatni boshqarish uchun **signals** mexanizmi, shablonlardagi yangi boshqaruv bloklari (@if, @for) va server tomonida render qilish (SSR). Kod **TypeScript 6** tilida yozilgan, asinxron oqimlar **RxJS** orqali boshqariladi, real vaqt aloqasi esa rasmiy **@microsoft/signalr** kutubxonasi yordamida o'rnatiladi.

React va Vue.js muqobil variantlar sifatida ko'rib chiqildi. Ular kutubxona sifatida yengilroq, lekin marshrutlash, formalar va holatni boshqarish uchun uchinchi tomon kutubxonalarini tanlash va birlashtirishni talab qiladi. Angularning qat'iy tuzilmasi va TypeScriptni birinchi darajali qo'llab-quvvatlashi server qismidagi C# kodi bilan uslubiy uyg'unlikni ta'minlaydi.

Infratuzilma.

Server ilovasi **Docker** konteyneriga ko'p bosqichli (multi-stage) usulda yig'iladi: birinchi bosqichda .NET SDK obrazida loyiha kompilyatsiya qilinadi, ikkinchi bosqichda esa faqat kompilyatsiya natijasi yengil ASP.NET runtime obraziga ko'chiriladi. Konteyner **Render** bulutli platformasida ishga tushirilgan. Kod **Git** tizimida, GitHub'dagi ikkita repozitoriyda (server va klient) yuritiladi. Har bir funksiya alohida tarmoqda ishlab chiqilib, Pull Request orqali asosiy tarmoqqa birlashtiriladi. Tanlangan texnologiyalar 2.1-jadvalda umumlashtirilgan.

2.1-jadval

Tizimda qo'llanilgan texnologiyalar

Qatlam	Texnologiya	Versiya	Vazifasi
Server	.NET, ASP.NET Core	8.0 (LTS)	Web API, middleware, DI, konfiguratsiya

Qatlam	Texnologiya	Versiya	Vazifasi
Server	ASP.NET Core SignalR	8.0	Real vaqtda ikki tomonlama aloqa (hab)
Server	Entity Framework Core, Npgsql	8.0.8	ORM, LINQ so‘rovlari, migratsiyalar
Server	FluentValidation	11.11	Kiruvchi ma’lumotlarni validatsiya qilish
Server	System.IdentityModel.Tokens.Jwt	8.3.1	JWT access tokenlarni yaratish va tekshirish
Server	Google.Apis.Auth	1.68	Google ID tokenini tekshirish
Server	SixLabors.ImageSharp	3.1.12	Rasmlarni tekshirish va thumbnail yaratish
Server	StackExchange.Redis	2.8.16	Onlayn holatni taqsimlangan saqlash
Server	Swashbuckle (Swagger)	6.6.2	API hujjatlari va sinov interfeysi
Ma’lumotlar	PostgreSQL	16+	Relatsion ma’lumotlar bazasi
Klient	Angular, Angular SSR	22	SPA freymvork, server tomonida render
Klient	TypeScript, RxJS	6.0, 7.8	Tiplashtirilgan til, reaktiv oqimlar
Klient	@microsoft/signalr	10.0	Brauzerdagi SignalR klienti
Infratuzilma	Docker, Render, Git/GitHub	—	Konteynerlash, joylashtirish, versiyalar nazorati

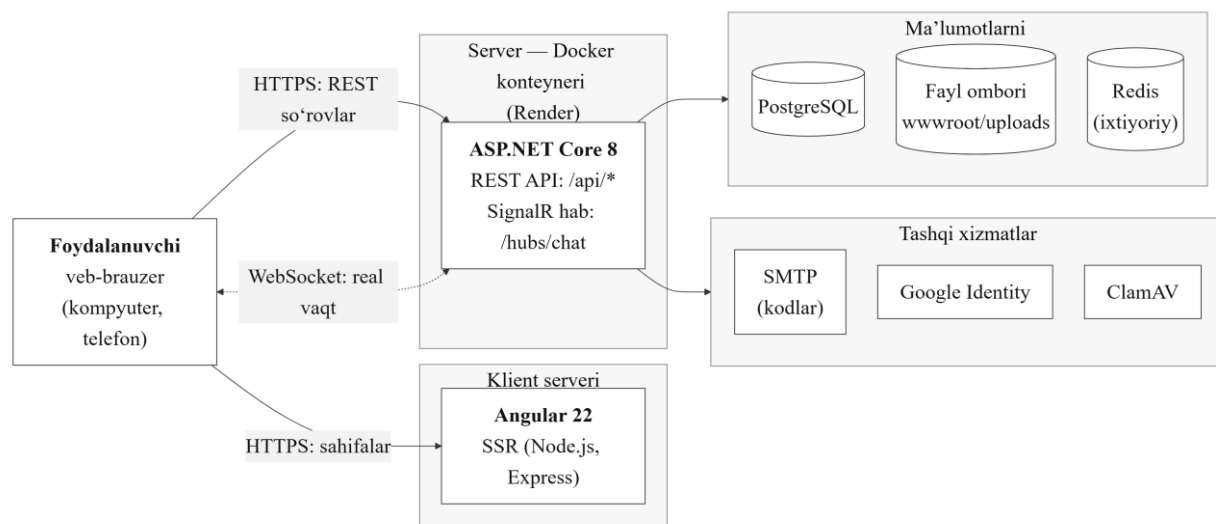
2.2. Tizim arxitekturası

Umumiy arxitektura.

“Milliy chat” klient-server arxitekturası asosida qurilgan. Foydalanuvchi brauzerida Angular ilovasi ishlaydi. U server bilan ikki kanal orqali aloqa qiladi: HTTPS ustidagi **REST API** (JSON formatidagi so‘rov va javoblar) va WebSocket ustidagi **SignalR** ulanishi. REST API holatni o‘zgartiruvchi barcha amallar uchun ishlatiladi: xabar yuborish, profilni tahrirlash, guruh yaratish va hokazo. SignalR ulanishi esa serverdan mijozga hodisalarni tezkor yetkazish uchun xizmat qiladi: yangi xabar, o‘qilganlik, “yozmoqda” holati, onlayn holat, guruh o‘zgarishlari.

Server qismi holatsiz (stateless) qilib loyihalangan: foydalanuvchi sessiyasi haqidagi barcha ma’lumotlar ma’lumotlar bazasida, autentifikatsiya esa har bir so‘rovdagi JWT tokenda saqlanadi. Shu sababli bir nechta server nusxasini ishga

tushirib, yuklamani ular o'rtasida taqsimlash mumkin. Bunda faqat SignalR ulanishlari holati va onlayn foydalanuvchilar ro'yxatini umumiy joyda (Redis) saqlash talab etiladi. Tizimning umumiy arxitekturasini va joylashtirish sxemasi 2.1-rasmda keltirilgan.



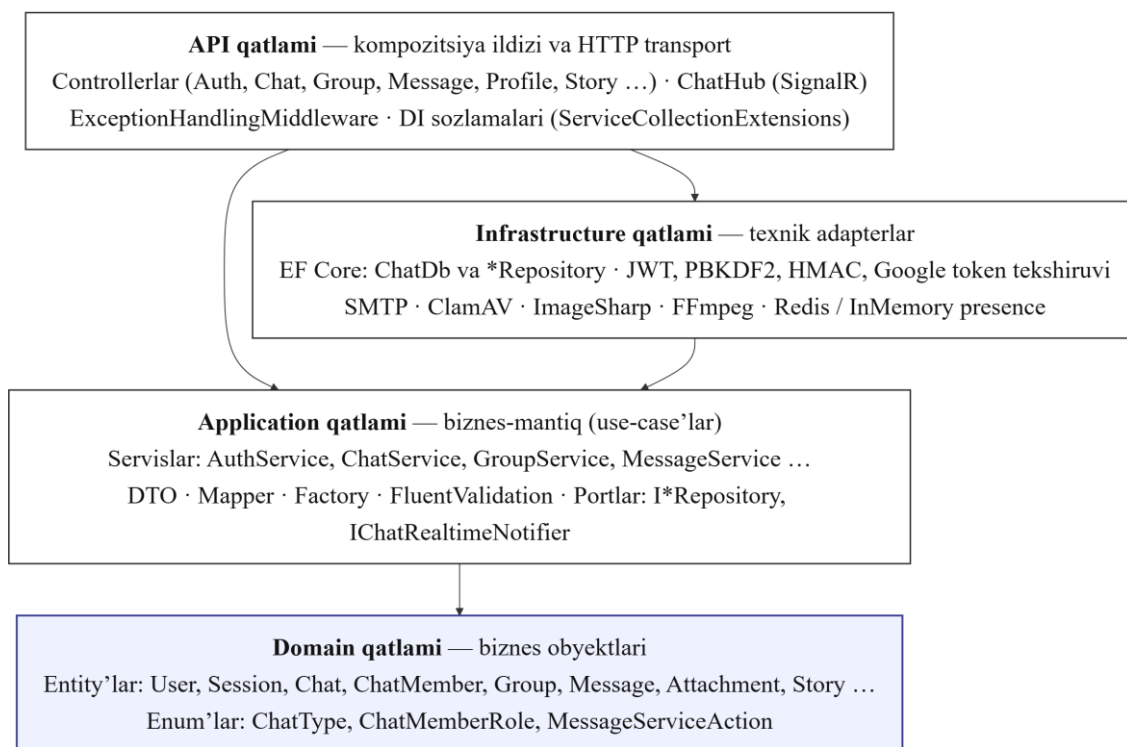
2.1-rasm. Tizimning umumiy arxitekturasini va joylashtirish sxemasi

Rasmdan ko'rinib turibdiki, tizim tashqi xizmatlar bilan ham ishlaydi. SMTP server bir martalik kodlarni elektron pochtaga yetkazadi. Google Identity xizmati Google hisobi orqali kirishda ID tokenini beradi, server esa uni Google'ning ochiq kalitlari yordamida tekshiradi. ClamAV antivirusi yuklanayotgan fayllarni tekshiradi. Yuklangan fayllar serverning fayl omborida (wwwroot/uploads katalogi) sana bo'yicha ajratilgan papkalarda saqlanadi, ma'lumotlar bazasida esa faqat ularning metama'lumotlari va yo'li yoziladi.

Server qismining qatlamli arxitekturasini.

Server kodi Robert Martin tomonidan taklif etilgan **toza arxitektura** (Clean Architecture) tamoyillari asosida tashkil etilgan. Uning asosiy g'oyasi — biznes-mantiqni texnik tafsilotlardan (ma'lumotlar bazasi, veb-freymvork, tashqi xizmatlar) ajratish. Bog'liqliklar faqat tashqi qatlamlardan ichki qatlamlarga yo'nalgan bo'lishi kerak. Natijada biznes-qoidalar ma'lumotlar bazasi yoki freymvork almashtirilganda o'zgarmaydi, ularni esa alohida sinash mumkin bo'ladi.

Yechim (solution) to'rtta loyihadan iborat. Ularning o'zaro bog'liqligi 2.2-rasmda ko'rsatilgan.



2.2-rasm. Server qismining Clean Architecture qatlamlari

- **Domain** — tizimning biznes obyektlari (entity): User, Session, Chat, ChatMember, Group, Message, Attachment, File, Photo, Story va boshqalar, shuningdek ChatType, ChatMemberRole, MessageServiceAction kabi sanab o'tiladigan turlar. Bu qatlam boshqa hech bir loyihaga bog'liq emas;
- **Application** — foydalanish holatlari (use-case) amalga oshiriladigan qatlam. Unda servislar (AuthService, ChatService, GroupService, MessageService, StoryService...), ma'lumot uzatish obyektlari (DTO), FluentValidation validatorlari, entity'larni DTOga o'giruvchi Mapper va yangi obyektlarni yaratuvchi Factory sinflari hamda tashqi dunyo bilan aloqa uchun “portlar” — interfeyslar (IMessageRepository, IChatRealtimeNotifier, IEmailSender, IFileStorage...) joylashgan. Application faqat Domain'ga bog'liq;

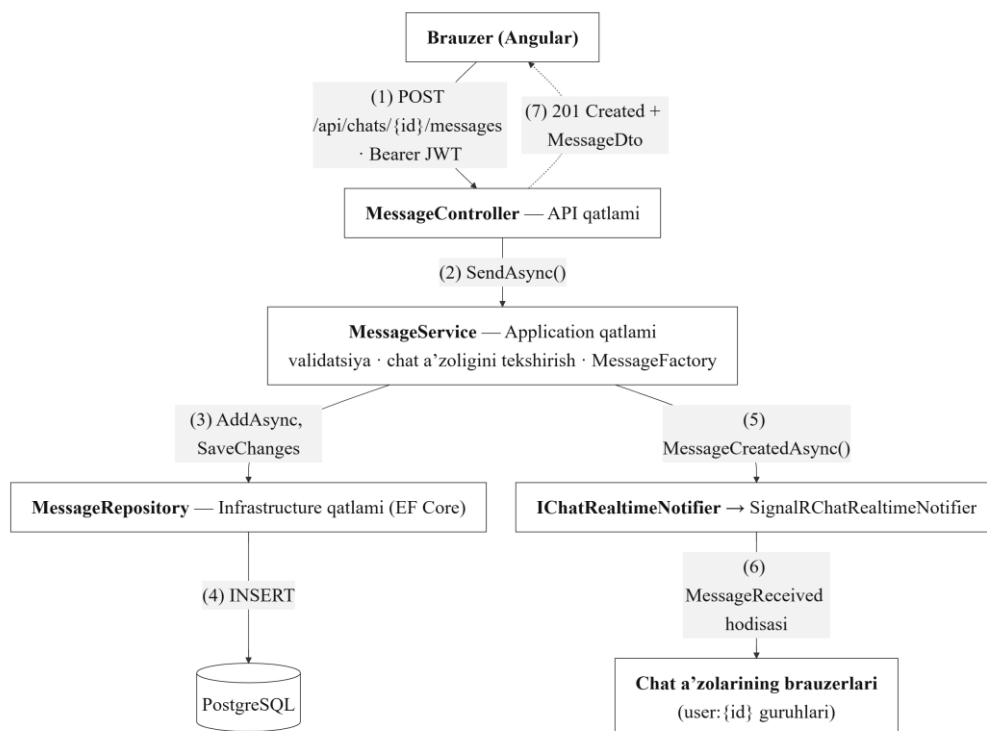
- **Infrastructure** — Application interfeyslarining texnik amalga oshirilishi: EF Core ma'lumotlar konteksti (ChatDb) va repozitoriylar, JWT tokenlar chiqaruvchisi, PBKDF2 va HMAC xeshlovchilari, Google token tekshiruvchisi, SMTP orqali xat yuboruvchi, ClamAV skaneri, ImageSharp va FFmpeg asosidagi media ishlov beruvchilar, onlayn holat kuzatuvchilari;
- **API** — kompozitsiya ildizi va HTTP transport qatlami: REST controllerlar, SignalR habi (ChatHub), xatolarni markazlashgan holda qayta ishlovchi middleware va barcha bog'liqliklarni ro'yxatga oluvchi kengaytma metodlar.

Application qatlami ma'lumotlar bazasi yoki SignalR haqida hech narsa bilmaydi. Masalan, xabar yuborilgandan so'ng MessageService faqat IChatRealtimeNotifier interfeysining MessageCreatedAsync metodini chaqiradi. Bu interfeysning SignalR orqali ishlaydigan amalga oshirilishi (SignalRChatRealtimeNotifier) API qatlamida joylashgan va bog'liqliklar konteynerida ro'yxatdan o'tkazilgan. Bu yondashuv "bog'liqliklarni teskari aylantirish" (Dependency Inversion) tamoyilining amaliy ko'rinishidir.

Application qatlami ichida kod texnik vazifalar bo'yicha emas, **funksiyalar (feature) bo'yicha** guruhlangan: Authentication, Chats, Groups, Messages, Contacts, Profiles, Stories, Users, Files. Har bir funksiya papkasi o'z servislari, DTOlari, interfeyslari, validatorlari, mapper va factory sinflarini o'z ichiga oladi. Bu tuzilma yangi funktsiyani qo'shishda o'zgarishlarni bitta joyda to'plash va kodni tushunishni osonlashtiradi. Loyihada qabul qilingan qoidalar (qatlam chegaralari, UTC vaqt, sahifalash, validatsiya, nomlash) repozitoriydagi AGENTS.md hujjatida qayd etilgan.

HTTP so'rov qatlamlar bo'ylab qanday o'tishi xabar yuborish misolida 2.3-rasmda ko'rsatilgan. Controller so'rovni qabul qilib, foydalanuvchi identifikatorini JWT tokenidan oladi va servisni chaqiradi. Servis ma'lumotlarni validatsiya qiladi, foydalanuvchining chat a'zosi ekanligini tekshiradi, Factory yordamida yangi Message obyektini yaratadi va repozitoriy orqali saqlaydi. Tranzaksiya

muvaffaqiyatli yakunlangach, real vaqt xabarnomasi chat a'zolariga yuboriladi. Bu tartib mijozlar hech qachon bazaga yozilmagan xabar haqida bildirishnoma olmasligini kafolatlaydi.



2.3-rasm. Xabar yuborish so'rovining qatlamlar bo'ylab o'tishi

Klient qismining arxitekturası.

Angular ilovasi ham qatlamlarga ajratilgan. **core** papkasida butun ilova uchun umumiy xizmatlar joylashgan: autentifikatsiya servisi (AuthService), har bir so'rovga token qo'shuvchi va muddati o'tgan tokenni avtomatik yangilovchi HTTP interceptor, himoyalangan sahifalarni qo'riqlovchi guard, mavzuni (yorug' va qorong'i) boshqaruvchi servis. **features** papkasida funksional modullar joylashgan: kirish sahifasi (auth) va chat moduli. Chat modulining data-access qatlamida ChatApiService (REST so'rovlari va ilova holati signallari) hamda ChatRealtimeService (SignalR ulanishi va hodisalar oqimi) servislari bor. Komponentlar faqat shu servislar bilan ishlaydi.

shared/ui papkasida loyihaning dizayn tizimi (design system) komponentlari joylashgan: tugma, ikonkali tugma, matn maydoni, checkbox, banner, tasdiqlash oynasi va boshqalar. Barcha ranglar CSS o'zgaruvchilari (design token) orqali

berilgan. Shu sababli yorug‘ va qorong‘i mavzular o‘rtasida o‘tish faqat o‘zgaruvchilar qiymatini almashtirish orqali amalga oshadi.

2.3. Ma’lumotlar bazasini loyihalash

Ma’lumotlar bazasi “kod birinchi” (code-first) yondashuvida loyihalandi: jadvallar tuzilmasi Domain qatlamidagi entity sinflari va Infrastructure qatlamidagi IEntityTypeConfiguration konfiguratsiyalari asosida EF Core tomonidan yaratiladi. Sxemadagi har bir o‘zgarish migratsiya ko‘rinishida saqlanadi va versiyalar nazorati tizimida kuzatiladi.

Jadvallar mantiqiy vazifasiga ko‘ra PostgreSQL’ning to‘qqizta sxemasiga (schema) ajratilgan. Bu yondashuv katta bazada yo‘nalishni osonlashtiradi, kelajakda esa turli xizmatlarga faqat kerakli sxemalarga kirish huquqini berish imkonini yaratadi. Jami 32 ta jadval loyihalangan. Ularning bir qismi (kanallar, qo‘ng‘iroqlar, stikerlar, botlar, papkalar, so‘rovnomalar, reaksiyalar) kelajakdagi funksiyalar uchun oldindan tayyorlangan. Sxemalar tarkibi 2.2-jadvalda keltirilgan.

2.2-jadval

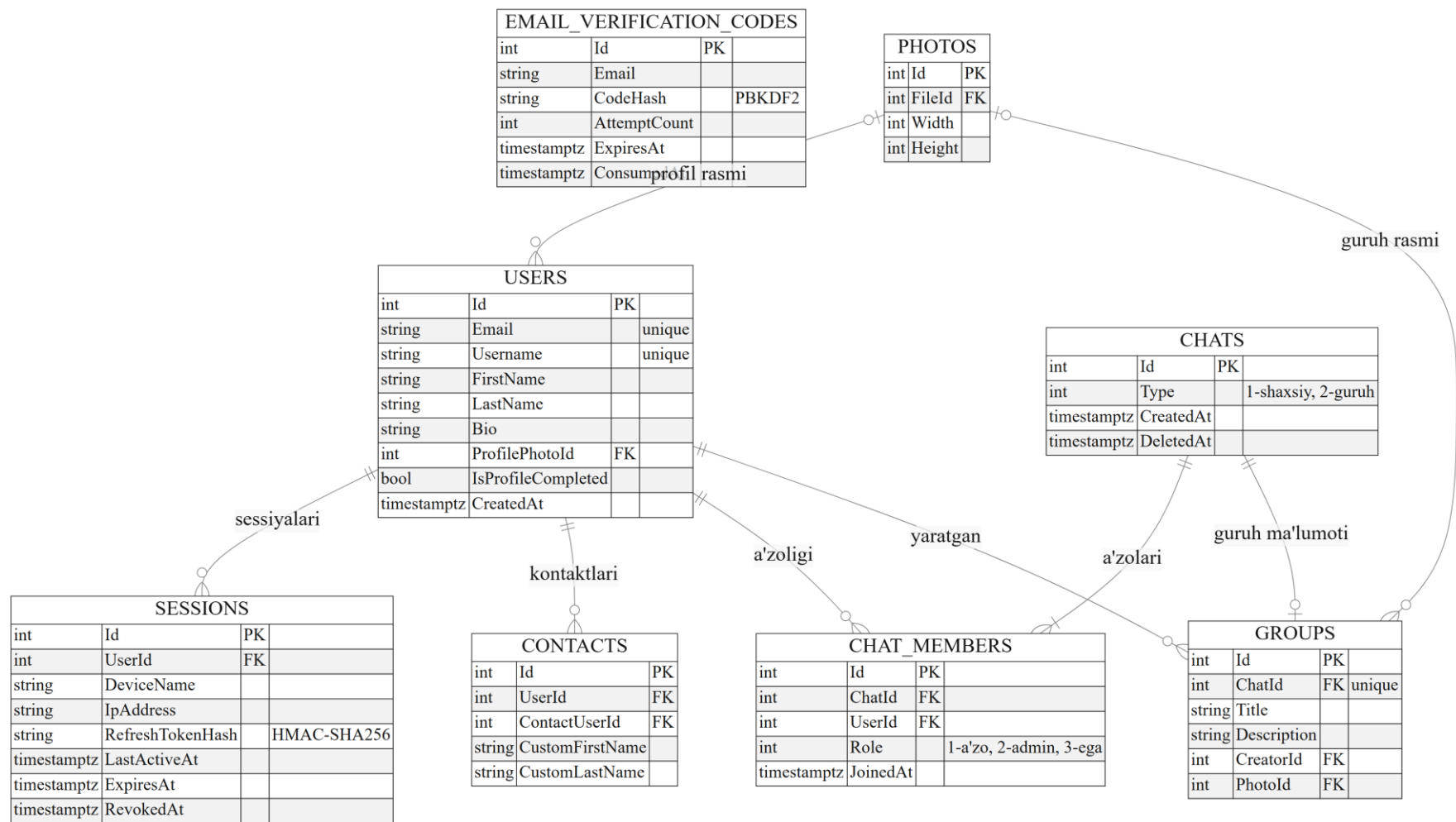
Ma’lumotlar bazasi sxemalari va jadvallari

Sxema	Jadvallar	Vazifasi
identity	users, contacts, blocked_users	Foydalanuvchilar va ular o‘rtasidagi aloqalar
security	sessions, email_verification_codes, two_factor_auth	Sessiyalar, bir martalik kodlar, qo‘shimcha himoya
chat	chats, chat_members, groups, channels, channel_subscribers, secret_chats	Chatlar, a‘zolik va rollar, guruh va kanallar
messaging	messages, attachments, message_views, reactions, polls, poll_options, poll_votes	Xabarlar, biriktirmalar, o‘qilganlik, reaksiya va so‘rovnomalar
storage	files, photos, stickers, sticker_sets	Fayllar metama’lumotlari, rasmlar, stikerlar
story	stories, story_views	Hikoyalar va ularni ko‘rishlar
personal	folders, folder_chats, saved_messages, subscriptions	Shaxsiy papkalar, saqlangan xabarlar, obunalar
call	calls, call_participants	Ovozli va video qo‘ng‘iroqlar (kelajak uchun)
bot	bots	Botlar platformasi (kelajak uchun)

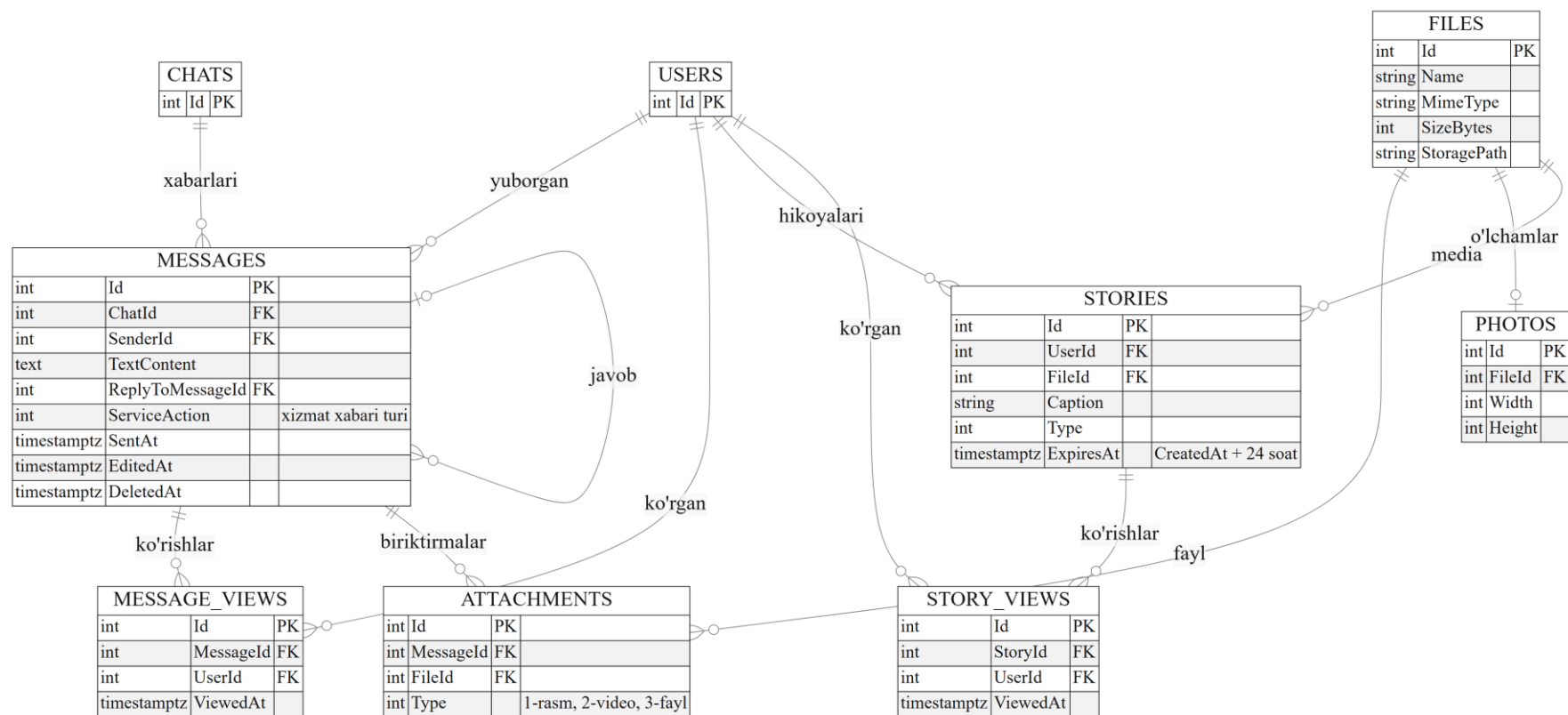
Tizimning joriy funksiyalarini ta'minlovchi asosiy jadvallar va ular o'rtasidagi bog'lanishlar ikkita ER-diagrammada ko'rsatilgan. 2.4-rasmda foydalanuvchilar, sessiyalar, chatlar va guruhlar, 2.5-rasmda esa xabarlar, fayllar va hikoyalar bilan bog'liq jadvallar tasvirlangan.

Modelning markazida **chats** jadvali turadi. Shaxsiy chat ham, guruh ham bitta chats yozuvi bilan ifodalanadi va ular Type maydoni (1 — shaxsiy, 2 — guruh) bilan farqlanadi. Chat ishtirokchilari **chat_members** jadvalida saqlanadi. Bu jadval users va chats o'rtasidagi “ko'pdan-ko'pga” bog'lanishni amalga oshiradi va har bir a'zoning rolini (1 — a'zo, 2 — administrator, 3 — ega) hamda qo'shilgan vaqtini saqlaydi. Guruhga xos ma'lumotlar (nomi, tavsifi, rasmi, yaratuvchisi) alohida **groups** jadvaliga chiqarilgan va chats bilan “birga-bir” bog'langan. Buni ChatId ustunidagi unique indeks ta'minlaydi. Bu yechim tufayli xabarlar, biriktirmalar, qidiruv va real vaqt mexanizmlari shaxsiy chatlar va guruhlar uchun bir xil ishlaydi.

users jadvalida Email va Username ustunlari unique indekslar bilan himoyalangan. Foydalanuvchi sessiyalari **sessions** jadvalida saqlanadi: qurilma nomi, IP-manzil, oxirgi faollik vaqti, amal qilish muddati, bekor qilingan vaqti (RevokedAt) va refresh tokenning xeshi. Refresh token xeshi ustunida ham unique indeks bor, bu token bo'yicha sessiyani tez topish imkonini beradi. Bir martalik kodlar **email_verification_codes** jadvalida faqat xesh ko'rinishida, urinishlar soni va amal qilish muddati bilan birga saqlanadi.



2.4-rasm. ER-diagramma: foydalanuvchilar, sessiyalar, chatlar va guruhlar



2.5-rasm. ER-diagramma: xabarlar, fayllar va hikoyalar

messages jadvali tizimdagi eng katta hajmli jadval hisoblanadi, shuning uchun uning tuzilmasiga alohida e'tibor berildi (2.3-jadval). Xabar boshqa xabarga javob bo'lishi mumkin, buning uchun jadval o'ziga o'zi ReplyToMessageId orqali bog'langan. Asl xabar o'chirilsa, bu maydon avtomatik NULL qiymatga o'tadi (ON DELETE SET NULL). ServiceAction maydoni oddiy xabarni guruhdagi xizmat xabaridan ("guruh yaratildi", "a'zo qo'shildi", "nom o'zgartirildi" va hokazo) farqlaydi.

2.3-jadval

"messaging.messages" jadvalining tuzilmasi

Ustun	Tur	Izoh
Id	integer, PK	Avtomatik oshuvchi identifikator, sahifalash kursori
ChatId	integer, FK → chats	Xabar tegishli chat (chat o'chirilsa, kaskad)
SenderId	integer, FK → users	Jo'natuvchi (foydalanuvchi o'chirilishi cheklangan)
TextContent	text, NULL	Xabar matni (faqat fayldan iborat xabarda bo'sh)
ReplyToMessageId	integer, FK, NULL	Javob berilgan xabar (ON DELETE SET NULL)
ServiceAction	integer, NULL	Xizmat xabari turi: 1 — guruh yaratildi, 2 — a'zolar qo'shildi, 3 — chiqarildi, 4 — chiqib ketdi, 5 — nom, 6 — rasm o'zgardi
SentAt	timestampz	Yuborilgan vaqt (UTC)
EditedAt	timestampz, NULL	Oxirgi tahrir vaqti
DeletedAt	timestampz, NULL	Yumshoq o'chirish belgisi

Loyihalash qarorlari.

- **Vaqt.** Barcha vaqt belgilari UTC'da saqlanadi va PostgreSQL'da "timestamp with time zone" turiga ega. Server kodida joriy vaqt TimeProvider abstraksiyasi orqali olinadi, bu esa vaqtga bog'liq mantiqni sinashni osonlashtiradi. Foydalanuvchining mahalliy vaqtiga o'girish faqat brauzerda amalga oshiriladi;
- **Yumshoq o'chirish (soft delete).** Chat yoki xabar o'chirilganda yozuv jismonan o'chirilmaydi, balki DeletedAt maydoniga vaqt yoziladi. EF Core'ning global so'rov filtrlari (HasQueryFilter) bunday yozuvlarni barcha so'rovlardan avtomatik chiqarib tashlaydi;

- **O‘qilganlik.** Xabar kim tomonidan va qachon o‘qilgani alohida message_views jadvalida saqlanadi. Bu yechim guruh chatlarida har bir a’zo uchun o‘qilmagan xabarlar sonini alohida hisoblash imkonini beradi;
- **Fayllar.** Fayl ma’lumotlar bazasida emas, diskda saqlanadi. files jadvalida faqat nomi, MIME turi, hajmi va saqlash yo‘li yoziladi, rasmlar uchun esa photos jadvalida o‘lchamlari qo‘shimcha saqlanadi. Bitta fayl xabarga biriktirma, profil rasmi, guruh rasmi yoki hikoya sifatida qayta ishlatilishi mumkin;
- **Hujjatlashtirilgan enum’lar.** Sanab o‘tiladigan turlar bazada butun son sifatida saqlanadi, har bir qiymatning ma’nosi esa ustun izohiga (masalan, “1 = Member, 2 = Admin, 3 = Creator”) avtomatik yoziladi;
- **Indeksrlar.** Tez-tez bajariladigan so‘rovlar uchun indekslar yaratilgan: users(Email), users(Username), sessions(RefreshTokenHash) — unique; sessions(UserId, RevokedAt, ExpiresAt), email_verification_codes(Email, Purpose, ExpiresAt) — tarkibiy; groups(ChatId) — unique.

Kursorli sahifalash.

Chatlar ro‘yxati va xabarlar tarixi doimiy o‘zgarib turadigan, tartiblangan ma’lumotlardir. Klassik “sahifa raqami” (OFFSET/LIMIT) usulida yangi xabar qo‘shilganda sahifalar siljiydi, natijada foydalanuvchi bir xabarni ikki marta ko‘radi yoki ayrim xabarlarni o‘tkazib yuboradi. Katta OFFSET qiymatlarida esa MBBT o‘tkazib yuboriladigan qatorlarni ham o‘qishga majbur bo‘ladi va so‘rov sekinlashadi.

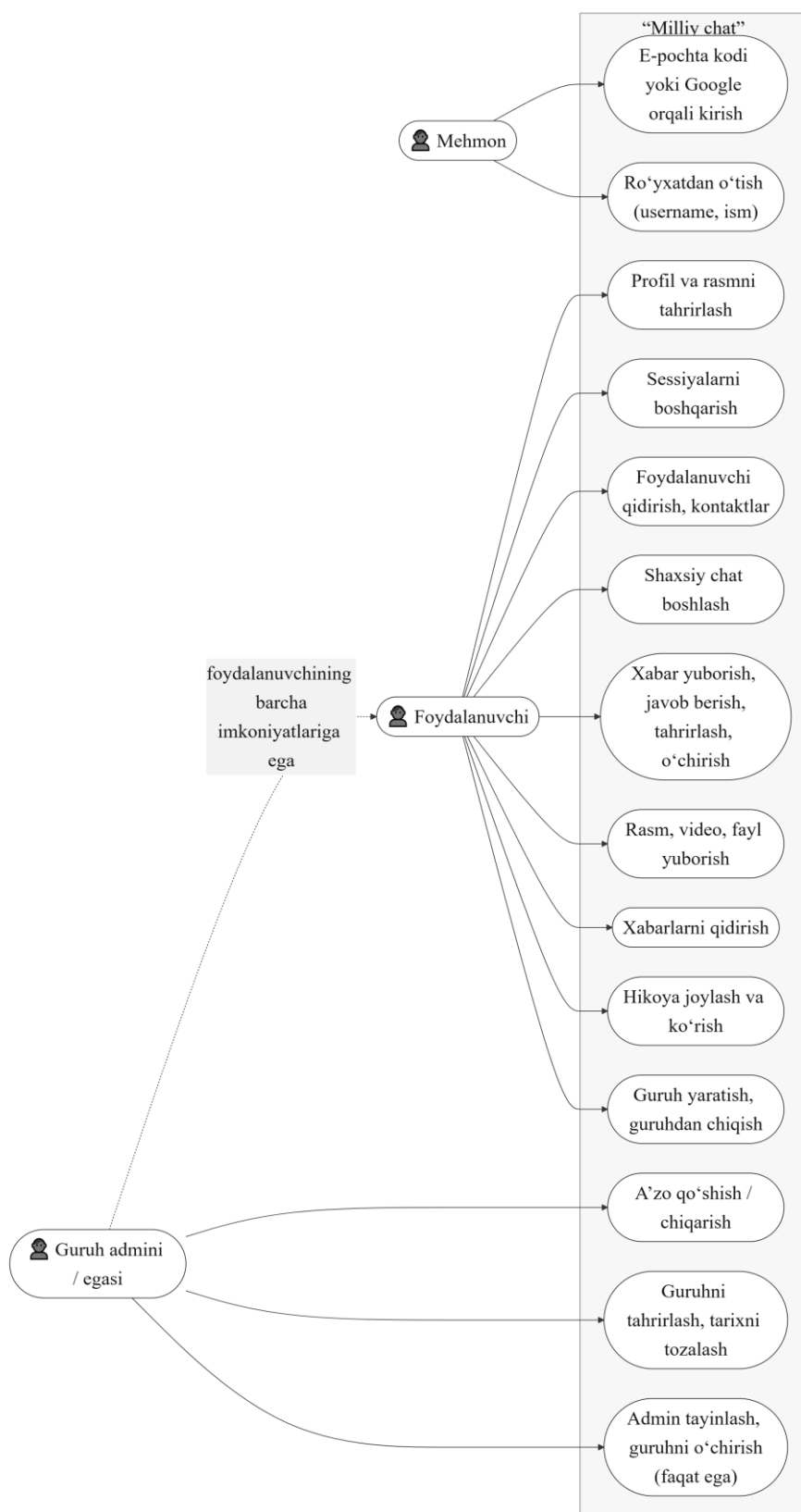
Shu sababli tizimda **kursorli (keyset) sahifalash** qo‘llanildi. Mijoz “beforeId” (shu identifikatordan oldingi yozuvlar) va “limit” (1 dan 100 gacha) parametrlarini yuboradi. Server yozuvlarni identifikator bo‘yicha kamayish tartibida saralaydi, limit+1 ta yozuvni o‘qiydi va javobda items (yozuvlar), nextCursor (keyingi so‘rov uchun kursor) va hasMore (yana yozuvlar bormi) maydonlarini qaytaradi. Bu so‘rov birlamchi kalit indeksi bo‘yicha bajariladi, shuning uchun uning tezligi tarix hajmiga bog‘liq emas. Chatlar ro‘yxatida kursor sifatida chatdagi

oxirgi xabarning identifikatori olinadi. Shu tufayli yangi xabar kelgan chat avtomatik ravishda ro'yxat boshiga chiqadi.

2.4. Tizimning UML modellari

Tizimning funksional imkoniyatlari va komponentlar o'rtasidagi o'zaro ta'sir UML (Unified Modeling Language) tilida modellashtirildi. Foydalanish holatlari diagrammasi (use-case) tizim nima qilishini, ketma-ketlik diagrammalari (sequence) esa asosiy jarayonlar qanday bajarilishini ko'rsatadi.

Tizimda uchta aktor mavjud. **Mehmon** — hali tizimga kirmagan shaxs: u faqat ro'yxatdan o'tishi yoki kirishi mumkin. **Foydalanuvchi** — autentifikatsiyadan o'tgan shaxs: u profilini boshqaradi, boshqa foydalanuvchilar bilan yozishadi, fayl va hikoyalar almashadi, guruh yaratadi. **Guruh administratori yoki egasi** foydalanuvchining barcha imkoniyatlariga ega, qo'shimcha ravishda esa guruh a'zolari va sozlamalarini boshqaradi. Foydalanish holatlari diagrammasi 2.6-rasmda keltirilgan.



2.6-rasm. Foydalanish holatlari (use-case) diagrammasi

Guruh chatlarida harakatlar rolga qarab cheklanadi. Ruxsatlar Telegram’dagi mantiqqa yaqin qilib loyihalangan va 2.4-jadvalda keltirilgan. Barcha ruxsatlar

server tomonida GroupService, ChatService va MessageService servislarida tekshiriladi. Klient interfeysi esa ruxsat berilmagan tugmalarni ko'rsatmaydi.

2.4-jadval

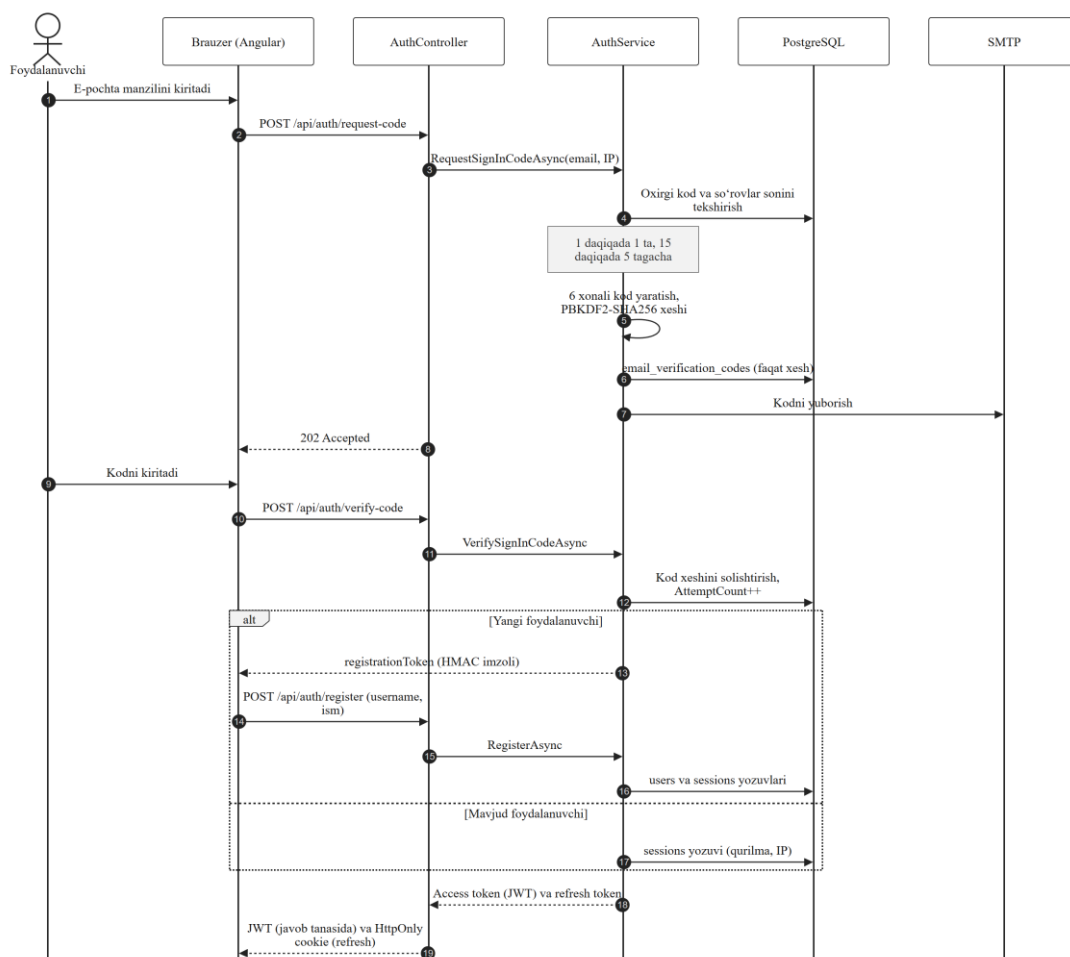
Guruh chatidagi rollar va ruxsatlar

Amal	A'zo	Administrator	Ega
Xabar yozish, o'qish, fayl yuborish	✓	✓	✓
A'zo qo'shish	—	✓	✓
A'zoni guruhdan chiqarish	—	Faqat oddiy a'zoni	✓ (hammani)
Administrator tayinlash yoki olib tashlash	—	—	✓
Guruh nomi, tavsifi va rasmini o'zgartirish	—	✓	✓
Chat tarixini tozalash	—	✓	✓
Guruhni butunlay o'chirish	—	—	✓
Guruhdan chiqish	✓	✓	✓ (egalik boshqa a'zoga o'tadi)

Elektron pochta orqali kirish jarayoni.

Tizimga kirish parolsiz amalga oshiriladi: foydalanuvchi elektron pochta manzilini kiritadi va unga yuborilgan 6 xonali kodni tasdiqlaydi (2.7-rasm). Server avval so'rovlar chastotasini tekshiradi: bitta manzilga daqiqada bir martadan ko'p kod yuborilmaydi, 15 daqiqalik oynada esa bitta manzil yoki bitta IP-manzildan beshtadan ortiq so'rov qabul qilinmaydi. Kod kriptografik tasodifiy sonlar generatori yordamida yaratiladi va bazaga faqat PBKDF2 xeshi ko'rinishida yoziladi. Kod 10 daqiqa amal qiladi, uni kiritishga esa beshtagacha urinish beriladi.

Agar bu manzil bilan birinchi marta kirilayotgan bo'lsa, server HMAC bilan imzolangan vaqtinchalik ro'yxatdan o'tish tokenini qaytaradi. Foydalanuvchi username va ismini kiritib, ro'yxatdan o'tishni yakunlaydi. Mavjud foydalanuvchi uchun esa darhol yangi sessiya yaratiladi. Har ikki holatda ham mijoz javob tanasida qisqa muddatli access tokeni, HttpOnly cookie'da esa refresh tokeni oladi.



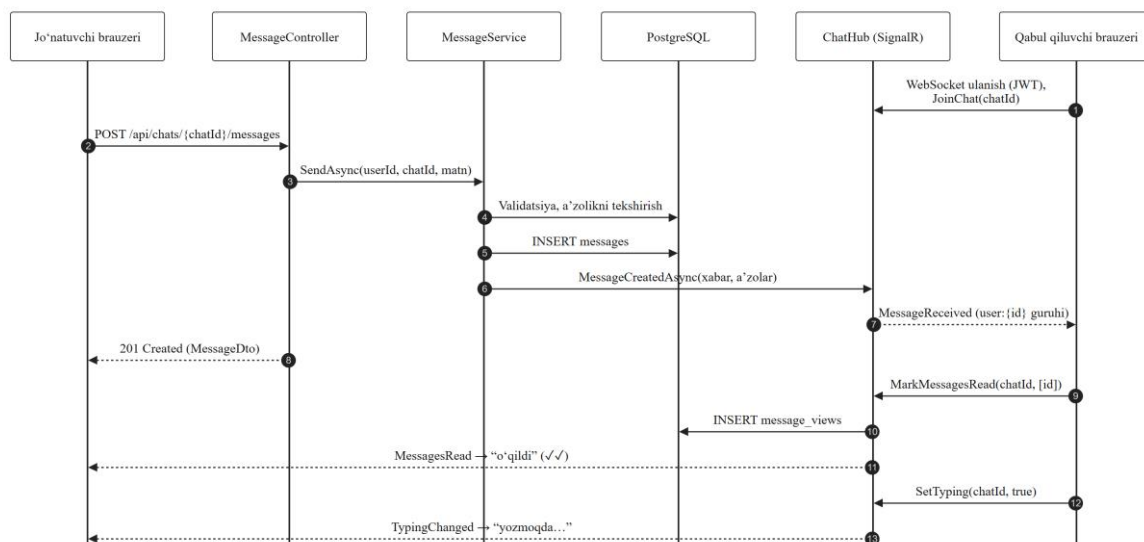
2.7-rasm. Elektron pochtaga yuborilgan kod orqali kirish ketma-ketlik diagrammasi

Real vaqtda xabar yuborish jarayoni.

Xabar almashish jarayoni 2.8-rasmda ko'rsatilgan. Brauzer tizimga kirgach, JWT token bilan SignalR habiga WebSocket ulanishini o'rnatadi. Server har bir ulanishni "user:{id}" nomli guruhga qo'shadi. Shu tufayli foydalanuvchining barcha qurilmalari va brauzer oynalari hodisalarni bir vaqtda oladi. Foydalanuvchi chatni ochganda ulanish qo'shimcha ravishda "chat:{id}" guruhiga qo'shiladi. Bu guruh faqat "yozmoqda" kabi tez-tez keladigan va faqat chatni ochib turganlar uchun dolzarb bo'lgan hodisalar uchun ishlatiladi.

Xabarning o'zi REST so'rovi orqali yuboriladi. Bu yondashuv xabarni saqlash va validatsiya qilishni oddiy HTTP semantikasi bilan (status kodlar, xatolar) bajarish imkonini beradi. Xabar bazaga yozilgach, u chat a'zolarining "user:{id}" guruhlariga MessageReceived hodisasi sifatida yuboriladi. Qabul qiluvchining brauzeri chat ochiq bo'lsa, darhol MarkMessagesRead hab metodini chaqiradi.

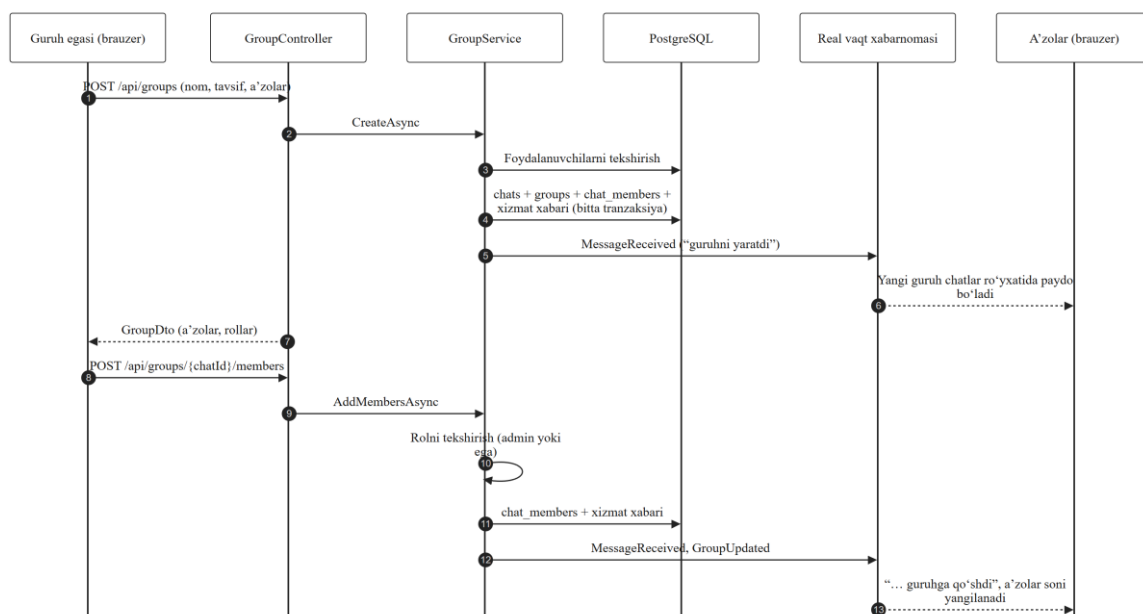
Server message_views jadvaliga yozuv qo‘shadi va jo‘natuvchiga MessagesRead hodisasini yuboradi. Natijada jo‘natuvchida xabar yonidagi bitta belgi ikki belgiga (✓✓) almashadi.



2.8-rasm. Real vaqtda xabar yuborish va o‘qilganlik belgisi ketma-ketlik diagrammasi

Guruh yaratish jarayoni.

Guruh yaratilganda chats, groups, chat_members jadvallariga yozuvlar va birinchi xizmat xabari (“... guruhini yaratdi”) bitta tranzaksiyada saqlanadi (2.9-rasm). Xizmat xabari muhim vazifani bajaradi: u barcha a’zolarga MessageReceived hodisasi sifatida yetib boradi va mijoz noma’lum chatdan xabar kelganda chatlar ro‘yxatini qayta yuklaydi. Shunday qilib, yangi guruh alohida hodisa turisiz, barcha a’zolarida bir zumda paydo bo‘ladi. A’zo qo‘shish, chiqarish, nom yoki rasmni o‘zgartirish ham xizmat xabarlari bilan qayd etiladi. Guruh ma’lumotlari o‘zgarganda esa qo‘shimcha GroupUpdated hodisasi yuboriladi.



2.9-rasm. Guruh yaratish va a'zo qo'shish ketma-ketlik diagrammasi

2.5. Xavfsizlikni loyihalash

Messenjer shaxsiy yozishmalarni saqlagani uchun xavfsizlik loyihalashning boshidan hisobga olindi. Himoya bir necha darajada tashkil etilgan: autentifikatsiya, sessiyalarni boshqarish, avtorizatsiya, kiruvchi ma'lumotlar va fayllarni tekshirish.

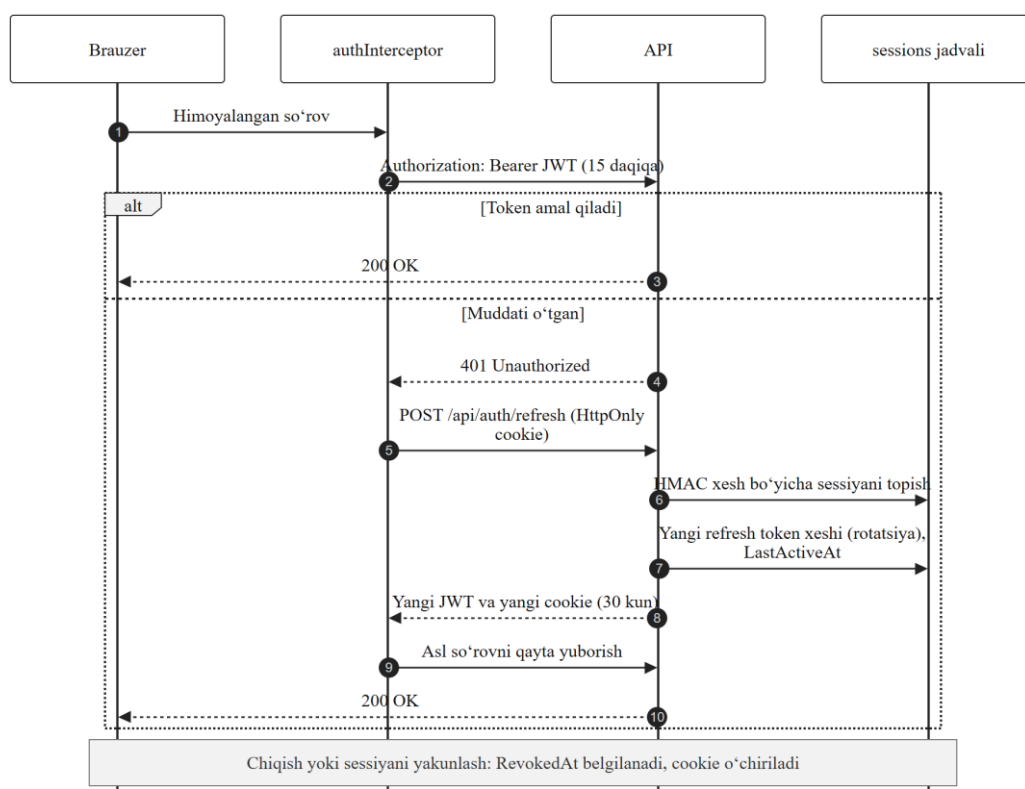
Autentifikatsiya va tokenlar.

Tizim parollardan foydalanmaydi, shuning uchun parollar bazasining o'g'irlanishi yoki zaif parollar muammosi yo'q. Bir martalik kodlar PBKDF2 algoritmi (HMAC-SHA256, 210 000 iteratsiya, 16 baytli tasodifiy tuz) bilan xeshlanadi. Kodni tekshirishda vaqtga bog'liq bo'lmagan (constant-time) taqqoslash qo'llanadi. Google orqali kirishda server brauzerdan kelgan ID tokenning imzosini, muddatini va auditoriyasini (loyihaning Client ID'si) Google.Apis.Auth kutubxonasi yordamida tekshiradi.

Muvaffaqiyatli kirishdan so'ng ikki turdagi token beriladi. **Access token** — HS256 algoritmi bilan imzolangan JWT. U foydalanuvchi identifikatori va sessiya identifikatorini (sid) o'z ichiga oladi va 15 daqiqa amal qiladi. U har bir API so'rovining Authorization sarlavhasida, SignalR ulanishida va media oqimlarida (video) esa access_token parametrda yuboriladi. **Refresh token** — 30 kun amal qiladigan kriptografik tasodifiy qator. U JavaScript'dan o'qib bo'lmaydigan

cookie'da (HttpOnly, ishlab chiqarish muhitida Secure, SameSite=Strict, faqat /api/auth yo'li uchun) saqlanadi. Ma'lumotlar bazasiga refresh tokenning o'zi emas, faqat HMAC-SHA256 xeshi yoziladi.

Access token muddati tugaganda klientdagi HTTP interceptor 401 javobini ushlaydi, /api/auth/refresh so'rovini yuboradi va asl so'rovni yangi token bilan takrorlaydi (2.10-rasm). Bir vaqtda bir nechta so'rov 401 olsa, ular bitta refresh so'rovini baham ko'radi. Har bir yangilashda refresh token almashtiriladi (rotatsiya). Shu sababli o'g'irlangan eski token qayta ishlatilsa, u endi hech qanday sessiyaga mos kelmaydi.



2.10-rasm. Access va refresh tokenlarning hayotiy sikli

Sessiyalarni boshqarish.

Har bir kirish alohida sessiya yaratadi. Unda qurilma nomi, tizim versiyasi, IP-manzil, yaratilgan va oxirgi faollik vaqti saqlanadi. Foydalanuvchi profil sozlamalarida barcha faol qurilmalarni ko'rishi, begona qurilmani chiqarib yuborishi yoki joriy qurilmadan tashqari barcha sessiyalarni bir vaqtda yakunlashi mumkin. Sessiya yakunlanganda uning RevokedAt maydoni to'ldiriladi va u bilan bog'liq

refresh token darhol yaroqsiz bo‘ladi. Access token esa muddati tugashi bilan, ya’ni ko‘pi bilan 15 daqiqada o‘z kuchini yo‘qotadi.

Avtorizatsiya.

Barcha controllerlar va SignalR habi [Authorize] atributi bilan himoyalangan. Ammo tokenning mavjudligi yetarli emas: har bir chat bilan bog‘liq amalda server foydalanuvchining shu chat a‘zosi ekanligini ma’lumotlar bazasidan tekshiradi. Masalan, xabarlarni o‘qish, yuborish, qidirish, media yuklash, JoinChat, SetTyping va MarkMessagesRead hab metodlari chaqirilganda shunday tekshiruv bajariladi. Foydalanuvchi faqat o‘z xabarini tahrirlashi yoki o‘chirishi mumkin. Guruhda esa 2.4-jadvaldagi rolga asoslangan qoidalar qo‘llanadi. Guruhdan chiqarilgan foydalanuvchi keyingi so‘rovdayoq chat ma’lumotlariga kirish huquqini yo‘qotadi.

Kiruvchi ma’lumotlar va fayllar.

API orqali keladigan har bir DTO uchun FluentValidation validatori yozilgan (matn uzunligi, identifikatorlar, sahifalash chegaralari va hokazo). Validatsiya ikki darajada bajariladi: ASP.NET Core’ning avtomatik tekshiruvi va servis ichidagi qayta tekshiruv. Kutilmagan xatolar ExceptionHandlingMiddleware tomonidan ushlanadi. Mijozga ichki tafsilotlarsiz, bir xil formatdagi javob qaytariladi, to‘liq ma’lumot esa server logiga yoziladi.

Yuklanadigan fayllar hajmi cheklangan: rasm uchun 10 MB, boshqa fayllar uchun 50 MB. Rasm faylning kengaytmasiga emas, uning haqiqiy tarkibiga qarab ImageSharp kutubxonasi yordamida tekshiriladi. Faqat JPEG, PNG va WebP formatlari qabul qilinadi, rasm qayta kodlanadi, ya’ni yashirin metama’lumotlar va zararli qo‘shimchalar olib tashlanadi. Katta rasmlar uchun 256 pikseli kichik nusxa (thumbnail) yaratiladi. Fayllar diskda tasodifiy (GUID) nom bilan saqlanadi. Chatga yuborilgan rasm, video va fayllar faqat shu chat a‘zosiga beriladi: server fayl biriktirilgan xabar chatining a‘zolari orasida so‘rovchi borligini tekshiradi. Sozlamalarda yoqilgan bo‘lsa, har bir fayl saqlashdan oldin ClamAV antivirusi bilan tekshiriladi.

Loyihalash jarayonida aniqlangan asosiy tahdidlar va ularga qarshi ko‘rilgan choralar 2.5-jadvalda umumlashtirilgan.

Asosiy tahdidlar va himoya choralari

Tahdid	Himoya choralari
Kodni tanlab topish (brute-force)	Kod 10 daqiqa amal qiladi, 5 tagacha urinish; kod so‘rash chastotasi e-pochta va IP bo‘yicha cheklangan
Ma’lumotlar bazasining sizib chiqishi	Kodlar PBKDF2, refresh tokenlar HMAC-SHA256 xeshi ko‘rinishida saqlanadi; parollar yo‘q
Tokenning o‘g‘irlanishi	Access token 15 daqiqa; refresh token HttpOnly cookie’da va har safar rotatsiya qilinadi; sessiyani masofadan yakunlash mumkin
CSRF (so‘rovni soxtalashtirish)	Refresh cookie SameSite=Strict va faqat /api/auth yo‘li uchun; boshqa so‘rovlar Authorization sarlavhasini talab qiladi
XSS (skript kiritish)	Angular shablonlari ma’lumotlarni avtomatik ekranlaydi; refresh token JavaScript’ga ko‘rinmaydi; access token qisqa muddatli
Begona chatga kirish	Har bir REST va hab chaqiruvida a’zolik tekshiruvi; guruhda rolga asoslangan ruxsatlar
Zararli yoki soxta fayl	Hajm cheklovleri, tarkib bo‘yicha format tekshiruvi, rasmni qayta kodlash, ClamAV skaneri
Tinglab olish (MITM)	Barcha trafik HTTPS/WSS orqali; ishlab chiqarish muhitida cookie faqat Secure rejimida

Kelajakda xavfsizlikni kuchaytirish uchun ikki bosqichli autentifikatsiya (bazada two_factor_auth jadvali tayyor), maxfiy chatlar uchun uchdan-uchgacha shifrlash (secret_chats jadvali) va foydalanuvchilarni bloklash (blocked_users jadvali) funksiyalarini qo‘shish rejalashtirilgan.

2-bob bo‘yicha xulosa

Ikkinchi bobda “Milliy chat” tizimi loyihalandi. Server qismi uchun .NET 8, ASP.NET Core, SignalR, Entity Framework Core va PostgreSQL, klient qismi uchun Angular 22 texnologiyalari tanlandi va asoslandi. Server kodi Domain, Application, Infrastructure va API qatlamlaridan iborat Clean Architecture asosida tashkil etildi. Bu biznes-mantiqni texnik tafsilotlardan ajratish va tizimni kengaytirishni osonlashtirish imkonini berdi.

Ma’lumotlar bazasi to‘qqizta sxemaga ajratilgan 32 ta jadvaldan iborat qilib loyihalandi. Shaxsiy va guruh chatlari yagona chats modeli orqali ifodalandi, katta hajmli ma’lumotlar uchun kursorli sahifalash tanlandi. Use-case va ketma-ketlik diagrammalari yordamida tizimga kirish, real vaqtda xabar almashish, guruh

yaratish va tokenlarni yangilash jarayonlari modellashtirildi. Xavfsizlik tizimi parolsiz autentifikatsiya, xeshlangan kod va tokenlar, qisqa muddatli JWT va rotatsiya qilinadigan refresh token, a'zolik va rolga asoslangan avtorizatsiya hamda fayllarni ko'p bosqichli tekshirish asosida qurildi. Keyingi bobda ushbu loyihaning dasturiy amalga oshirilishi ko'rib chiqiladi.

III-BOB. “MILLIY CHAT” DASTURINI ISHLAB CHIQISH

Ushbu bobda ikkinchi bobda loyihalangan tizimning dasturiy amalga oshirilishi yoritiladi. Server qismi 193 ta C# faylidan (migratsiyalarsiz 6,7 mingga yaqin qator) iborat. Klient qismi 27 ta Angular komponenti va 9 mingdan ortiq TypeScript, HTML va SCSS qatoridan tashkil topgan. Ishlab chiqish jarayonida server repozitoriysida 65 ta, klient repozitoriysida 24 ta commit qilindi. Quyida keltirilgan kod parchalari loyihaning haqiqiy manba kodidan olingan. Ayrim joylarda ular qisqartirilgan va bunday joylar “...” belgisi bilan ko‘rsatilgan.

3.1. Server qismini ishlab chiqish

Loyiha tuzilmasi va ilovani ishga tushirish.

Server yechimi (NationalChat.sln) ikkinchi bobda tavsiflangan to‘rtta loyihadan (Domain, Application, Infrastructure, API) va unit testlar loyihasidan (tests/NationalChat.Tests) iborat. Ilovaning kirish nuqtasi — API loyihasidagi Program.cs fayli (3.1-listing). U faqat ilovani yig‘ish va so‘rovlarni qayta ishlash konveyerini (middleware pipeline) sozlash bilan shug‘ullanadi. Barcha servislarni ro‘yxatdan o‘tkazish esa ServiceCollectionExtensions sinfidagi mavzuli metodlarga ajratilgan: API transporti, autentifikatsiya, ma’lumotlar bazasi, repozitoriylar, Application servislari, xavfsizlik va elektron pochta.

3.1-listing. Program.cs — so‘rovlarni qayta ishlash konveyeri

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddServices(builder.Configuration, builder.Environment);

var app = builder.Build();

app.UseMiddleware<ExceptionHandlerMiddleware>();
app.UseSwagger();
app.UseSwaggerUI();
app.UseHttpsRedirection();
app.UseCors("Client");
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
app.MapHub<ChatHub>("/hubs/chat");

app.Run();
```

Konveyerdagi tartib muhim. Xatolarni ushlovchi middleware eng birinchi turadi, shuning uchun u keyingi bosqichlarda yuz bergan har qanday istisnoni ushlay oladi. Autentifikatsiya avtorizatsiyadan oldin bajariladi. REST controllerlar va SignalR habi eng oxirida ulanadi.

REST API.

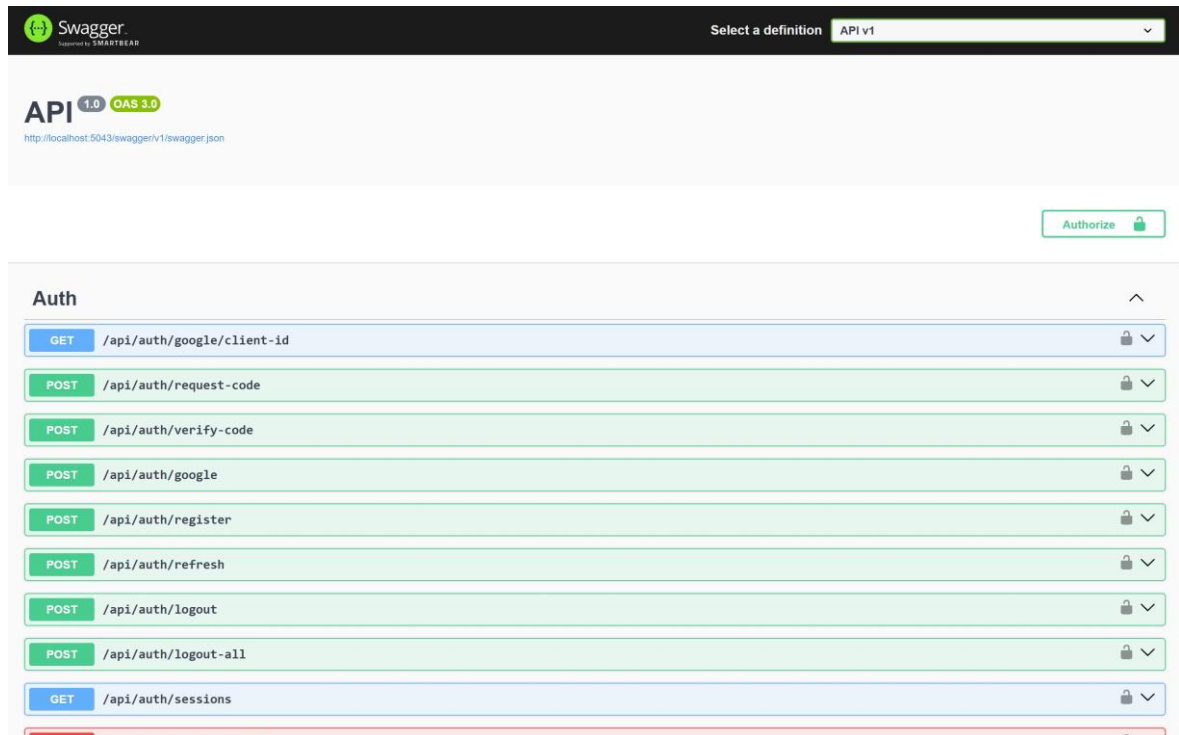
Server 9 ta controller orqali jami 56 ta HTTP endpointni taqdim etadi (3.1-jadval). Endpointlar resurslarga yo'naltirilgan (RESTful) tamoyil asosida nomlangan: resurslar ko'plik shaklida (chats, messages, groups), amal esa HTTP metodi (GET, POST, PUT, DELETE) orqali ifodalanadi. Masalan, "POST /api/chats/{chatId}/messages" chatga xabar qo'shadi, "PUT /api/groups/{chatId}/members/{userId}/role" esa guruh a'zosining rolini o'zgartiradi.

3.1-jadval

Server API controllerlari va endpointlari

Controller	Asosiy yo'l	Soni	Vazifasi
AuthController	/api/auth	11	Kod so'rash va tasdiqlash, Google, ro'yxatdan o'tish, tokenni yangilash, chiqish, sessiyalar
ProfileController	/api/users/me	5	Profilni olish va tahrirlash, profil rasmi
UserController	/api/users	1	Foydalanuvchilarni qidirish
ContactController	/api/contacts	3	Kontaktlar ro'yxati, qo'shish, o'chirish
ChatController	/api/chats	3	Chatlar ro'yxati, shaxsiy chat, chatni o'chirish
GroupController	/api/groups	8	Guruh yaratish, tahrirlash, rasm, a'zolar va rollar, chiqish
MessageController	/api/chats/{id}/messages	7	Xabarlar tarixi, yuborish, tahrirlash, o'chirish, qidiruv, kontekst
MessageAttachmentController	/api/chats/..., /api/media/...	10	Rasm, video, fayl yuklash va berish, ulashilgan media
StoryController	/api/stories	8	Hikoyalar lentasi, yaratish, media, ko'rishlar
Jami		56	

API avtomatik ravishda OpenAPI (Swagger) spetsifikatsiyasi bilan hujjatlashtiriladi. Swagger UI interfeysida barcha endpointlar, ularning parametrlari va javob sxemalarini ko‘rish hamda JWT token bilan to‘g‘ridan-to‘g‘ri sinab ko‘rish mumkin (3.1-rasm). Bu imkoniyat klient va server qismlarini parallel ishlab chiqishda qo‘l keldi.



3.1-rasm. Swagger UI: server API hujjatlari

Javob formati, validatsiya va xatolar.

Barcha endpointlar yagona formatdagi javob qaytaradi: succeeded (muvaffaqiyat belgisi), message (xabar) va data (natija). Buning uchun API qatlamidagi Result va Result<T> sinflaridan foydalaniladi. Mijoz tomonida javoblarni qayta ishlash bitta ApiResponse<T> interfeysi orqali amalga oshiriladi. Xato holatlarida foydalanuvchiga ko‘rsatiladigan xabarlar o‘zbek tilida va ayblovchi bo‘lmagan uslubda yozilgan, masalan: “Bu amal uchun ruxsat yo‘q.”, “Rasm hajmi 10 MB dan oshmasligi kerak.”.

Kiruvchi ma’lumotlar FluentValidation kutubxonasi yordamida tekshiriladi. Application qatlamida har bir so‘rov DTOsi uchun alohida validator sinfi yozilgan (masalan, CreateGroupRequestValidator, SendMessageRequestValidator).

Validator ikki joyda ishlaydi: ASP.NET Core'ning avtomatik validatsiyasi so'rov controllerga yetib kelishidan oldin, servis esa biznes-qoidalarini qo'llashdan oldin tekshiradi. Kutilmagan istisnalar ExceptionHandlingMiddleware tomonidan ushlanadi. U xatoni so'rov metodi, yo'li va TraceId bilan logga yozadi va mijozga 500 status kodli, ichki tafsilotlarsiz standart javob qaytaradi.

Kursorli sahifalashni amalga oshirish.

Kursorli sahifalash barcha repozitoriylar uchun umumiy bo'lgan IQueryable kengaytma metodi sifatida yozilgan (3.2-listing). Metod identifikatorni tanlovchi ifodani (Expression) qabul qiladi, "Id < beforeId" shartini ifodalar daraxti orqali dinamik quradi va limitdan bittaga ko'p yozuv o'qiydi. Qo'shimcha yozuv mavjudligi hasMore qiymatini aniqlaydi, alohida COUNT so'roviga esa hojat qolmaydi.

3.2-listing. Kursorli sahifalash (QueryableExtensions)

```
public static async Task<CursorPagedResponse<TResult>>
ToCursorPagedResponseAsync<TEntity, TResult>(
    this IQueryable<TEntity> query, CursorPagingRequest pagination,
    Expression<Func<TEntity, int>> idSelector, Expression<Func<TEntity, TResult>>
resultSelector,
    Func<TResult, int> cursorSelector, CancellationToken cancellationToken = default)
{
    if (pagination.BeforeId is not null)
    {
        var beforeId = Expression.Lambda<Func<TEntity, bool>>(
            Expression.LessThan(idSelector.Body,
            Expression.Constant(pagination.BeforeId.Value)),
            idSelector.Parameters);
        query = query.Where(beforeId);
    }

    var results = await query.OrderByDescending(idSelector)
        .Take(pagination.Limit + 1)
        .Select(resultSelector)
        .ToListAsync(cancellationToken);

    var hasMore = results.Count > pagination.Limit;
    IReadOnlyList<TResult> items = hasMore ? results.Take(pagination.Limit).ToArray()
: results;
    int? nextCursor = hasMore ? cursorSelector(items[^1]) : null;
    return new(items, nextCursor, hasMore);
}
```

Natija to'g'ridan-to'g'ri DTOga proyeksiya qilinadi (Select). Shu sababli EF Core ma'lumotlar bazasidan faqat kerakli ustunlarni o'qiydi va entity obyektlarini kuzatishga resurs sarflamaydi. Proyeksiya ifodalari Application qatlamidagi Mapper

sinflarida (ChatListMapper.Projection, MessageMapper.Projection) saqlanadi. Masalan, chatlar ro'yxati so'rovi bitta SQL so'rovida har bir chat uchun oxirgi xabarni, suhbatdosh ma'lumotlarini, guruh nomi va a'zolar sonini hamda o'qilmagan xabarlar sonini hisoblaydi.

3.2. Real vaqtda xabar almashishni amalga oshirish

Real vaqt aloqasi ChatHub SignalR habi va IChatRealtimeNotifier interfeysining SignalR orqali ishlaydigan amalga oshirilishi yordamida quriladi. Hab mijoz chaqira oladigan to'rtta metodni taqdim etadi. Server esa mijozlarga o'nta turdagi hodisa yuboradi (3.2-jadval).

3.2-jadval

SignalR habi metodlari va hodisalari

Nomi	Yo'nalish	Vazifasi
JoinChat(chatId)	Mijoz → server	Ochilgan chatning “chat: {id}” guruhiga qo'shish (a'zolik tekshiriladi)
LeaveChat(chatId)	Mijoz → server	Chat guruhidan chiqish
SetTyping(chatId, isTyping)	Mijoz → server	“Yozmoqda” holatini chatdagi boshqalarga uzatish
MarkMessagesRead(chatId, ids)	Mijoz → server	Xabarlarini o'qilgan deb belgilash (100 tagacha)
MessageReceived	Server → mijoz	Yangi xabar yoki xizmat xabari
MessageUpdated / MessageDeleted	Server → mijoz	Xabar tahrirlandi yoki o'chirildi
MessagesRead	Server → mijoz	Jo'natuvchiga xabarlar o'qilgani haqida (✓✓)
TypingChanged	Server → mijoz	Suhbatdosh yozmoqda yoki to'xtadi
ChatCleared / ChatDeleted	Server → mijoz	Chat tarixi tozalandi yoki chat o'chirildi (a'zolikdan chiqarilganda ham)
GroupUpdated	Server → mijoz	Guruh nomi, rasmi, a'zolari yoki rollari o'zgardi
UserPresenceChanged	Server → mijoz	Foydalanuvchi onlayn bo'ldi yoki oflayn holatga o'tdi
ProfilePhotoUpdated	Server → mijoz	Foydalanuvchi profil rasmini yangiladi

Hab metodlari ham REST endpointlari kabi himoyalangan: har bir chaqiruvda foydalanuvchining shu chat a'zosi ekanligi tekshiriladi (3.3-listing). Bu tekshiruv

bo‘lmasa, xabarni boshqa foydalanuvchi chat identifikatorini taxmin qilib, begona chatdagi “yozmoqda” hodisalarini tinglashi yoki xabarlarini o‘qilgan deb belgilashi mumkin bo‘lardi.

3.3-listing. ChatHub: o‘qilganlik belgisini qayd etish va a’zolikni tekshirish

```
public async Task MarkMessagesRead(int chatId, IReadOnlyCollection<int> messageIds)
{
    await EnsureChatMembershipAsync(chatId);
    var ids = messageIds.Where(id => id > 0).Distinct().Take(100).ToArray();
    if (ids.Length == 0) return;

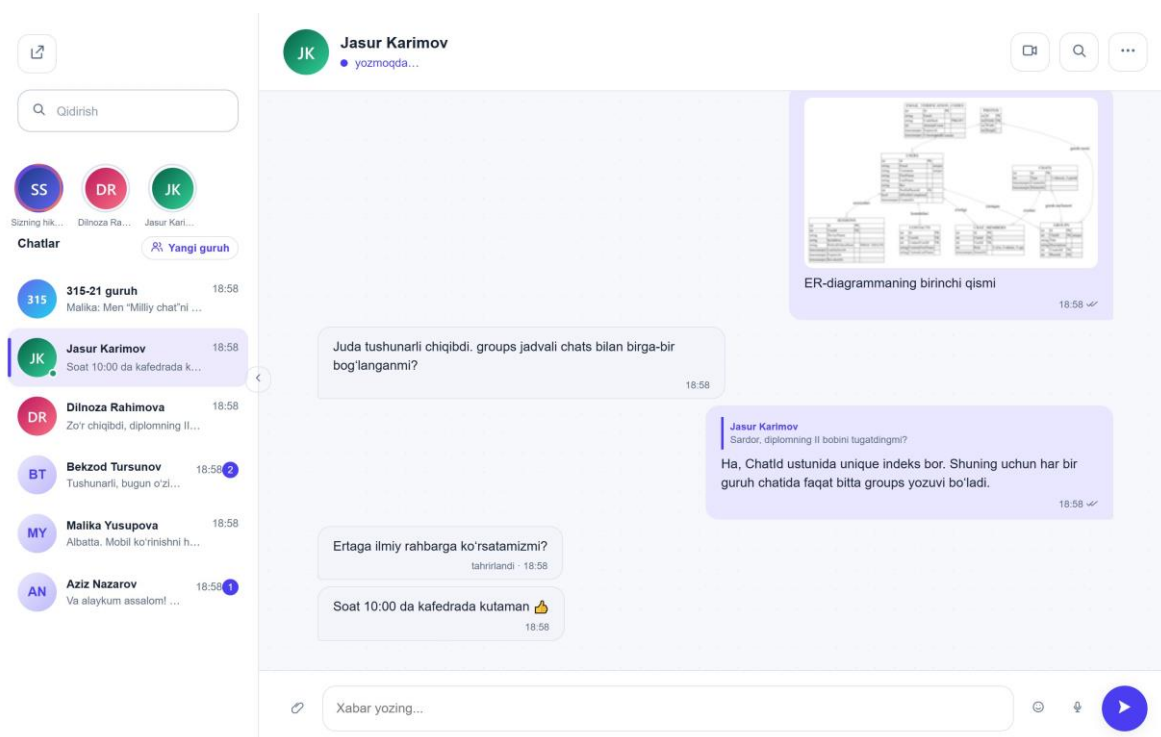
    var readerUserId = GetCurrentUser();
    await messageRepository.MarkAsReadAsync(chatId, readerUserId, ids,
        timeProvider.GetUtcNow().UtcDateTime, Context.ConnectionAborted);
    await realtimeNotifier.MessagesReadAsync(chatId, readerUserId, ids,
        await messageRepository.GetMemberUserIdsAsync(chatId,
Context.ConnectionAborted),
        Context.ConnectionAborted);
}

private async Task EnsureChatMembershipAsync(int chatId)
{
    if (!await messageRepository.IsChatMemberAsync(chatId, GetCurrentUser(),
Context.ConnectionAborted))
        throw new HubException("Bu chatga kirish huquqi yo'q.");
}
```

Onlayn holatni kuzatish IPresenceTracker interfeysi orqali abstraksiyalangan. InMemoryPresenceTracker har bir foydalanuvchi uchun faol ulanishlar to‘plamini ConcurrentDictionary’da saqlaydi. RedisPresenceTracker esa xuddi shu ma’lumotni Redis to‘plamlarida (SET) saqlaydi va bir nechta server nusxasi ishlaganda qo‘llanadi. Foydalanuvchining birinchi ulanishi paydo bo‘lganda u bilan umumiy chati bor foydalanuvchilarga “onlayn” hodisasi yuboriladi. Oxirgi ulanish uzilganda esa server 8 soniya kutadi (grace period). Sahifani yangilash yoki qisqa tarmoq uzilishida foydalanuvchi qayta ulanib ulguradi, natijada suhbatdoshlarda “oflayn” holati bir zumga ham ko‘rinmaydi. Shundan so‘ng sessiyaning oxirgi faollik vaqti yangilanadi va “oxirgi marta ... onlayn edi” ma’lumoti hisoblanadi.

Klient tomonida ChatRealtimeService servisi SignalR ulanishini boshqaradi. Ulanish avtomatik qayta ulanish siyosati (0, 2, 5 va 10 soniyadan keyin urinish) bilan yaratiladi. Qayta ulanilgach, ochiq turgan chatga JoinChat avtomatik qayta chaqiriladi. Kiruvchi hodisalar RxJS Subject’lari orqali ChatApiService’ga uzatiladi va u ilova holatini (chatlar ro‘yxati, xabarlar, “yozmoqda” holati) yangilaydi. 3.2-

rasmda suhbatdosh javob yozayotgan paytdagi shaxsiy chat ko‘rsatilgan: sarlavhada “yozmoqda...” yozuvi, jo‘natilgan xabarlar yonida esa o‘qilganlik belgisi ko‘rinib turibdi.



3.2-rasm. Shaxsiy chat: real vaqtda “yozmoqda” holati va o‘qilganlik belgilari

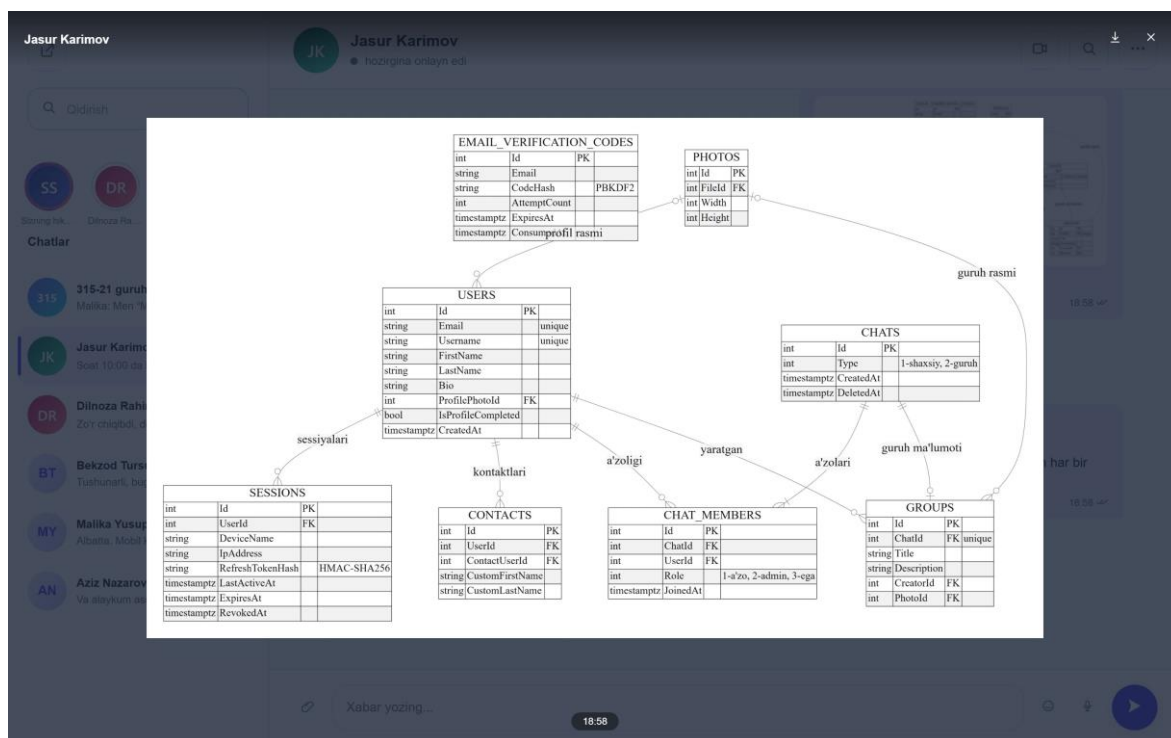
3.3. Media fayllar bilan ishlash

Rasm, video va ixtiyoriy fayllar multipart/form-data formatida yuklanadi. Barcha turdagi fayllar bir xil konveyerdan o‘tadi. Umumiy qadamlar Application qatlamidagi FileService servisinde, texnik amallar esa Infrastructure qatlamidagi LocalImageStorage sinfida amalga oshirilgan:

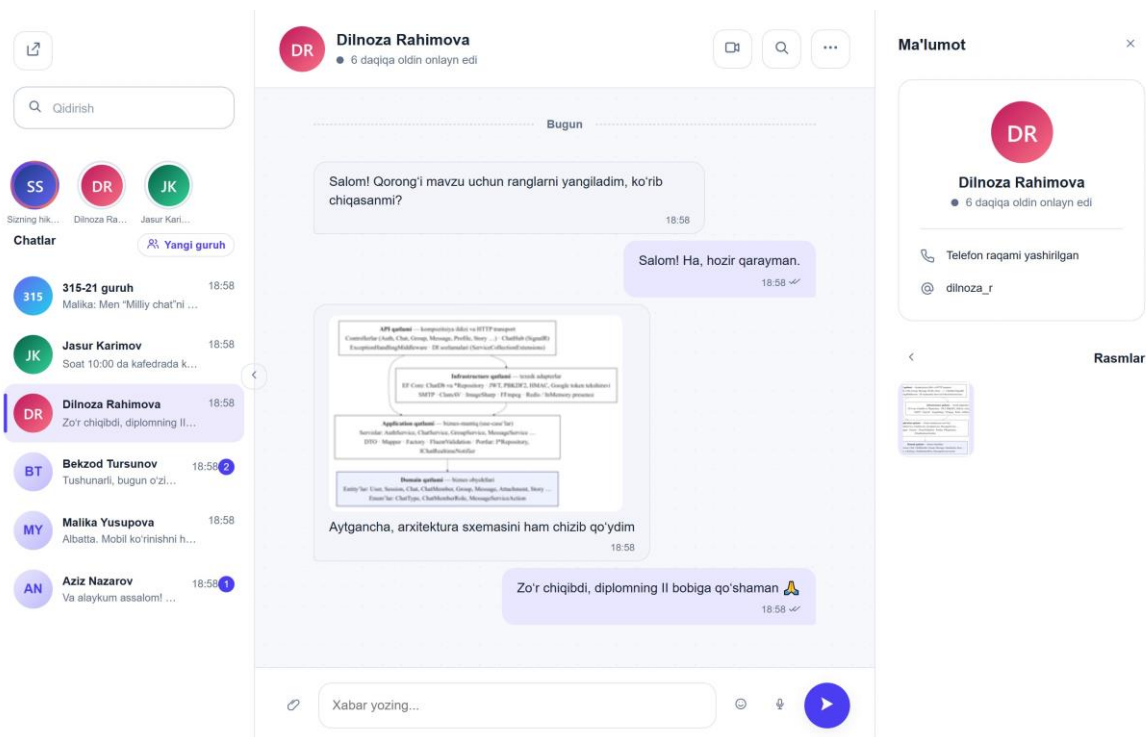
1. fayl tanlangani va hajmi tekshiriladi (rasm — 10 MB, fayl — 50 MB, xabardagi video — 200 MB gacha);
2. fayl ClamAV antivirusi bilan tekshiriladi (sozlamalarda yoqilgan bo‘lsa);
3. rasm formati kengaytma bo‘yicha emas, tarkib bo‘yicha aniqlanadi (JPEG, PNG, WebP), rasm 90% sifat bilan qayta kodlanadi va 256 pikseli kichik nusxa (thumbnail) yaratiladi. Video uchun esa ffprobe yordamida o‘lchamlar aniqlanadi va ffmpeg yordamida birinchi kadrda muqova rasm olinadi;

4. fayl diskka yil/oy bo'yicha ajratilgan papkaga GUID nom bilan yoziladi;
5. ma'lumotlar bazasiga files va photos yozuvlari, xabar va biriktirma (attachment) qo'shiladi;
6. chat a'zolariga MessageReceived hodisasi yuboriladi.

Trafikni tejash uchun chatda avval faqat rasmning o'lchamlari va hajmi ko'rsatiladi. Rasmning kichik nusxasi foydalanuvchi so'raganda yuklanadi, to'liq o'lchamdagi asl nusxa esa ko'rish oynasida ochiladi (3.3-rasm). Videolar HTTP Range so'rovlari orqali oqim (streaming) ko'rinishida beriladi, shuning uchun videoni to'liq yuklamasdan ham istalgan joyidan ko'rish mumkin. Chatdagi barcha media fayllar ma'lumot panelida turlari bo'yicha (rasmlar, videolar, fayllar) guruhlab ko'rsatiladi (3.4-rasm).



3.3-rasm. Rasmni to'liq o'lchamda ko'rish oynasi



3.4-rasm. Chatdagi ulashilgan media paneli

3.4. Guruh chatlari va hikoyalar

Guruh chatlari GroupService servisinde amalga oshirilgan. Servis har bir amaldan oldin foydalanuvchining guruhdagi rolini aniqlaydi va 2.4-jadvaldagi ruxsatlarni qoʻllaydi. Oʻzgarishlar esa xizmat xabarlarini va real vaqt hodisalari bilan birga saqlanadi. Eng murakkab holatlardan biri — guruh egasining guruhdan chiqishi (3.4-listing). Bu holatda guruh egasiz qolmasligi uchun egalik eng oldin tayinlangan administratorga, administratorlar boʻlmasa esa guruhga eng oldin qoʻshilgan aʼzoga oʻtadi. Guruhda hech kim qolmasa, u yumshoq oʻchiriladi.

3.4-listing. GroupService.LeaveAsync: guruhdan chiqish va egalikni oʻtkazish (qisqartirilgan)

```
var now = timeProvider.GetUtcNow().UtcDateTime;
RemoveMember(group, leaver);

if (group.Chat.Members.Count == 0)
{
    group.Chat.DeletedAt = now;
    await repository.SaveChangesAsync(cancellationToken);
    await realtimeNotifier.ChatDeletedAsync(chatId, [currentUserId],
    cancellationToken);
    return new(true, null);
}

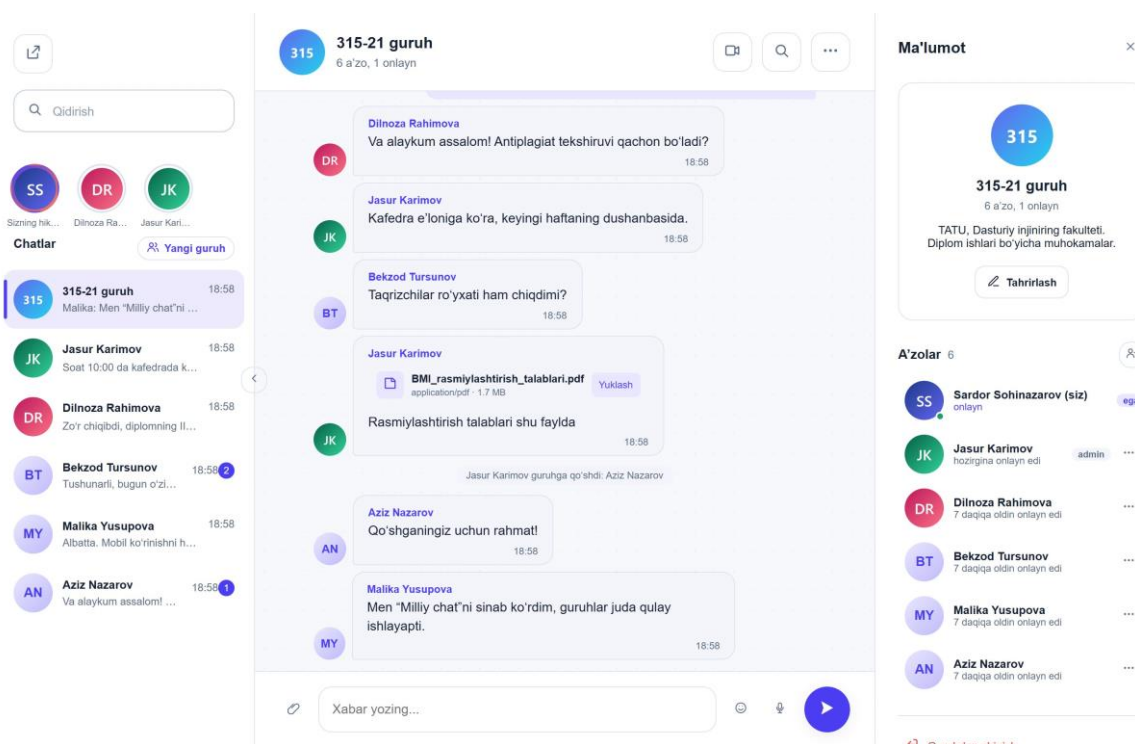
if (leaver.Role == ChatMemberRole.Creator)
{

```

```
// Ownership passes to the longest-serving admin, or to the longest-serving
member.
var successor = group.Chat.Members
    .OrderByDescending(member => member.Role == ChatMemberRole.Admin)
    .ThenBy(member => member.JoinedAt)
    .ThenBy(member => member.Id)
    .First();
successor.Role = ChatMemberRole.Creator;
group.CreatorId = successor.UserId;
}

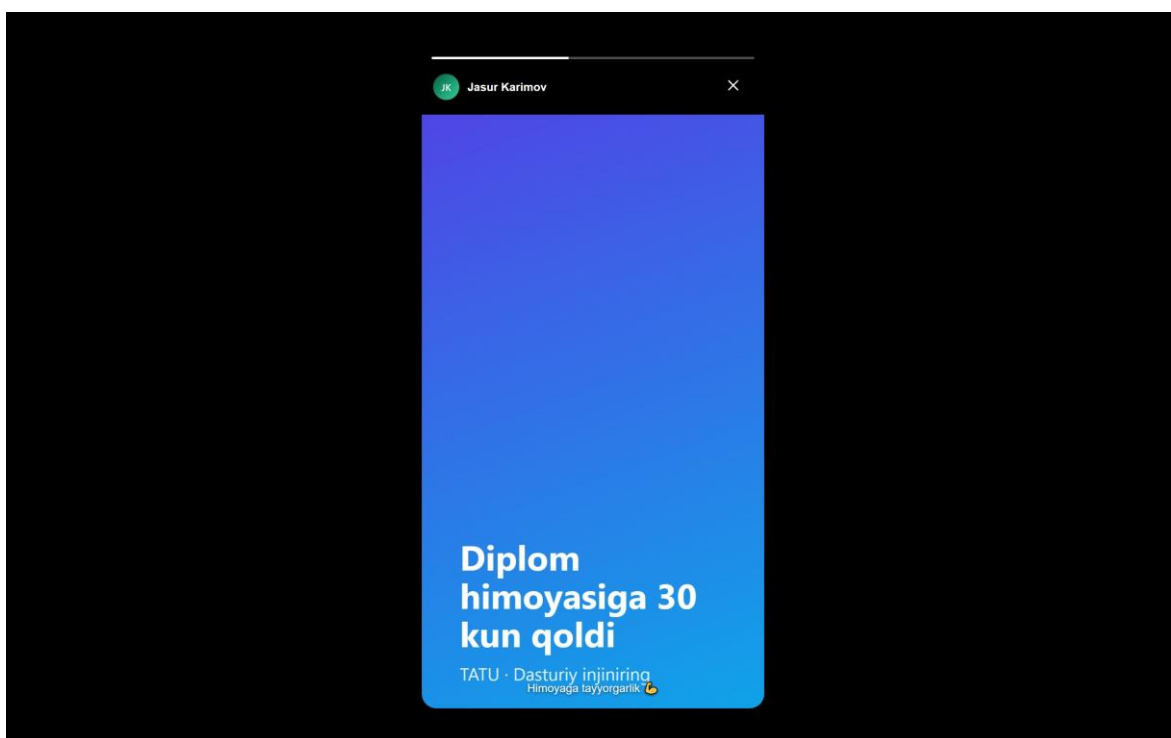
var serviceMessage = AddServiceMessage(group, leaver.User,
MessageServiceAction.MemberLeft, ...);
await repository.SaveChangesAsync(cancellationToken);
```

Xizmat xabarlarining matni klient tomonida xabar turi (ServiceAction) va unda saqlangan ism yoki nom asosida yig‘iladi. Masalan, “Jasur Karimov guruhga qo‘shdi: Aziz Nazarov”, “Sardor Sohinazarov guruh rasmini yangiladi”. Bunday xabarlar chat markazida kulrang yozuv ko‘rinishida chiqadi va ularni tahrirlab, o‘chirib yoki ularga javob berib bo‘lmaydi. Guruhda boshqa a‘zolarining xabarlari ustida jo‘natuvchining ismi va avatari ko‘rsatiladi. Sarlavhada a‘zolar va onlayn a‘zolar soni, kimdir yozayotgan bo‘lsa esa uning ismi chiqadi. Ma’lumot panelida guruh rasmi, tavsifi va a‘zolar ro‘yxati rollari bilan ko‘rsatiladi. Guruh egasi va administratorlarga esa a‘zolarini boshqarish menyusi ochiladi (3.5-rasm).



3.5-rasm. Guruh chati va guruh haqidagi ma’lumot paneli

Hikoyalar (stories) StoryService servisida amalga oshirilgan. Hikoya rasm yoki video (MP4, WebM, MOV, 50 MB gacha) va qisqa izohdan iborat. U joylangan paytdan boshlab 24 soat davomida ko‘rinadi ($ExpiresAt = CreatedAt + 24 \text{ soat}$). Hikoyalar lentasi foydalanuvchining o‘zi va u bilan chati bor foydalanuvchilarning amaldagi hikoyalarini qaytaradi. Chatlar ro‘yxati ustidagi hikoyalar panelida ko‘rilmagan hikoyasi bor foydalanuvchilar rangli halqa bilan ajratiladi. Hikoya ochilganda story_views jadvaliga yozuv qo‘shiladi. Hikoya muallifi esa uni kim va qachon ko‘rganini bilishi mumkin. Hikoya ko‘ruvchi oynasi 3.6-rasmda keltirilgan: yuqorida progress chiziqlari, muallif va vaqt, pastda esa izoh joylashgan.



3.6-rasm. Hikoyani ko‘rish oynasi

3.5. Klient qismini ishlab chiqish

Klient ilovasi Angular 22 ning mustaqil (standalone) komponentlaridan tashkil topgan. Ilova konfiguratsiyasida (app.config.ts) marshrutizator, SSR sahifasini brauzerda “jonlantirish” (hydration) va HTTP klient ulanadi. HTTP klientga authInterceptor ham qo‘shiladi. Ikkita marshrut mavjud: /auth (kirish va ro‘yxatdan o‘tish) va /chat (asosiy messenjer sahifasi). /chat sahifasi authGuard bilan

himoyalangan. Har ikkala sahifa ham kerak bo'lganda yuklanadi (lazy loading), bu esa boshlang'ich yuklanish hajmini kamaytiradi.

Ilova holati Angular signals mexanizmi orqali boshqariladi. ChatApiService'da chatlar ro'yxati, ochiq chat xabarlar, joriy guruh ma'lumotlari va "yozmoqda" holati signal ko'rinishida saqlanadi. Komponentlar ulardan computed() yordamida hosila qiymatlar oladi, masalan sarlavhadagi "6 a'zo, 2 onlayn" yoki "Jasur yozmoqda..." matni. Signal o'zgarganda Angular faqat unga bog'liq qismlarni qayta chizadi. Holat bitta servisda to'plangani uchun REST javoblari ham, SignalR hodisalari ham bir xil yo'l bilan qo'llanadi.

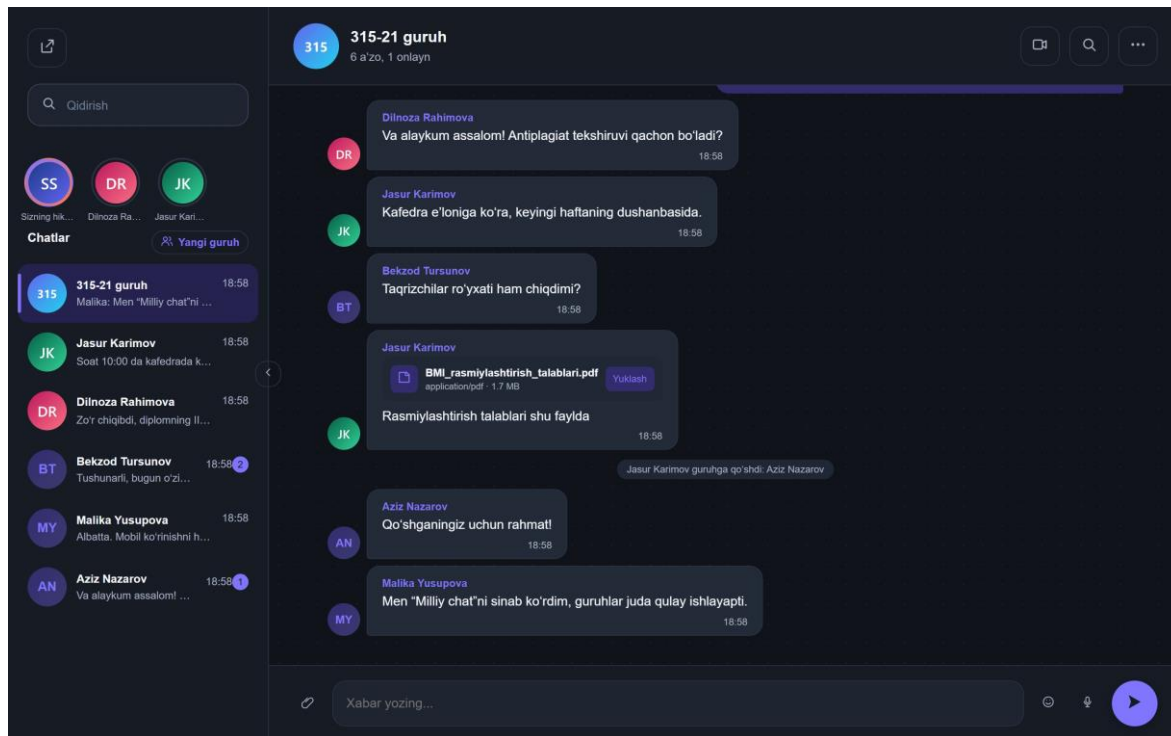
authInterceptor har bir API so'roviga access tokenni qo'shadi. Server 401 status kodini qaytarsa, interceptor tokenni avtomatik yangilab, so'rovni qayta yuboradi (3.5-listing). Bir vaqtda bir nechta so'rov muvaffaqiyatsiz bo'lsa ham, AuthService ular uchun bitta umumiy refresh so'rovini ishlatadi. Yangilash imkonsiz bo'lsa, foydalanuvchi kirish sahifasiga yo'naltiriladi.

3.5-listing. authInterceptor: muddati o'tgan tokenni avtomatik yangilash (TypeScript)

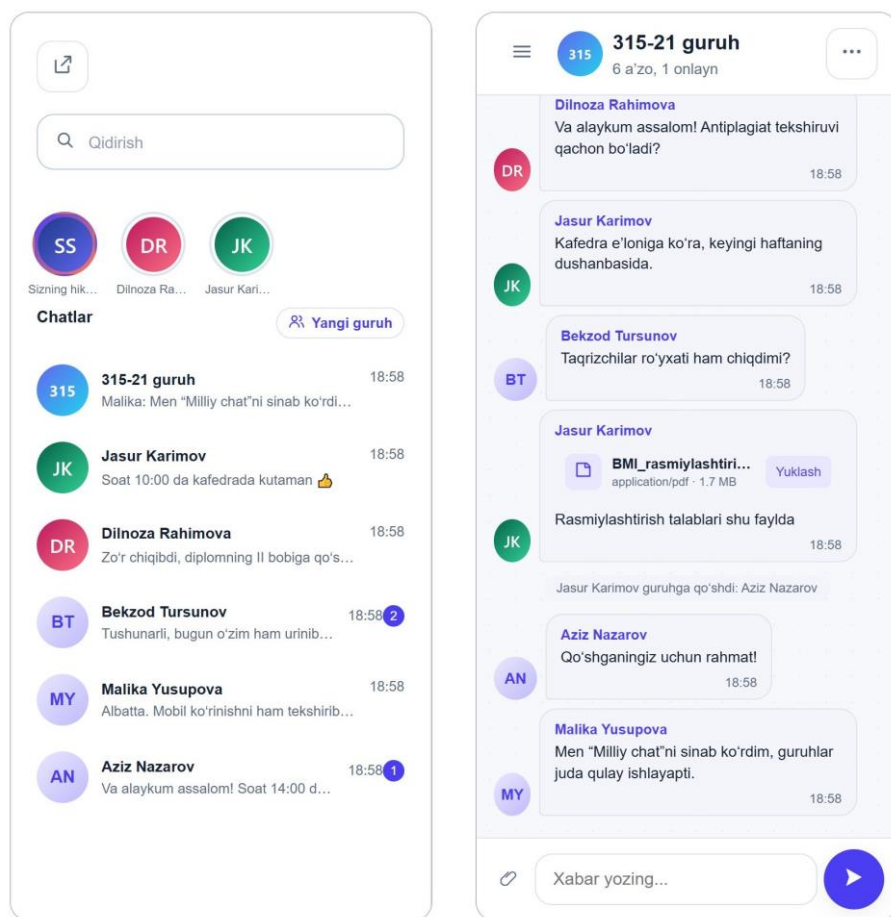
```
return next(authenticatedRequest).pipe(
  catchError((error: HttpErrorResponse) => {
    if (error.status !== 401 || !isApiRequest || isRefreshRequest || isAnonymous ||
    !auth.accessToken())
      return throwError(() => error);
    return auth.refreshAccessToken().pipe(
      switchMap((token) =>
        next(request.clone({ setHeaders: { Authorization: `Bearer ${token}` } })),
      ),
      catchError((refreshError) => {
        auth.clearToken();
        router.navigateByUrl('/auth');
        return throwError(() => refreshError);
      })
    );
  })
);
```

Interfeys loyiha uchun ishlab chiqilgan dizayn tizimiga asoslangan. Umumiy UI komponentlar (tugma, ikonkali tugma, matn maydoni, checkbox, banner, tasdiqlash oynasi, ikonkalar to'plami) shared/ui papkasida joylashgan. Ranglar, soyalar va radiuslar CSS o'zgaruvchilari (design tokens) orqali berilgan. Qorong'i mavzu shu o'zgaruvchilarning muqobil qiymatlari yordamida amalga oshirilgan (3.7-rasm). Foydalanuvchi tanlagan mavzu brauzer xotirasida saqlanadi.

Interfeys moslashuvchan (responsive) qilib yaratilgan. Katta ekranda uch ustunli joylashuv (chatlar ro'yxati, suhbat, ma'lumot paneli) ishlatiladi. Planshetda chatlar ro'yxati yig'iladi. Telefonda esa bir vaqtda bitta ekran ko'rsatiladi: chatlar ro'yxati yoki suhbat (3.8-rasm).



3.7-rasm. Qorong'i mavzudagi interfeys



3.8-rasm. Mobil qurilmadagi ko‘rinish: chatlar ro‘yxati va guruh chati

3.6. Joylashtirish va testlash

Joylashtirish.

Server ilovasi Docker konteyneri ko‘rinishida joylashtiriladi (3.6-listing). Birinchi bosqichda .NET SDK obrazida bog‘liqliklar tiklanadi va loyiha Release rejimida nashr qilinadi. Loyiha fayllari manba kodidan oldin nusxalanadi, shuning uchun Docker kodi o‘zgarganda ham bog‘liqliklar qatlamini keshdan qayta ishlatadi. Ikkinchi bosqichda faqat nashr natijasi kichik hajmli ASP.NET runtime obraziga ko‘chiriladi. Shu tufayli yakuniy obrazda kompilyator va manba kodi bo‘lmaydi.

3.6-listing. Server ilovasining Dockerfile fayli

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY src/API/API.csproj src/API/
COPY src/Application/Application.csproj src/Application/
COPY src/Domain/Domain.csproj src/Domain/
COPY src/Infrastructure/Infrastructure.csproj src/Infrastructure/
RUN dotnet restore src/API/API.csproj
COPY src/ src/
RUN dotnet publish src/API/API.csproj -c Release -o /app/publish --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
WORKDIR /app
COPY --from=build /app/publish .
ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080
ENTRYPOINT ["dotnet", "API.dll"]
```

Konteyner Render bulut platformasida “Web Service” sifatida ishga tushiriladi. Platforma GitHub repozitoriysidagi o‘zgarishlardan so‘ng obrazni avtomatik qayta yig‘adi. Maxfiy sozlamalar (ma’lumotlar bazasiga ulanish satri, JWT va HMAC kalitlari, SMTP va Google sozlamalari) kodda saqlanmaydi. Ular muhit o‘zgaruvchilari orqali beriladi, appsettings fayllari esa Git’dan chiqarib tashlangan. Klient ilovasining ishlab chiqarish konfiguratsiyasi (environment.prod.ts) bulutdagi API manziliga yo‘naltirilgan.

Unit testlar.

Biznes-mantiqning to‘g‘riligini tekshirish uchun xUnit freymvorkida NationalChat.Tests test loyihasi yaratildi. Tashqi bog‘liqliklar (real vaqt xabarnomasi, fayl servisi, repozitoriylar) NSubstitute kutubxonasi yordamida soxtalashtirildi (mock). Guruh servisi uchun esa IGroupRepository interfeysining xotirada ishlaydigan soxta amalga oshirilishi yozildi. Kursorli sahifalash EF Core InMemory provayderida sinaldi. Vaqtga bog‘liq mantiq qat’iy vaqt qaytaradigan TimeProvider yordamida tekshirildi. Testlarni bajarish natijasi 3.9-rasmda, ularning tarkibi esa 3.3-jadvalda keltirilgan.

```

PS> dotnet test tests\NationalChat.Tests --logger "console;verbosity=normal"
Test run for C:\Users\sohinazarov_s\Desktop\diplom_ish\NationalChat\Tests\NationalChat.Tests\bin\Debug\net8.0\NationalChat.Tests.dll (.NETCoreApp,Version=v8.0)
A total of 1 test files matched the specified pattern.
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.5.3.146b60a9e56a (64-bit .NET 8.0.31)
[xUnit.net 00:00:00.00] Discovering: NationalChat.Tests
[xUnit.net 00:00:00.10] Discovered: NationalChat.Tests
[xUnit.net 00:00:00.10] Starting: NationalChat.Tests
  Passed Security.Pbkdf2OneTimeCodeHasherTests.Verify_Authentication_ReturnsFalse [3 ms]
  Passed Security.HmacRefreshTokenHasherTests.Hash_IsDeterministicForSameKey [2 ms]
  Passed Security.HmacRefreshTokenHasherTests.Constructor_ShortKey_Throws [1 ms]
  Passed Security.HmacRefreshTokenHasherTests.Hash_DependOnKey [1 ms]
  Passed Validation.ValidatorTests.UpdateRole_OnlyMemberOrAdmin(role: Admin, expected: True) [15 ms]
  Passed Validation.ValidatorTests.UpdateRole_OnlyMemberOrAdmin(role: Member, expected: True) [1 ms]
  Passed Validation.ValidatorTests.UpdateRole_OnlyMemberOrAdmin(role: Creator, expected: False) [8 ms]
  Passed Validation.ValidatorTests.AddMembers_NonPositiveId_IsInvalid [26 ms]
  Passed Validation.ValidatorTests.CursorPagination_LimitMustBeBetween1And100(limit: 1, expected: True) [10 ms]
  Passed Validation.ValidatorTests.CursorPagination_LimitMustBeBetween1And100(limit: 101, expected: False) [1 ms]
  Passed Validation.ValidatorTests.CursorPagination_LimitMustBeBetween1And100(limit: 0, expected: False) [1 ms]
  Passed Validation.ValidatorTests.CursorPagination_LimitMustBeBetween1And100(limit: 100, expected: True) [1 ms]
  Passed Validation.ValidatorTests.CreateGroup_TitleLongerThanLimit_IsInvalid [1 ms]
  Passed Validation.ValidatorTests.CreateGroup_WithoutMembers_IsInvalid [1 ms]
  Passed Chats.ChatPermissionTests.Delete_ByCreatorOrPrivateParticipant_IsAllowed(type: Private, role: Member) [71 ms]
  Passed Chats.ChatPermissionTests.Delete_ByCreatorOrPrivateParticipant_IsAllowed(type: Group, role: Creator) [1 ms]
  Passed Groups.GroupServiceTests.Get_NonMember_ReturnsNull [76 ms]
  Passed Chats.ChatPermissionTests.DeleteGroup_ByNonCreator_IsDenied [2 ms]
  Passed Groups.GroupServiceTests.UpdateRole_OnlyCreatorCanPromote [5 ms]
  Passed Groups.GroupServiceTests.Create_EmptyTitle_FailsValidation [1 ms]
  Passed Chats.ChatPermissionTests.ClearGroupHistory_ByRegularMember_IsForbidden [7 ms]
  Passed Groups.GroupServiceTests.AddMembers_ByAdmin_AddsMemberAndDeletesServiceMessage [9 ms]
  Passed Groups.GroupServiceTests.RemoveMember_ByCreator_RemovesAndNotifiesRemovedUser [4 ms]
  Passed Groups.GroupServiceTests.AddMembers_OverMemberLimit_IsRejected [1 ms]
  Passed Groups.GroupServiceTests.Leave_Creator_TransfersOwnershipToLastAdmin [2 ms]
  Passed Groups.GroupServiceTests.Create_ValidRequest_CreatesGroupWithCreatorRoleAndServiceMessage [4 ms]
  Passed Groups.GroupServiceTests.Update_TitleChange_WritesTitleChangedServiceMessage [1 ms]
  Passed Groups.GroupServiceTests.Leave_LastMember_DeletesChat [1 ms]
  Passed Groups.GroupServiceTests.UpdateRole_CreatorRoleCannotBeGranted [1 ms]
  Passed Groups.GroupServiceTests.Create_UnknownMember_IsRejected [1 ms]
  Passed Groups.GroupServiceTests.AddMembers_AlreadyInGroup_IsRejected [1 ms]
  Passed Groups.GroupServiceTests.AddMembers_ByRegularMember_IsForbidden [1 ms]
  Passed Groups.GroupServiceTests.RemoveMember_AdminCannotRemoveCreator [1 ms]
  Passed Security.Pbkdf2OneTimeCodeHasherTests.Verify_wrongCode_ReturnsFalse [233 ms]
[xUnit.net 00:00:00.84] Finished: NationalChat.Tests
  Passed Pagination.CursorPaginationTests.LastPage_ReturnsCursor [184 ms]
  Passed Pagination.CursorPaginationTests.FirstPage_ReturnsNewestItemsAndCursor [28 ms]
  Passed Pagination.CursorPaginationTests.BeforeId_ReturnsOlderItemsOnly [12 ms]
  Passed Security.Pbkdf2OneTimeCodeHasherTests.Verify_correctCode_ReturnsTrue [228 ms]
  Passed Security.Pbkdf2OneTimeCodeHasherTests.Hash_SameCodeTwice_UsesDifferentSaltAndIterationPrefix [230 ms]
Test Run Successful.
Total tests: 39
Passed: 39
Total time: 1.2569 Seconds

```

3.9-rasm. Unit testlarni bajarish natijasi (dotnet test)

3.3-jadval

Unit testlar tarkibi va natijalari

Test sinfi	Soni	Tekshiriladigan holatlar	Natija
GroupServiceTests	15	Guruh yaratish va xizmat xabari; noma'lum foydalanuvchini rad etish; a'zo va admin ruxsatlari; admin egani chiqara olmasligi; rolni faqat ega o'zgartirishi; egalikning eng eski admin'ga o'tishi; oxirgi a'zo chiqqanda guruhning o'chishi; a'zolar limiti	O'tdi
ChatPermissionTests	4	Guruhni faqat ega o'chira olishi, shaxsiy chatni ishtirokchi o'chira olishi, tarixni oddiy a'zo tozalay olmasligi	O'tdi
Pbkdf2OneTimeCodeHasherTests	4	To'g'ri va noto'g'ri kod, har safar yangi tuz, buzilgan xesh	O'tdi
HmacRefreshTokenHasherTests	3	Determinizm, kalitga bog'liqlik, qisqa kalitni rad etish	O'tdi
CursorPaginationTests	3	Birinchi sahifa, beforeId bo'yicha oldingi yozuvlar, oxirgi sahifa	O'tdi
ValidatorTests	10	Sahifalash chegaralari, guruh nomi uzunligi, a'zolar ro'yxati, ruxsat etilgan rollar	O'tdi
Jami	39		39 / 39

Barcha 39 ta test muvaffaqiyatli o'tdi, umumiy bajarilish vaqti taxminan 1,3 soniyani tashkil etdi. Eng uzoq davom etgan testlar PBKDF2 xeshlash testlari bo'ldi

(har biri ~230 ms). Bu 210 000 iteratsiyali xeshlash kodni tanlab topishni qanchalik qimmatlashtirishini amalda ko'rsatadi.

Integratsion va qo'lda sinovlar.

Unit testlardan tashqari, tizim lokal muhitda PostgreSQL ma'lumotlar bazasi bilan birgalikda sinovdan o'tkazildi. Guruh chatlari funksiyasi uchun API darajasida 43 ta tekshiruvdan iborat avtomatik ssenariy yozildi. U bir nechta foydalanuvchini ro'yxatdan o'tkazib, guruh yaratish, ruxsatlar, xizmat xabarlar, tarixni tozalash, egalikni o'tkazish va guruhni o'chirish holatlarini HTTP so'rovlari orqali tekshiradi. Barcha 43 ta tekshiruv muvaffaqiyatli o'tdi. Foydalanuvchi interfeysi brauzerda bir nechta akkaunt bilan qo'lda sinaldi. Asosiy ssenariylar 3.4-jadvalda keltirilgan.

3.4-jadval

Qo'lda o'tkazilgan asosiy sinov ssenariylari

№	Ssenariy	Kutilgan natija	Natija
1	Yangi e-pochta bilan kirish va ro'yxatdan o'tish	Kod keladi, profil yaratiladi, chat sahifasi ochiladi	Mos
2	Noto'g'ri kodni 5 marta kiritish	Kod bekor qilinadi, shundan keyin to'g'ri kod ham qabul qilinmaydi	Mos
3	Ikki brauzerda xabar almashish	Xabar sahifani yangilamasdan keladi, ✓✓ belgisi paydo bo'ladi	Mos
4	Suhbatdosh matn yozmoqda	Sarlavhada "yozmoqda..." ko'rinadi, 3 soniyada yo'qoladi	Mos
5	Rasm va PDF fayl yuborish	Fayllar yuklanadi, ulashilgan media panelida ko'rinadi	Mos
6	Guruh yaratish va a'zo qo'shish	Guruh barcha a'zolarida darhol paydo bo'ladi, xizmat xabari chiqadi	Mos
7	Oddiy a'zo guruhni boshqarishga urinadi	Boshqaruv tugmalari ko'rinmaydi, API 400 qaytaradi	Mos
8	Guruhdan chiqarilgan a'zo	Chat uning ro'yxatidan real vaqtda yo'qoladi, kirish huquqi yo'qoladi	Mos
9	Boshqa qurilma sessiyasini yakunlash	Qurilma ro'yxatdan o'chadi, uning tokeni yangilanmaydi	Mos
10	Telefon o'lchamida (375 px) foydalanish	Gorizontaal aylantirish yo'q, barcha amallar mavjud	Mos

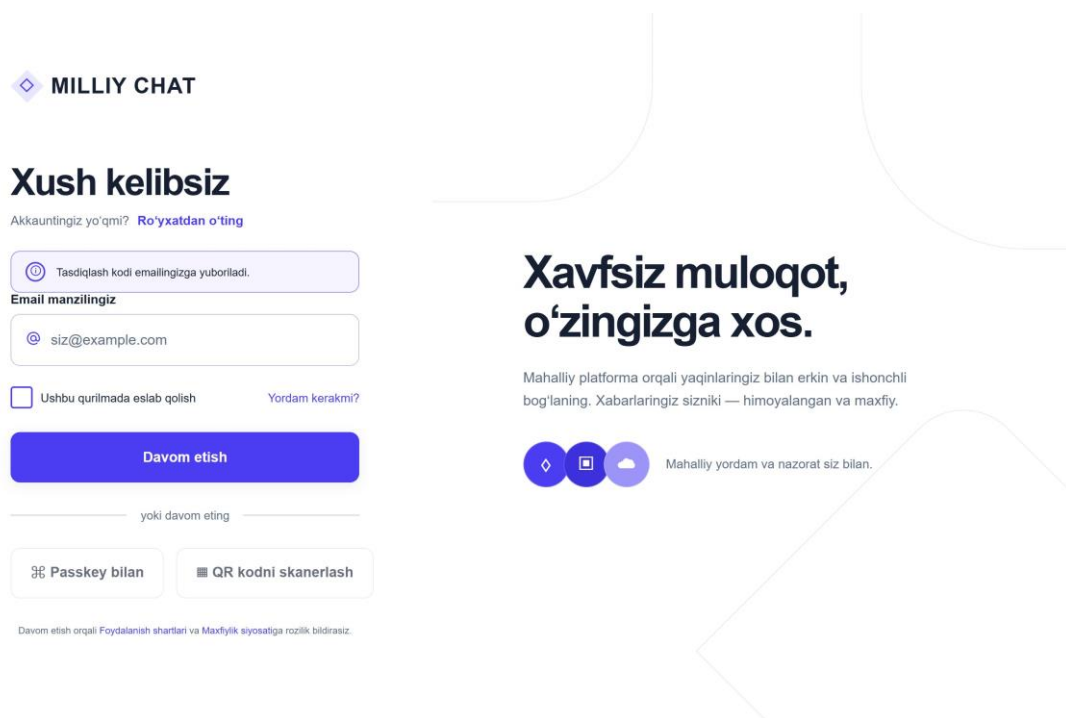
Sinovlar davomida ikkita kamchilik aniqlandi va tuzatildi. Uzun guruh nomi mobil ekranda sarlavhani ekran chegarasidan chiqarib yuborayotgan edi. Guruh

tarixi tozalanganda esa klient xizmat xabarlarini ham ekrandan olib tashlab, server bilan nomuvofiqlik yuzaga kelayotgan edi.

3.7. Foydalanish yo‘riqnomasi

Tizimga kirish va ro‘yxatdan o‘tish.

Ilova ochilganda kirish sahifasi ko‘rsatiladi (3.10-rasm). Foydalanuvchi elektron pochta manzilini kiritib, “Davom etish” tugmasini bosadi yoki Google hisobi orqali kiradi. Shundan so‘ng pochtaga 6 xonali tasdiqlash kodi yuboriladi va kodni kiritish oynasi ochiladi (3.11-rasm). Manzil tizimda birinchi marta ishlatilayotgan bo‘lsa, kod tasdiqlangach profil yaratish oynasi chiqadi. Unda ism, familiya va noyob username kiritiladi (3.12-rasm).



3.10-rasm. Tizimga kirish sahifasi

Kodni tasdiqlang

yangi.talaba@example.com manziliga yuborilgan 6 xonali kodni kiriting.

Kod 10 daqiqada davomida amal qiladi.

Tasdiqlash kodi

0 0 0 0 0

Kodni tekshirish

< Email manzilini o'zgartirish

Davom etish orqali Foydalanish shartlari va Maxfiylik siyosatiga rozilik bildirasiz.

Xavfsiz muloqot, o'zingizga xos.

Mahalliy platforma orqali yaqinlaringiz bilan erkin va ishonchli bog'laning. Xabarlaringiz sizniki — himoyalangan va maxfiy.



Mahalliy yordam va nazorat siz bilan.

3.11-rasm. Tasdiqlash kodini kiritish oynasi

Profil yarating

Siz uchun Milliy chatdagi profilningizni sozlaymiz.

Ma'lumotlaringiz faqat profilningizda ko'rsatiladi.

Ism

Familiya

Nodira

Qodirova

Username

@ nodira_q

5-50 belgi: harf, raqam va pastki chiziq ishlatiladi.

Profilni yaratish

< Email manzilini o'zgartirish

Davom etish orqali Foydalanish shartlari va Maxfiylik siyosatiga rozilik bildirasiz.

Xavfsiz muloqot, o'zingizga xos.

Mahalliy platforma orqali yaqinlaringiz bilan erkin va ishonchli bog'laning. Xabarlaringiz sizniki — himoyalangan va maxfiy.



Mahalliy yordam va nazorat siz bilan.

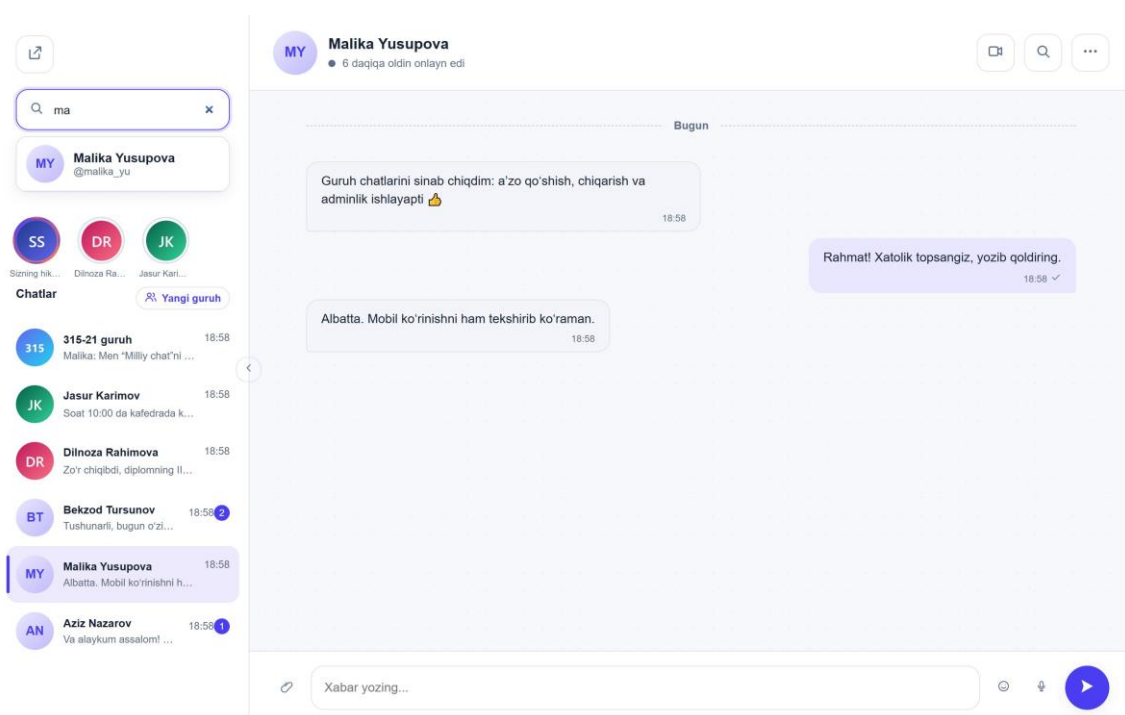
3.12-rasm. Profil yaratish (ro'yxatdan o'tish) oynasi

Suhbat boshlash va xabar almashish.

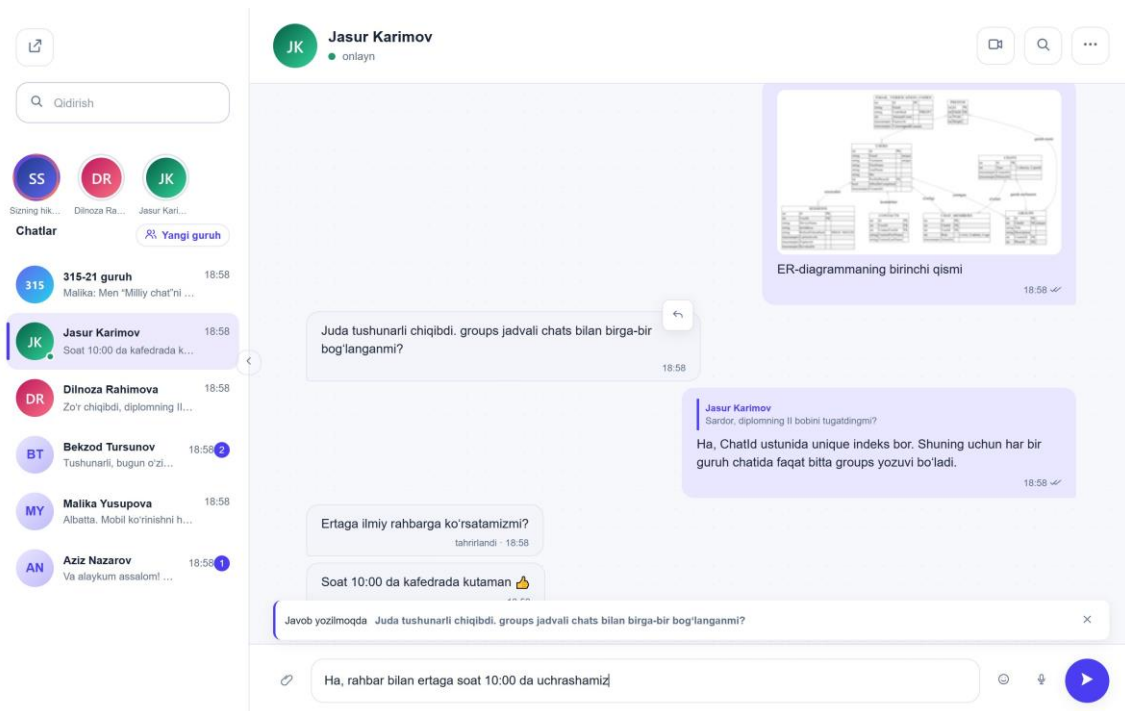
Asosiy sahifaning chap qismida qidiruv maydoni, hikoyalar paneli va chatlar ro'yxati joylashgan. Ro'yxatda har bir chatning oxirgi xabari, vaqti va o'qilmagan xabarlar soni ko'rsatiladi. Yangi suhbat boshlash uchun qidiruv maydoniga

foydalanuvchining ismi yoki username'i yoziladi va natijalardan kerakli shaxs tanlanadi (3.13-rasm).

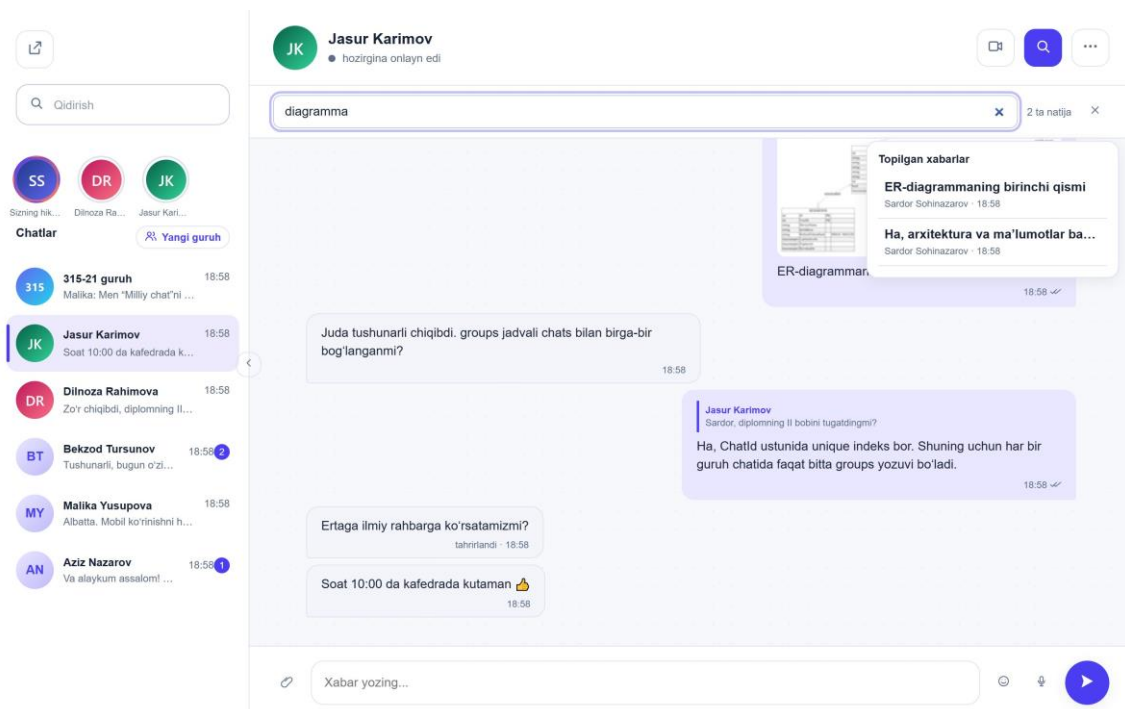
Xabar pastdagi maydonga yoziladi va Enter tugmasi yoki yuborish tugmasi bilan jo'natiladi. Qisqich (📎) tugmasi orqali rasm, video yoki fayl biriktiriladi. Sichqoncha xabar ustiga olib borilganda amallar paneli chiqadi: javob berish, o'z xabarini tahrirlash va o'chirish. Javob berish tanlanganda yozish maydoni ustida asl xabar ko'rsatiladi (3.14-rasm). Suhbat sarlavhasidagi qidiruv tugmasi chat ichida matn bo'yicha qidirish imkonini beradi. Topilgan natija bosilganda chat o'sha xabarga aylantiriladi va xabar ajratib ko'rsatiladi (3.15-rasm).



3.13-rasm. Foydalanuvchilarni qidirish



3.14-rasm. Xabarga javob yozish

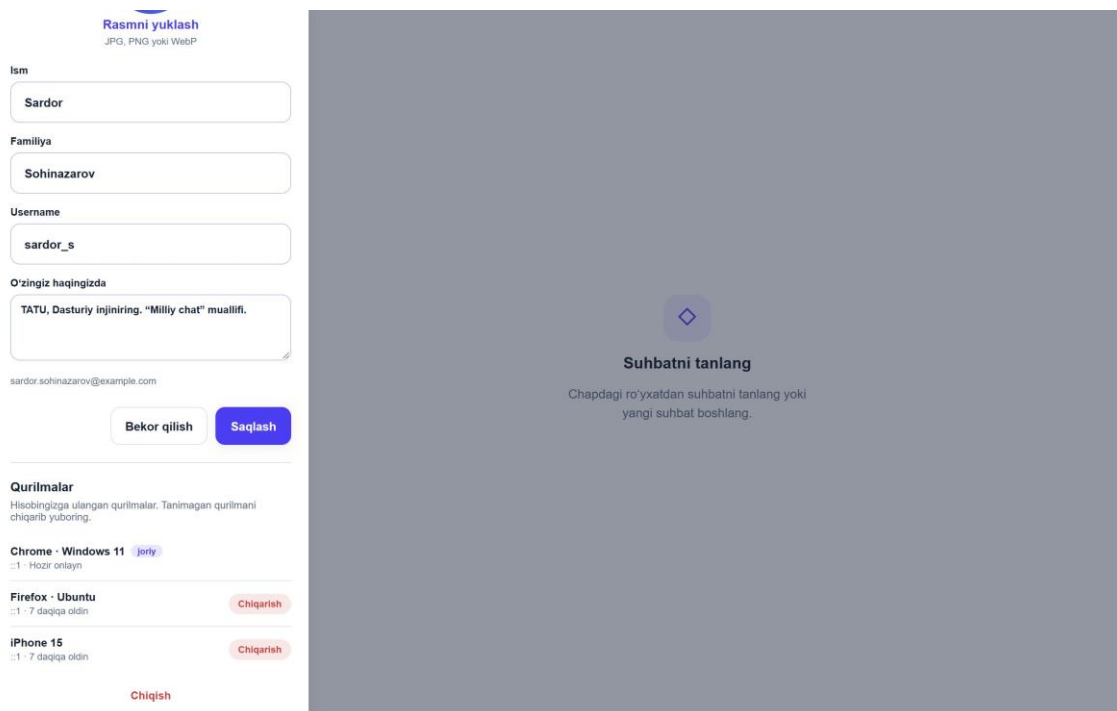


3.15-rasm. Chat ichida xabarlarni qidirish

Profil va qurilmalar.

Sidebar'ning yuqori qismidagi profil tugmasi orqali profil oynasi ochiladi. U yerda ism, familiya, username va o'zi haqidagi ma'lumotni tahrirlash, profil rasmini yuklash, yorug' va qorong'i mavzu o'rtasida almashish mumkin. "Qurilmalar" bo'limida hisobga ulangan barcha qurilmalar ro'yxati turadi. Joriy qurilma alohida

belgilanadi, boshqa qurilmalarni esa “Chiqarish” tugmasi bilan hisobdan chiqarish mumkin (3.16-rasm).

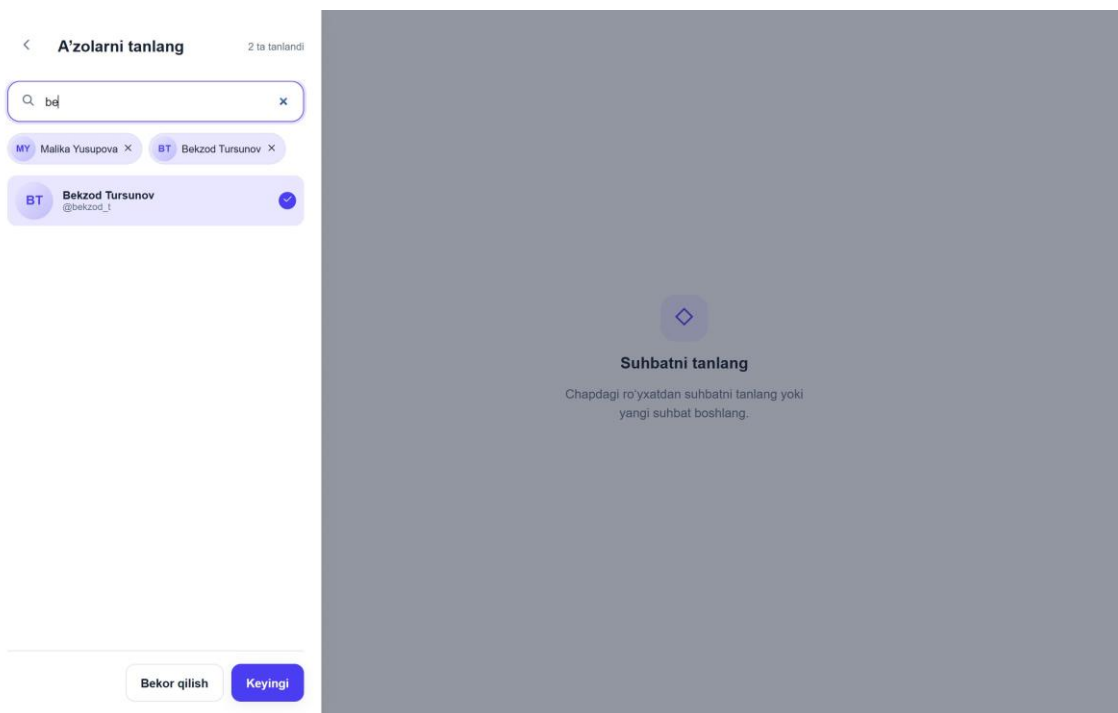


3.16-rasm. Profilni tahrirlash va qurilmalarni boshqarish

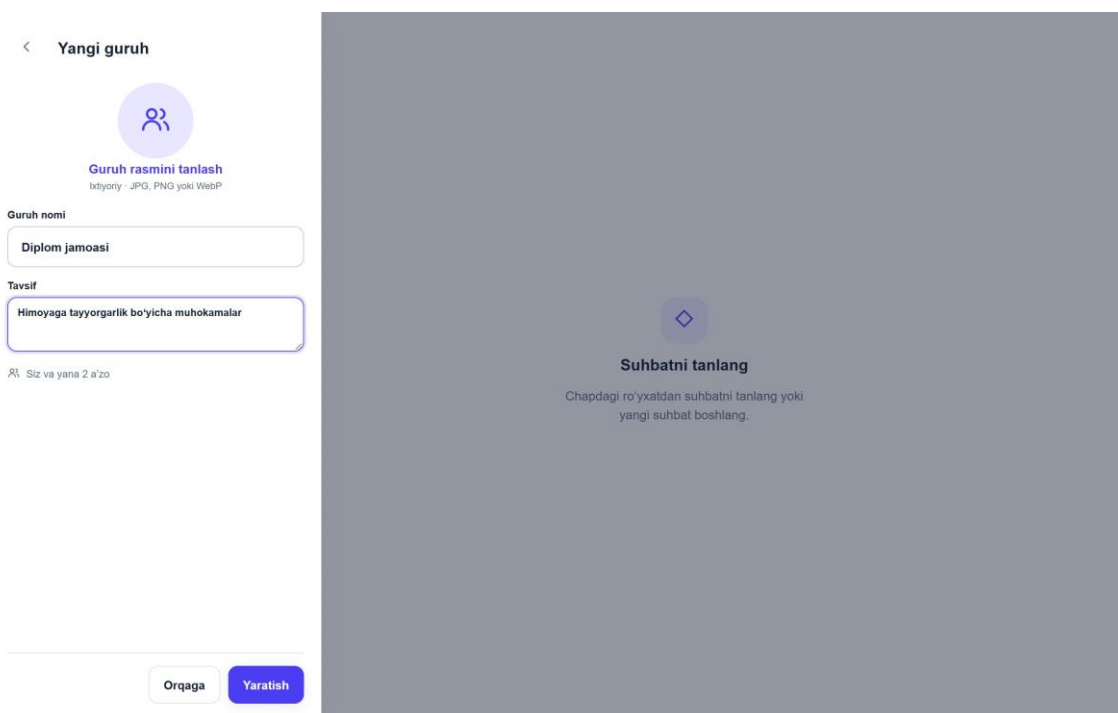
Guruh yaratish va boshqarish.

Guruh yaratish uchun chatlar ro‘yxati ustidagi “Yangi guruh” tugmasi bosiladi. Birinchi qadamda qidiruv orqali a‘zolar tanlanadi. Tanlangan foydalanuvchilar yuqorida belgilar (chip) ko‘rinishida ko‘rsatiladi (3.17-rasm). “Keyingi” tugmasi bosilgach, guruh nomi, tavsifi va ixtiyoriy rasmi kiritiladi va “Yaratish” tugmasi bosiladi (3.18-rasm). Yangi guruh barcha a‘zolarning chatlar ro‘yxatida darhol paydo bo‘ladi.

Guruh sarlavhasi bosilganda ma’lumot paneli ochiladi (3.5-rasm). Guruh egasi va administratorlar bu yerdan guruhni tahrirlashi, yangi a‘zo qo‘shishi va a‘zo yonidagi menyu orqali uni guruhdan chiqarishi mumkin. Guruh egasi qo‘shimcha ravishda administrator tayinlashi va guruhni butunlay o‘chirishi mumkin. Istalgan a‘zo “Guruhdan chiqish” tugmasi orqali guruhni tark eta oladi.



3.17-rasm. Guruh yaratish: a'zolarni tanlash



3.18-rasm. Guruh yaratish: nom va tavsifni kiritish

3-bob bo'yicha xulosa

Uchinchi bobda “Milliy chat” tizimining dasturiy amalga oshirilishi ko‘rib chiqildi. Server qismida 56 ta endpointdan iborat REST API, yagona javob formati, FluentValidation va markazlashgan xatolarni qayta ishlash, kursorli sahifalash, to‘rt metodli va o‘n hodisali SignalR habi, onlayn holatni kuzatish, media fayllarni

xavfsiz qayta ishlash konveyeri, guruh chatlari va hikoyalar amalga oshirildi. Klient qismida Angular signals asosidagi holat boshqaruvi, tokenni avtomatik yangilovchi interceptor, dizayn tizimi, qorong‘i mavzu va moslashuvchan interfeys yaratildi.

Tizim Docker konteyneri ko‘rinishida bulutli serverga joylashtirildi. Uning to‘g‘ri ishlashi 39 ta unit test, API darajasidagi 43 ta integratsion tekshiruv va qo‘lda o‘tkazilgan sinov ssenariylari bilan tasdiqlandi. Bob yakunida skrinshotlar bilan foydalanish yo‘riqnomasi keltirildi.

IV-BOB. HAYOT FAOLIYATI XAVFSIZLIGI

Hayot faoliyati xavfsizligi inson faoliyati jarayonida yuzaga keladigan xavfli va zararli omillarni aniqlash, ularning ta'sirini kamaytirish hamda favqulodda vaziyatlarda odamlar hayoti va sog'lig'ini saqlashga qaratilgan bilimlar va chora-tadbirlar tizimidir. "Milliy chat" kabi dasturiy mahsulotni ishlab chiqish va unga xizmat ko'rsatish jamoasi ish vaqtining asosiy qismini kompyuter qarshisida o'tkazadi. Shu sababli ushbu bobda ikki masala ko'rib chiqiladi: dasturchining ish joyini ergonomik talablar asosida tashkil etish hamda IT-kompaniya ofisida evakuatsiya tadbirlarini rejalashtirish va avariya yoki yong'in sodir bo'lganda xodimlarning harakatlari.

4.1. Dasturchining ergonomik ish joyini tashkil etish

Ergonomika (yunoncha *ergon* — ish va *nomos* — qonun) inson, mehnat vositalari va ish muhitining o'zaro moslashuvini o'rganadigan fan. Uning maqsadi mehnatni xavfsiz, qulay va samarali qilishdir. Ergonomik ish joyida jihozlar insonning antropometrik va fiziologik xususiyatlariga moslashtiriladi, aksincha emas.

Dasturchi mehnati aqliy mehnat turiga kiradi. Unda axborotni qabul qilish va qayta ishlash, diqqatni uzoq vaqt jamlash va ko'rish organlariga yuqori yuklama ustunlik qiladi. Jismoniy yuklama esa kam va asosan statik bo'ladi: xodim soatlab bir xil holatda o'tiradi, faqat barmoqlar va ko'zlar faol ishlaydi. Ish joyi noto'g'ri tashkil etilganda bu omillar surunkali charchoq va kasb kasalliklariga olib keladi.

Kompyuter bilan ishlashdagi xavfli va zararli omillar.

Dasturchining ish joyida quyidagi asosiy xavfli va zararli ishlab chiqarish omillari mavjud:

- **ko'rish analizatoriga yuklama** — ekranga uzoq tikilish, kam ko'z qirpish, yorqinlik kontrasti va aks etishlar ko'zning quruqlashishi, charchashi va ko'rishning pasayishiga ("kompyuter ko'rish sindromi") olib keladi;

- **statik yuklama va gipodinamiya** — noto‘g‘ri o‘tirish bo‘yin, yelka va bel og‘riqlarini, qon aylanishining sustlashishini keltirib chiqaradi;
- **qo‘l va bilakdagi takroriy harakatlar** — klaviatura va sichqoncha bilan uzoq ishlash bilak (tunnel) sindromi va paylarning yallig‘lanishiga sabab bo‘lishi mumkin;
- **noqulay mikroiklim** — kompyuterlar va monitorlar issiqlik chiqaradi, konditsionerlar esa havoni quritadi va sovuq havo oqimlarini hosil qiladi;
- **yoritilganlikning yetarli emasligi yoki ortiqchaligi**, ekrandagi yaltirash va aks etishlar;
- **shovqin** — sovutish ventilyatorlari, printerlar, ochiq ofisdagi suhbatlar diqqatni chalg‘itadi;
- **elektr toki urish xavfi** — shikastlangan kabellar, yerga ulanmagan jihozlar va ortiqcha yuklangan uzaytirgichlar;
- **psixosotsional zo‘riqish** — muddatlar bosimi, uzoq vaqt diqqatni jamlash, tungi navbatchiliklar va tizim avariyalarini bartaraf etish.

Ish joyini tashkil etishga qo‘yiladigan talablar O‘zbekiston Respublikasining Mehnat kodeksi va “Mehnatni muhofaza qilish to‘g‘risida”gi Qonuni, sanitariya qoidalari va normalari hamda ISO 9241-5 (ish joyini joylashtirish va gavda holatiga talablar), GOST 12.2.032-78 (o‘tirib bajariladigan ishlarda ish joyi), GOST 12.1.005-88 (ish zonasi havosi) va KMK 2.01.05-98 (tabiiy va sun‘iy yoritish) standartlari bilan belgilanadi.

Ish xonasiga qo‘yiladigan talablar.

Kompyuterli ish joylari joylashgan xonada bitta ish joyiga to‘g‘ri keladigan maydon kamida 6 m², hajm esa kamida 20 m³ bo‘lishi tavsiya etiladi. Xonada tabiiy va sun‘iy yoritish bo‘lishi lozim. Derazalar shimol yoki shimoli-sharq tomonga qaragani ma’qul. Ular jalyuzi yoki pardalar bilan jihozlanadi.

Monitorlar derazaga nisbatan yon tomondan joylashtiriladi: yorug‘lik chap tomondan tushishi kerak. Deraza ekran orqasida bo‘lsa, ko‘z yorqin fon va nisbatan qorong‘i ekran o‘rtasida doimo moslashishga majbur bo‘ladi. Deraza xodimning orqasida bo‘lsa, ekranda yaltirash hosil bo‘ladi. Bir monitorning ekrani bilan

ikkinchisining orqa tomoni orasidagi masofa kamida 2 m, yon tomonlari orasidagi masofa esa kamida 1,2 m bo'lishi kerak.

Xona devorlari va shifti yaltiramaydigan (matli) materiallar bilan pardoatlanadi. Yorug'likni qaytarish ko'effitsiyentlari shift uchun 0,7–0,8, devorlar uchun 0,5–0,6, pol uchun 0,3–0,5 bo'lishi tavsiya etiladi. Xona har kuni nam usulda tozalanadi va har bir ish soatidan so'ng shamollatiladi.

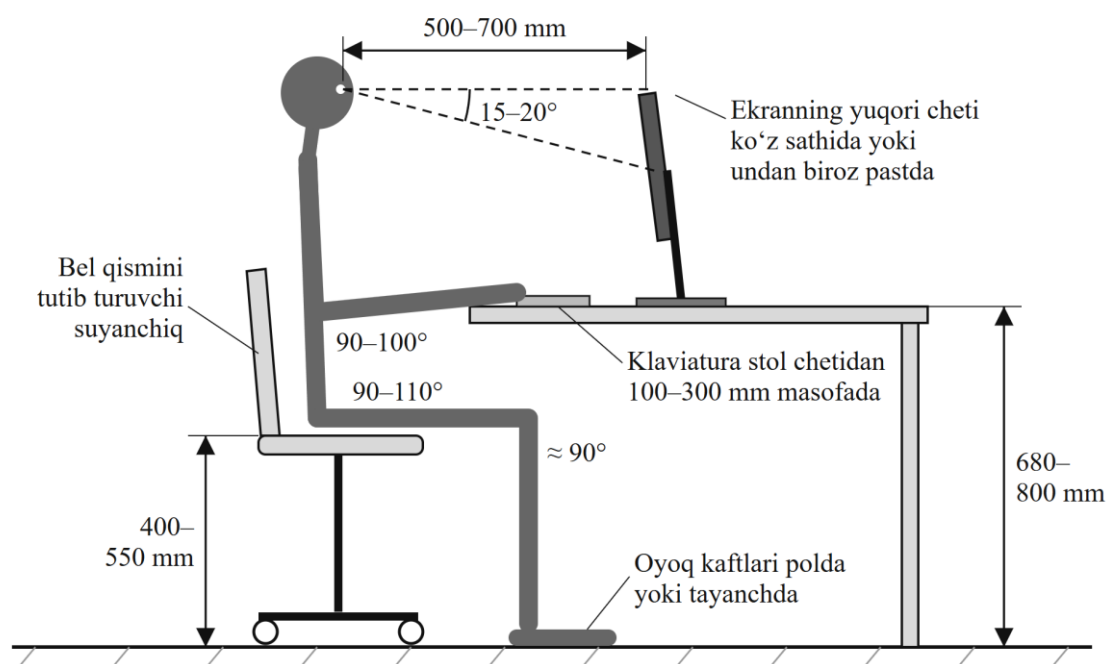
Ish joyi jihozlarining ergonomik parametrlari.

Dasturchining ish joyi ish stoli, sozlanadigan stul, monitor, klaviatura, sichqoncha va zarur bo'lsa oyoq tayanchidan iborat. Ularning tavsiya etiladigan o'lchamlari 4.1-jadvalda, xodimning to'g'ri gavda holati esa 4.1-rasmda keltirilgan.

4.1-jadval

Ish joyi elementlarining ergonomik parametrlari

Element	Parametr	Tavsiya etiladigan qiymat
Ish stoli	Ish yuzasining balandligi	680–800 mm (sozlanmaydigan stol uchun 725 mm)
	Ish yuzasining o'lchamlari	eni 800–1400 mm, chuqurligi 800–1000 mm
	Oyoqlar uchun bo'shliq	balandligi ≥ 600 mm, eni ≥ 500 mm, chuqurligi tizza sathida ≥ 450 mm, oyoq sathida ≥ 650 mm
Stul	O'rindiq balandligi	400–550 mm, sozlanadigan
	O'rindiqlarning eni va chuqurligi	kamida 400 mm, old qirrasi yumaloqlangan
	Suyanchiq	balandligi 300 ± 20 mm, eni ≥ 380 mm, qiyalik burchagi $0 \dots 30^\circ$ oralig'ida sozlanadi
	Qo'l tayanchlari	uzunligi ≥ 250 mm, eni 50–70 mm, balandligi sozlanadi
Monitor	Ko'zdan ekrangacha masofa	500–700 mm
	Joylashuv balandligi	ekranning yuqori cheti ko'z sathida yoki undan pastroq, nigoh ekran markaziga $15\text{--}20^\circ$ pastga yo'naladi
Klaviatura	Stol chetidan masofa	100–300 mm, bilak uchun tayanch joy qoldiriladi
Oyoq tayanchi	O'lchamlari	eni ≥ 300 mm, chuqurligi ≥ 400 mm, qiyaligi $0 \dots 20^\circ$



4.1-rasm. Kompyuterda ishlovchi xodimning to‘g‘ri gavda holati va ish joyining asosiy o‘lchamlari

To‘g‘ri gavda holatining asosiy qoidasi “to‘g‘ri burchaklar” qoidasi deb ataladi: tirsaklar 90–100°, son-chanoq bo‘g‘imlari 90–110° va tizzalar taxminan 90° burchak ostida bukiladi. Oyoq kaftlari polga yoki oyoq tayanchiga to‘liq tiraladi. Bel suyanchiqqa tayanadi, yelkalar bo‘shashgan holatda bo‘ladi. Bilaklar klaviatura ustida to‘g‘ri tutiladi, yuqoriga yoki yon tomonga bukilmaydi. Sichqoncha klaviatura bilan bir sathda va tanaga yaqin joylashtiriladi.

Dasturchilar ko‘pincha ikki monitor bilan ishlaydi. Bunda asosiy monitor to‘g‘ridan qarshida, ikkinchisi esa yonida, ko‘zdan bir xil masofada va xodimga qaratib joylashtiriladi. Ikkala monitor teng ishlatilsa, ularning tutashgan joyi xodim ro‘parasida bo‘ladi. Noutbuk bilan uzoq ishlaganda uning ekrani ko‘z sathiga maxsus tagliklar bilan ko‘tariladi, alohida klaviatura va sichqoncha ulanadi. Aks holda boshni doimo pastga egib o‘tirishga to‘g‘ri keladi.

Ish joyi faqat jismoniy jihozlardan iborat emas. Dasturiy interfeys ham ergonomik talablarga javob berishi kerak (ISO 9241-110). Ishlab chiqilgan “Milliy chat” interfeysida matn va fon o‘rtasida yetarli kontrast, qorong‘i mavzu, tizim shrift o‘lchamini hisobga oladigan maketlar va mobil qurilmalarga moslashuv ko‘zda

tutilgani foydalanuvchilarning ko‘z charchog‘ini kamaytirishga xizmat qiladi. Dasturchining o‘z ish vositalarida (kod muharriri, terminal) ham qorong‘i mavzu, katta shrift va yaxshi o‘qiladigan monoshirinli shriftlardan foydalanish tavsiya etiladi.

Mikroiqlim, yoritish va shovqin.

Dasturchi mehnati energiya sarfi bo‘yicha Ia toifadagi yengil ishlarga kiradi (energiya sarfi 139 W gacha). Bunday ishlar uchun ish zonasi mikroiqlimining maqbul ko‘rsatkichlari, yoritilganlik va shovqin me‘yorlari 4.2-jadvalda keltirilgan.

4.2-jadval

Dasturchi ish joyi uchun mikroiqlim, yoritilganlik va shovqin me‘yorlari

Ko‘rsatkich	Yilning sovuq davri	Yilning iliq davri
Havo harorati, °C	22–24	23–25
Nisbiy namlik, %	40–60	40–60
Havo harakatining tezligi, m/s	≤ 0,1	≤ 0,1
Stol yuzasining yoritilganligi, lk	300–500	300–500
Ekran yuzasining yoritilganligi, lk	≤ 300	≤ 300
Yoritilganlik pulsatsiyasi koeffitsiyenti, %	≤ 5	≤ 5
Shovqin darajasi, dBA	≤ 50	≤ 50

Umumiy yoritish uchun diffuzorli (sochuvchi) LED yoritgichlardan foydalaniladi. Ular yorug‘likni bir tekis taqsimlaydi va miltillamaydi. Rang harorati 4000–5000 K bo‘lgan neytral oq yorug‘lik ko‘zni kamroq charchatadi. Yoritgichlar monitorlar bilan parallel qatorlarda joylashtiriladi, ularning yorug‘ligi ekranda aks etmasligi kerak. Zarur bo‘lsa, hujjatlar bilan ishlash uchun yaltiramaydigan mahalliy yoritgich qo‘shiladi.

Shovqinni kamaytirish uchun printerlar va boshqa shovqinli jihozlar alohida joyga chiqariladi, ochiq ofislarda tovush yutuvchi panellar va to‘siqlar o‘rnatiladi, qo‘ng‘iroqlar va muhokamalar uchun alohida xonalar ajratiladi. Server uskunalari ish xonalarida emas, alohida server xonasida joylashtiriladi.

Ish xonasining sun'iy yoritilishini hisoblash.

Misol sifatida dasturchilar bo'limi uchun o'lchamlari $A \times B \times H = 6 \times 6 \times 3$ m bo'lgan xonani ko'rib chiqamiz. Xonaning maydoni $S = 36 \text{ m}^2$, hajmi $V = 108 \text{ m}^3$. Bitta ish joyiga kamida 6 m^2 maydon va 20 m^3 hajm to'g'ri kelishi kerakligidan xonaga joylashtirish mumkin bo'lgan ish joylari soni:

$$n_{\max} = \min(S / 6; V / 20) = \min(36 / 6; 108 / 20) = \min(6; 5,4) = 5 \quad (4.1)$$

Demak, xonada 5 ta ish joyi tashkil etiladi. Umumiy sun'iy yoritish yorug'lik oqimidan foydalanish koeffitsiyenti usuli bilan hisoblanadi. Avval xona indeksi aniqlanadi:

$$i = A \cdot B / (h \cdot (A + B)), \quad (4.2)$$

bu yerda h — yoritgichlarning ish yuzasidan balandligi. Yoritgichlar shiftga o'rnatiladi, ish yuzasi poldan 0,8 m balandlikda joylashgan: $h = 3,0 - 0,8 = 2,2$ m. U holda $i = 36 / (2,2 \cdot 12) \approx 1,36$. Shift, devor va polning yorug'lik qaytarish koeffitsiyentlari mos ravishda 70, 50 va 30 % bo'lganda LED panellar uchun bu indeksga yorug'lik oqimidan foydalanish koeffitsiyenti $\eta \approx 0,54$ to'g'ri keladi. Kerakli yoritgichlar soni quyidagi formula bilan topiladi:

$$N = E_n \cdot S \cdot K_z \cdot z / (\Phi \cdot \eta), \quad (4.3)$$

bu yerda $E_n = 400 \text{ lk}$ — me'yoriy yoritilganlik; $K_z = 1,4$ — zaxira koeffitsiyenti (yoritgichlarning ifloslanishi va eskirishini hisobga oladi); $z = 1,1$ — yoritilganlikning notekislik koeffitsiyenti; $\Phi = 3600 \text{ lm}$ — bitta yoritgichning yorug'lik oqimi ($600 \times 600 \text{ mm}$ o'lchamli, 36 W quvvatli LED panel). Hisoblash natijasi:

$$N = 400 \cdot 36 \cdot 1,4 \cdot 1,1 / (3600 \cdot 0,54) = 22\,176 / 1944 \approx 11,4 \quad (4.4)$$

Yoritgichlar soni butun songa yaxlitlanadi va simmetrik joylashuv uchun $N = 12$ qabul qilinadi (4 ta ustun $\times$ 3 ta qator). Haqiqiy o'rtacha yoritilganlik quyidagiga teng:

$$E_h = N \cdot \Phi \cdot \eta / (S \cdot K_z \cdot z) = 12 \cdot 3600 \cdot 0,54 / (36 \cdot 1,4 \cdot 1,1) \approx 421 \text{ lk} \quad (4.5)$$

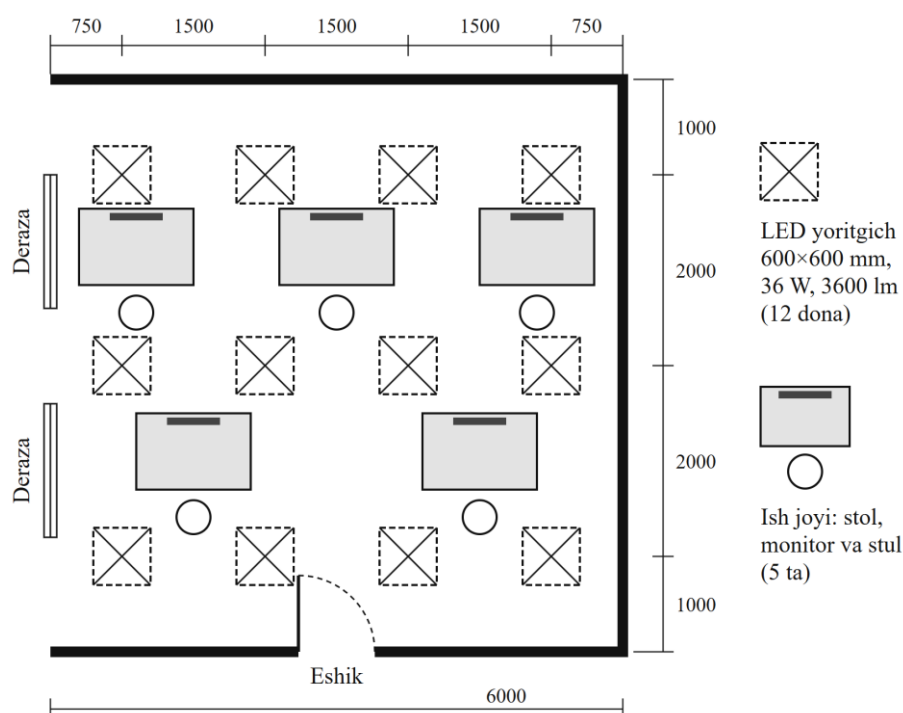
Haqiqiy yoritilganlik me'yordan 5 % ga yuqori. Bu ruxsat etilgan $-10...+20$ % oralig'ida. Yoritish tizimining umumiy quvvati $12 \cdot 36 = 432 \text{ W}$, solishtirma

quvvati 12 W/m². Hisoblash natijalari 4.3-jadvalda umumlashtirilgan, yoritgichlar va ish joylarining joylashuvi 4.2-rasmda ko‘rsatilgan.

4.3-jadval

Ish xonasining sun‘iy yoritilishini hisoblash natijalari

Parametr	Belgilanishi	Qiymati
Xona o‘lchamlari	$A \times B \times H$	6 × 6 × 3 m
Yoritgichlarning ish yuzasidan balandligi	h	2,2 m
Xona indeksi	i	1,36
Yorug‘lik oqimidan foydalanish koeffitsiyenti	η	0,54
Me‘yoriy yoritilganlik	E_n	400 lk
Zaxira va notekislik koeffitsiyentlari	K_z, z	1,4; 1,1
Bitta yoritgichning yorug‘lik oqimi	Φ	3600 lm
Yoritgichlar soni	N	12 ta
Haqiqiy yoritilganlik	E_h	421 lk
Umumiy / solishtirma quvvat	P, p	432 W; 12 W/m ²



4.2-rasm. Dasturchilar xonasida yoritgichlar va ish joylarining joylashuv sxemasi

Xonani shamollatish uchun zarur havo miqdori bitta xodimga to‘g‘ri keladigan hajmga bog‘liq. Bu hajm 20–40 m³ bo‘lganda (bizda $108 / 5 = 21,6$ m³) har bir xodimga soatiga kamida 30 m³ toza havo berilishi kerak:

$$L = n \cdot L_1 = 5 \cdot 30 = 150 \text{ m}^3/\text{soat}; \quad K = L / V = 150 / 108 \approx 1,4 \text{ soat}^{-1} \quad (4.6)$$

Demak, xonada havo soatiga kamida 1,4 marta almashishi kerak. Buni tabiiy shamollatish yetarli bo‘lmaganda kichik quvvatli so‘ruvchi-beruvchi ventilyatsiya yoki rekuperatorli konditsioner ta‘minlaydi.

Mehnat va dam olish rejimi.

Kompyuter bilan ishlashda uzluksiz ish vaqti 2 soatdan oshmasligi kerak. Har 45–60 daqiqada 5–10 daqiqalik yoki har 2 soatda 15 daqiqalik tanaffus qilish tavsiya etiladi. Tanaffus vaqtida ekrandan uzoqlashish, yurish va yengil jismoniy mashqlar bajarish lozim. Dasturchilar orasida keng tarqalgan “Pomodoro” usuli (25 daqiqa ish va 5 daqiqa dam olish) ham bu talabga mos keladi.

Ko‘z charchog‘ini kamaytirish uchun “20-20-20” qoidasi qo‘llaniladi: har 20 daqiqada 20 soniya davomida kamida 20 fut (6 m) uzoqlikdagi narsaga qarash. Ko‘z uchun mashqlar (ko‘zni yumib-ochish, nigohni uzoq va yaqin nuqtalarga ko‘chirish) ham foydali. Bo‘yin, yelka, bel va bilaklar uchun qisqa mashqlar kun davomida bir necha marta bajariladi.

Ish beruvchi kompyuterda ishlovchi xodimlarni dastlabki va davriy tibbiy ko‘riklardan o‘tkazishi, ularga mehnatni muhofaza qilish bo‘yicha yo‘l-yo‘riq berishi kerak. Masofadan (uydan) ishlaganda ham xuddi shu talablar amal qiladi. Shu sababli masofaviy ishlaydigan dasturchilarga ergonomik stul va monitor bilan ta‘minlash yoki ish joyini o‘zi tashkil etishi uchun tavsiyalar berish maqsadga muvofiq.

Elektr xavfsizligi.

Ish joyidagi barcha jihozlar 220 V kuchlanishli tarmoqqa ulanadi. Shu sababli rozetkalar yerga ulash kontakti bilan jihozlanadi, kabellar maxsus kanallarda yoki stol ostidagi tokchalarda yotqiziladi va oyoq ostida qolmaydi. Uzaytirgichlarni bir-biriga ketma-ket ulash, shikastlangan kabel va vilkalardan foydalanish, jihozlar korpusini ochish taqiqlanadi. Server va tarmoq uskunalari uzluksiz quvvat manbalari (UPS) orqali ulanadi. Elektr jihozlarini ta‘mirlash faqat tegishli ruxsatga ega mutaxassis tomonidan bajariladi.

4.2. Evakuatsiya tadbirlarini rejalashtirish hamda avariya va yong'inlar sodir bo'lganda xodimlarning harakatlari

Favqulodda vaziyat deganda odamlar qurbon bo'lishiga, ularning sog'lig'iga yoki atrof-muhitga zarar yetishiga, katta moddiy yo'qotishlarga va hayot faoliyati sharoitlarining buzilishiga olib kelgan yoki olib kelishi mumkin bo'lgan avariya, halokat, tabiiy ofat yoki boshqa hodisa natijasida muayyan hududda yuzaga kelgan holat tushuniladi. Kelib chiqishiga ko'ra favqulodda vaziyatlar texnogen, tabiiy va ekologik turlarga bo'linadi.

IT-kompaniya ofisi uchun eng ehtimoliy favqulodda vaziyatlar yong'in, zilzila (Toshkent shahri yuqori seysmik faollik zonasida joylashgan), elektr ta'minotidagi avariya, isitish yoki suv quvurlarining yorilishi va gaz sizishidir. Bunday vaziyatlarda xodimlar hayotini saqlashning asosiy usuli ularni o'z vaqtida va tartibli evakuatsiya qilishdir.

Bu sohadagi asosiy talablar "Favqulodda vaziyatlarda aholini va hududlarni muhofaza qilish to'g'risida"gi, "Fuqaro muhofazasi to'g'risida"gi va "Yong'in xavfsizligi to'g'risida"gi qonunlar hamda GOST 12.1.004-91 "Yong'in xavfsizligi. Umumiy talablar" standarti bilan belgilanadi.

IT-ofisda yong'in chiqish sabablari va o't o'chirish vositalari.

Ofis binolarida yong'inlarning aksariyati elektr tarmog'i va jihozlar bilan bog'liq. Asosiy sabablar:

- elektr tarmog'ining ortiqcha yuklanishi: bitta rozetkaga ko'p jihoz ulash, uzaytirgichlarni ketma-ket ulash;
- eskirgan yoki shikastlangan izolyatsiya tufayli qisqa tutashuv, rozetka va vilkalardagi bo'sh kontaktlar;
- noutbuk va powerbanklarning litiy-ion akkumulyatorlari shikastlanishi yoki qizib ketishi, sifatsiz zaryadlovchi qurilmalar;
- UPS akkumulyatorlarining nosozligi, server xonasida sovutish tizimining ishdan chiqishi;
- qarovsiz qoldirilgan isitish asboblari, elektr choynaklar va boshqa maishiy jihozlar;

- chekish qoidalarini buzish va ochiq olovdan ehtiyotsiz foydalanish.

Ofisdagi yonuvchi materiallar (qog‘oz, mebel, plastik korpuslar, kabel izolyatsiyasi) yonganda ko‘p miqdorda zaharli tutun ajraladi. Yong‘inlarda halok bo‘lganlarning aksariyati olovdan emas, aynan tutun va is gazidan zaharlanishdan jabrlanadi. Shu sababli tutunni erta aniqlash va odamlarni tezkor evakuatsiya qilish hal qiluvchi ahamiyatga ega.

Bino avtomatik yong‘in signalizatsiyasi (tutun va issiqlik datchiklari), odamlarni xabardor qilish va evakuatsiyani boshqarish tizimi hamda birlamchi o‘t o‘chirish vositalari bilan jihozlanadi. Server xonasida suv bilan o‘chirish uskunalarga zarar yetkazgani uchun avtomatik gazli o‘t o‘chirish tizimi o‘rnatiladi. Yong‘in sinflari va ularni o‘chirish vositalari 4.4-jadvalda keltirilgan.

4.4-jadval

Yong‘in sinflari va o‘t o‘chirish vositalari

Sinf	Yonuvchi material	Qo‘llaniladigan vositalar
A	Qattiq materiallar: qog‘oz, yog‘och, mato, plastmassa	suv, ko‘pikli va kukunli (ABCE) o‘t o‘chirgichlar, ichki yong‘in krani
B	Yonuvchi suyuqliklar: erituvchilar, bo‘yoqlar, yoqilg‘i	ko‘pikli, kukunli va karbonat angidridli o‘t o‘chirgichlar
C	Yonuvchi gazlar: tabiiy gaz, propan	kukunli o‘t o‘chirgichlar; avvalo gaz berilishi to‘xtatiladi
D	Metallar: magniy, natriy, alyuminiy kukuni	maxsus kukunli o‘t o‘chirgichlar
E	Kuchlanish ostidagi elektr jihozlari	karbonat angidridli (OU) va kukunli o‘t o‘chirgichlar; suv va ko‘pik ishlatilmaydi

Kompyuterlar o‘rnatilgan xonalarda asosan karbonat angidridli o‘t o‘chirgichlar (OU-3, OU-5) ishlatiladi. Ular elektr o‘tkazmaydi va uskunalarda qoldiq qoldirmaydi. Bunday o‘t o‘chirgich bilan ishlaganda uning karnayini qo‘l bilan ushlab bo‘lmaydi: gaz kengayganda karnay -70°C gacha soviydi va terini sovuq urishi mumkin. O‘t o‘chirgichlar ko‘rinadigan joylarda, poldan 1,5 m dan baland bo‘lmagan joyda osiladi. Yong‘in chiqishi mumkin bo‘lgan joydan eng yaqin o‘t o‘chirgichgacha masofa 20 m dan oshmasligi kerak.

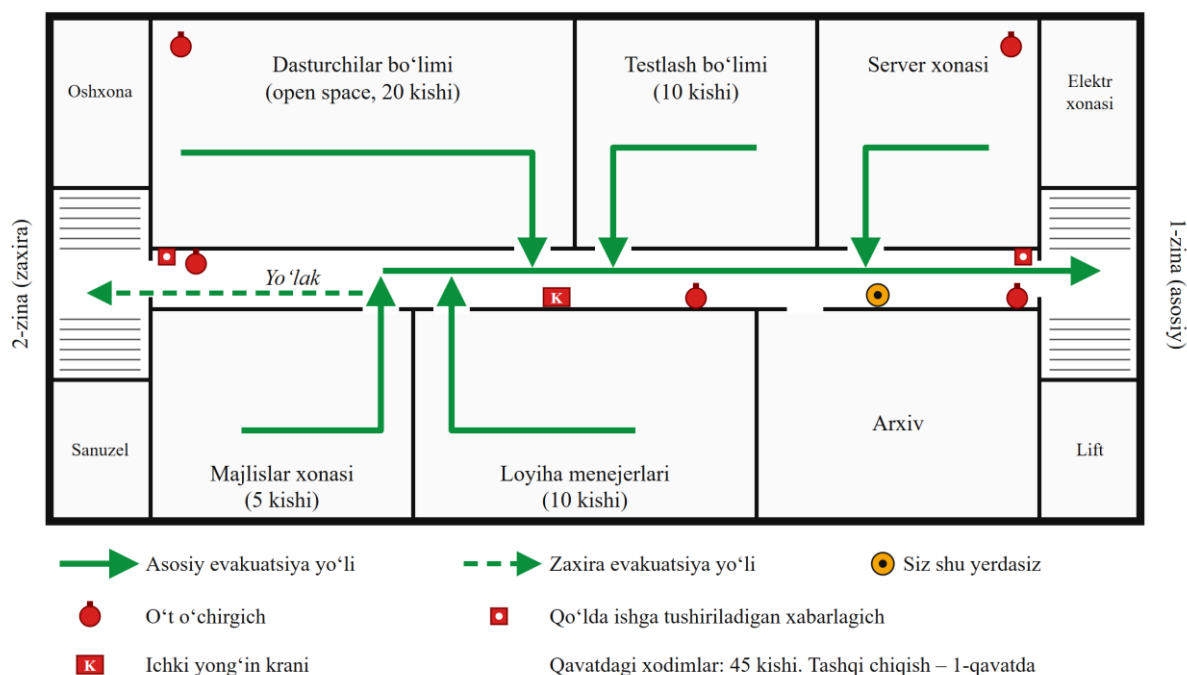
Evakuatsiya tadbirlarini rejalashtirish.

Evakuatsiya — odamlarni xavfli omillar ta'sir qila boshlashidan oldin binodan yoki uning qismidan xavfsiz hududga uyushgan holda chiqarish jarayoni. Evakuatsiya yo'li — evakuatsiya chiqishiga olib boradigan, yong'in xavfli omillaridan himoyalangan yo'l. Evakuatsiya chiqishi esa to'g'ridan-to'g'ri tashqariga, xavfsiz zinapoyaga yoki qo'shni yong'inga qarshi bo'linmaga olib boradigan chiqishdir.

Evakuatsiyani rejalashtirish tashkiliy va texnik tadbirlardan iborat. Texnik tadbirlarga quyidagilar kiradi:

- har bir qavatda kamida ikkita, bir-biridan uzoqda joylashgan evakuatsiya chiqishi;
- evakuatsiya yo'llari va eshiklarning me'yoriy kengligi (yo'laklar kamida 1–1,2 m, eshiklar kamida 0,8 m); eshiklar chiqish yo'nalishida ochiladi va ichkaridan kalitsiz ochiladi;
- yo'laklar va zinapoyalarni mebel, jihozlar va qutilar bilan to'sib qo'yishni taqiqlash;
- avariya (evakuatsiya) yoritishi va ISO 7010 standartiga mos yashil rangli “Favqulodda chiqish” belgilari;
- yong'in signalizatsiyasi, odamlarni ovozli xabardor qilish tizimi va qo'lda ishga tushiriladigan xabarlagichlar;
- har bir qavatda evakuatsiya reja-sxemalari.

Evakuatsiya reja-sxemasi grafik va matnli qismlardan iborat. Grafik qismda qavatning rejasi, “Siz shu yerdasiz” belgisi, asosiy (yashil yaxlit strelkalar) va zaxira (yashil punktir strelkalar) evakuatsiya yo'llari, evakuatsiya chiqishlari, o't o'chirgichlar, ichki yong'in kranlari va xabarlagichlarning joylashuvi ko'rsatiladi. Matnli qismda xodimlarning harakatlar tartibi va mas'ul shaxslar ro'yxati beriladi. Reja-sxemalar ko'zga yaxshi ko'rinadigan joylarda, zinapoyalar va chiqishlar yonida, qorong'ida ko'rinadigan (fotolyuminessent) materialdan tayyorlanib osiladi. Namunaviy IT-kompaniya ofisining uchinchi qavati uchun evakuatsiya reja-sxemasi 4.3-rasmda keltirilgan.



4.3-rasm. IT-kompaniya ofisi 3-qavatining evakuatsiya reja-sxemasi (namuna)

Tashkiliy tadbirlar quyidagilarni o'z ichiga oladi:

- tashkilot rahbarining buyrug'i bilan yong'in xavfsizligi uchun mas'ul shaxs, har bir qavat bo'yicha evakuatsiya mas'ullari va birinchi yordam ko'rsatuvchi xodimlar tayinlanadi;
- binodan tashqarida xavfsiz masofada xodimlar yig'iladigan joy belgilanadi va xodimlar ro'yxatlari tayyorlanadi;
- barcha xodimlar ishga qabul qilinganda kirish yo'riqnomasidan, keyinchalik esa davriy yo'riqnomadan o'tkaziladi, yong'in-texnik minimum bo'yicha o'qitiladi;
- evakuatsiya bo'yicha amaliy mashg'ulotlar muntazam, odatda yarim yilda kamida bir marta o'tkaziladi;
- o't o'chirish vositalari, signalizatsiya va avariya yoritishining sozligi belgilangan muddatlarda tekshiriladi;
- yangi xodimlar va mehmonlar evakuatsiya yo'llari va yig'ilish joyi bilan tanishtiriladi.

Evakuatsiya vaqtini hisoblash.

Evakuatsiya yo'llarining yetarliligini baholash uchun hisobiy evakuatsiya vaqti aniqlanadi. Hisoblash GOST 12.1.004-91 standartida keltirilgan odamlar oqimining harakat parametrlari asosida soddalashtirilgan usulda bajariladi. Evakuatsiya yo'li o'lchamlari va odamlar soni bir xil bo'lgan uchastkalarga ajratiladi. Har bir uchastkadagi odamlar oqimining zichligi quyidagicha aniqlanadi:

$$D_i = N_i \cdot f / (l_i \cdot \delta_i), \quad (4.7)$$

bu yerda N_i — uchastkadagi odamlar soni; $f = 0,1 \text{ m}^2$ — yozgi kiyimdagi katta yoshli odamning gorizontaal proyeksiya maydoni; l_i va δ_i — uchastkaning uzunligi va kengligi, m. Zichlikka qarab 4.5-jadvaldan harakat tezligi v_i va oqim intensivligi q_i olinadi. Uchastkadan o'tish vaqti va umumiy hisobiy evakuatsiya vaqti:

$$t_i = l_i / v_i; \quad t_h = t_1 + t_2 + \dots + t_n \quad (4.8)$$

Eshiklar va yo'l torayadigan joylarda tirbandlik bor-yo'qligi oqim intensivligi orqali tekshiriladi:

$$q_e = q_i \cdot \delta_i / \delta_e \leq q_{\max}, \quad (4.9)$$

bu yerda δ_e — eshik kengligi; $q_{\max} = 19,6 \text{ m/min}$ — eshikdan o'tishdagi oqim intensivligining eng katta qiymati. Bu shart bajarilsa, eshik oldida odamlar to'planib qolmaydi.

4.5-jadval

Odamlar oqimining harakat parametrlari (GOST 12.1.004-91 bo'yicha, parcha)

Oqim zichligi $D, \text{ m}^2/\text{m}^2$	Gorizontaal yo'l: $v, \text{ m/min}$	Gorizontaal yo'l: $q, \text{ m/min}$	Eshik: $q, \text{ m/min}$	Zinadan pastga: $v, \text{ m/min}$	Zinadan pastga: $q, \text{ m/min}$
0,01	100	1,0	1,0	100	1,0
0,05	100	5,0	5,0	100	5,0
0,10	80	8,0	8,7	95	9,5
0,20	60	12,0	13,4	68	13,6
0,30	47	14,1	16,5	52	15,6
0,40	40	16,0	18,4	40	16,0
0,50	33	16,5	19,6	31	15,5

4.3-rasmdagi qavat uchun hisoblashni bajaramiz. Qavatda 45 nafar xodim ishlaydi. Eng og‘ir holat ko‘rib chiqiladi: 2-zina yong‘in tufayli to‘sib qo‘yilgan va barcha xodimlar asosiy (1-) zina orqali chiqadi. Hisobiy yo‘l dasturchilar bo‘limining eng uzoq ish joyidan boshlanadi va to‘rtta uchastkadan iborat:

1. dasturchilar bo‘limi ichidagi o‘tish joyi: $l_1 = 12$ m, $\delta_1 = 1,5$ m, $N_1 = 20$ kishi;
2. yo‘lak: bo‘lim eshigidan zinagacha $l_2 = 17$ m, $\delta_2 = 2,0$ m, $N_2 = 45$ kishi;
3. 3-qavatdan 1-qavatgacha zina marshlari va maydonchalari: $l_3 = 2 \cdot 10 = 20$ m, $\delta_3 = 1,35$ m, $N_3 = 45$ kishi;
4. 1-qavat vestibulidan tashqi chiqishgacha: $l_4 = 15$ m, $\delta_4 = 3,0$ m, $N_4 = 45$ kishi.

Masalan, birinchi uchastka uchun $D_1 = 20 \cdot 0,1 / (12 \cdot 1,5) \approx 0,11$. 4.5-jadvaldagi qiymatlar orasida chiziqli interpolyatsiya qilib $v_1 \approx 78$ m/min va $q_1 \approx 8,4$ m/min topiladi, $t_1 = 12 / 78 \approx 0,15$ min. Qolgan uchastkalar ham xuddi shunday hisoblanadi. Natijalar 4.6-jadvalda keltirilgan.

4.6-jadval

Evakuatsiya vaqtini hisoblash natijalari

Uchastka	l , m	δ , m	N , kishi	D , m ² /m ²	v , m/min	t , min
1. Bo‘lim ichidagi o‘tish joyi	12	1,5	20	0,11	78	0,15
2. Yo‘lak	17	2,0	45	0,13	74	0,23
3. Zina (pastga)	20	1,35	45	0,17	77	0,26
4. Vestibul	15	3,0	45	0,10	80	0,19
Jami hisobiy evakuatsiya vaqti t_h						0,83

Eshiklarda tirbandlik yo‘qligini (4.9) formula bilan tekshiramiz. Bo‘lim eshigi ($\delta = 1,2$ m): $q = 8,4 \cdot 1,5 / 1,2 \approx 10,5$ m/min. Zinaga kirish eshigi ($\delta = 1,2$ m): $q = 9,3 \cdot 2,0 / 1,2 \approx 15,5$ m/min. Tashqi eshik ($\delta = 1,5$ m): $q = 8,0 \cdot 3,0 / 1,5 = 16,0$ m/min. Barcha qiymatlar $q_{\max} = 19,6$ m/min dan kichik, ya‘ni odamlar oqimi to‘xtovsiz harakatlanadi.

Demak, hisobiy evakuatsiya vaqti $t_h \approx 0,83$ min (taxminan 50 soniya). Xabardor qilish tizimi ishga tushgandan so‘ng evakuatsiya boshlanishidagi

kechikishni taxminan 1 daqiqa deb olsak, qavatdan to'liq chiqish uchun taxminan 2 daqiqa kerak bo'ladi. Bu vaqt yong'in xavfli omillari (tutun, harorat, kislorod kamayishi) evakuatsiya yo'llarida kritik qiymatga yetish vaqtidan kichik bo'lishi shart. Bu vaqt binoning yong'in xavfini baholash hujjatlarida aniqlanadi. Hisob ko'rsatadiki, eng tor joy zina hisoblanadi. Shu sababli zinaga olib boradigan eshiklar doimo yopiq holatda (o'zi yopiladigan qurilma bilan), lekin qulflanmagan bo'lishi, zina maydonchalarida esa hech qanday buyum saqlanmasligi kerak.

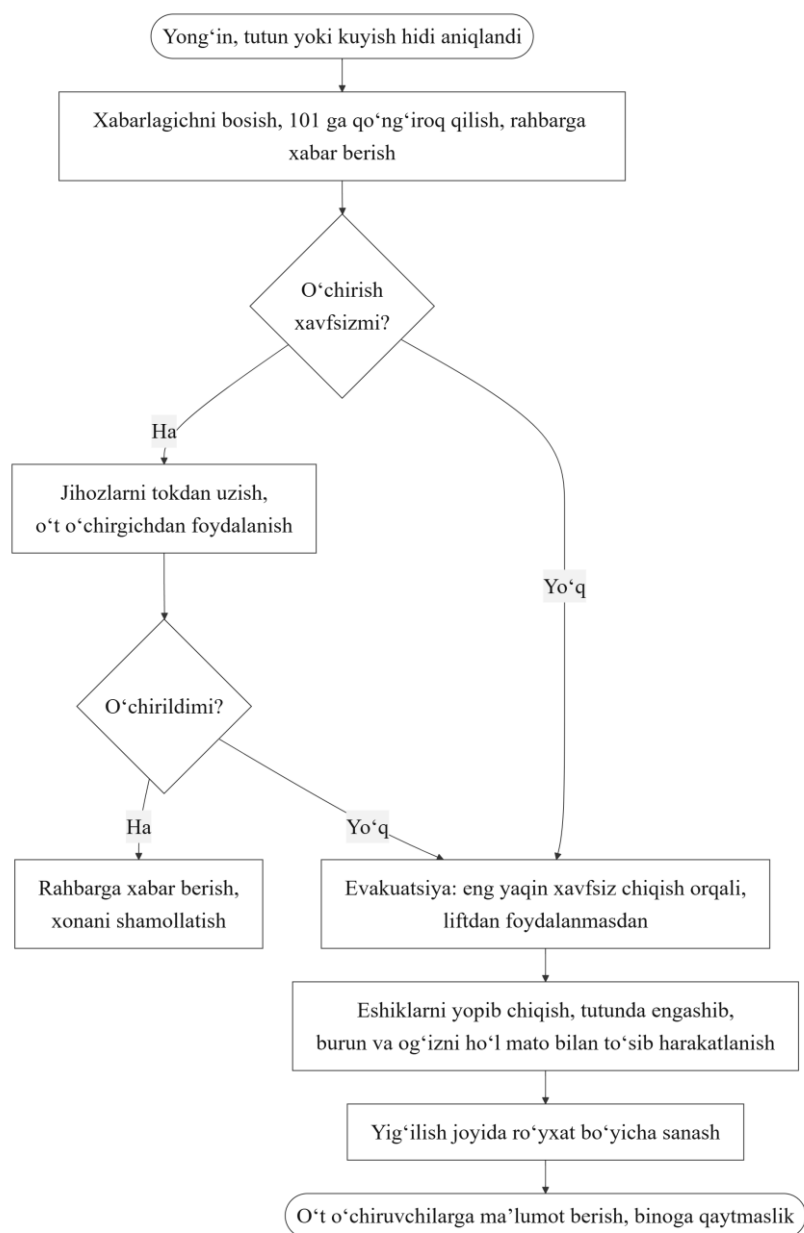
Yong'in sodir bo'lganda xodimlarning harakatlari.

Yong'in chiqqanda vahimaga tushmaslik va oldindan o'rgatilgan tartibda harakat qilish muhim. Xodimlarning harakatlar algoritmi 4.4-rasmda ko'rsatilgan va quyidagi qoidalarga asoslanadi:

1. Yong'in, tutun yoki kuyish hidini sezgan xodim darhol qo'lda ishga tushiriladigan xabarlagichni bosadi va 101 raqamiga (yoki yagona 112 raqamiga) qo'ng'iroq qiladi. Bunda binoning manzili, qavati, nima yonayotgani, odamlar bor-yo'qligi va o'z ismi aytiladi. So'ngra bo'lim rahbari va yong'in xavfsizligi uchun mas'ul shaxsga xabar beriladi.
2. Yong'in endi boshlangan bo'lsa va bu xavfsiz bo'lsa, elektr jihozlari tarmoqdan uziladi (xona yoki qavat avtomat o'chirgichi o'chiriladi) va birlamchi vositalar bilan o'chirishga harakat qilinadi. Kuchlanish ostidagi jihozlarni suv bilan o'chirish mumkin emas.
3. Yong'inni 1–2 daqiqada o'chirib bo'lmasa yoki tutun ko'paysa, darhol evakuatsiya boshlanadi. Evakuatsiya reja-sxemasida ko'rsatilgan eng yaqin xavfsiz chiqishdan foydalaniladi. Lift dan foydalanish taqiqlanadi, chunki u to'xtab qolishi va tutunli qavatda ochilishi mumkin.
4. Xonadan chiqqanda eshik yopiladi, lekin qulflanmaydi. Bu tutun va olovning tarqalishini sekinlashtiradi. Derazalarni ochish mumkin emas, chunki havo oqimi yonishni kuchaytiradi.
5. Tutunli yo'lakda engashib yoki kerak bo'lsa emaklab harakatlaniladi, chunki pastda havo tozaroq bo'ladi. Burun va og'iz ho'l mato bilan

to'siladi, yo'nalishni yo'qotmaslik uchun bir qo'l bilan devorni ushlab boriladi.

6. Qo'ldan kelsa, mehmonlarga va harakatlanishi cheklangan hamkasblarga yordam beriladi. Shaxsiy buyumlarni olish uchun ortga qaytilmaydi.
7. Yig'ilish joyida qavat mas'uli xodimlarni ro'yxat bo'yicha sanaydi. Yetib kelmagan xodimlar va ular bo'lishi mumkin bo'lgan joy haqida yetib kelgan o't o'chirish bo'linmasiga darhol xabar beriladi. O't o'chiruvchilar ruxsat bermaguncha binoga qaytilmaydi.



4.4-rasm. Yong'in sodir bo'lganda xodimlarning harakatlar algoritmi

Yong'in xavfsizligi uchun mas'ul shaxs o't o'chirish xizmati chaqirilganini tekshiradi, evakuatsiyani boshqaradi, ventilyatsiya va konditsionerlarni o'chirishni tashkil etadi (ular tutunni tarqatadi), o't o'chirish bo'linmasini kutib oladi va unga yong'in joyi, odamlar va server xonasidagi gazli o't o'chirish tizimi haqida ma'lumot beradi.

Agar kiyimga olov tushsa, yugurish mumkin emas, chunki havo oqimi olovni kuchaytiradi. Yerga yotib, dumalab yoki qalin mato bilan o'rab olovni o'chirish kerak. Kuyish jarohatida kuygan joy 10–20 daqiqa davomida salqin oqar suv ostida sovutiladi va toza bog'lam qo'yiladi. Tutundan zaharlangan odam toza havoga olib chiqiladi va 103 raqami orqali tez tibbiy yordam chaqiriladi.

Boshqa avariyalarda xodimlarning harakatlari.

- **Zilzila.** Silkinish paytida zinapoya va liftga yugurish xavfli. Mustahkam stol ostiga yashirinish yoki ichki kapital devor burchagida, eshik o'rnida turish, derazalar, shkaflar va osma jihozlardan uzoqlashish kerak. Silkinish tugagach, bino evakuatsiya yo'llari bo'ylab tark etiladi, yig'ilish joyi elektr uzatish liniyalari va binolardan uzoqda tanlanadi.
- **Server xonasida gazli o't o'chirish tizimining ishga tushishi.** Ogohlantiruvchi ovozli va yorug'lik signali berilganda xona darhol tark etiladi va eshik yopiladi. O't o'chiruvchi gaz xonadagi kislorod miqdorini kamaytiradi, shuning uchun xona shamollatilmaguncha unga kirish taqiqlanadi.
- **Elektr toki urishi.** Jabrlanuvchiga yalang qo'l bilan tegish mumkin emas. Avval tok manbai o'chiriladi yoki jabrlanuvchi quruq yog'och kabi dielektrik buyum yordamida tokdan ajratiladi. So'ngra 103 raqami chaqiriladi va zarur bo'lsa yurak-o'pka reanimatsiyasi boshlanadi.
- **Gaz sizishi.** Olov yoqish, elektr kalitlari va qurilmalarni yoqish yoki o'chirish mumkin emas, chunki uchqun portlashga sabab bo'lishi mumkin. Derazalar ochiladi, xonadagilar chiqariladi va 104 raqamiga xabar beriladi.

- **Suv quvurlarining yorilishi.** Suv bosishi mumkin bo'lgan xonalardagi elektr ta'minoti o'chiriladi, uskunalar balandroq joyga ko'chiriladi va xo'jalik xizmatiga xabar beriladi.

Favqulodda vaziyatlarda xodimlarni tezkor xabardor qilish uchun asosiy ovozli xabar berish tizimidan tashqari korporativ aloqa vositalaridan ham foydalanish mumkin. Masalan, ishlab chiqilgan "Milliy chat" tizimida tashkilotning barcha xodimlari a'zo bo'lgan guruh (200 nafargacha a'zo) yaratilib, unda faqat administratorlar xabar e'lon qiladi. Bunday guruh orqali evakuatsiya haqida xabar, yig'ilish joyi va keyingi ko'rsatmalar bir necha soniyada barcha xodimlarning telefon va kompyuterlariga yetkaziladi. Xodimlar esa xabarga javob yozib, xavfsiz joyga yetib kelganini tasdiqlaydi. Bu vosita yong'in signalizatsiyasi va ovozli xabar berish tizimining o'rnini bosmaydi, balki ularni to'ldiradi. U ayniqsa masofadan ishlayotgan va binoda bo'lmagan xodimlarni xabardor qilishda foydali. Tizim serverlari mamlakat ichida joylashgani favqulodda vaziyatlarda aloqaning mustaqilligini oshiradi.

4-bob bo'yicha xulosa

To'rtinchi bobda hayot faoliyati xavfsizligining dasturiy ta'minot ishlab chiquvchilar uchun muhim bo'lgan ikki masalasi ko'rib chiqildi. Birinchi bo'limda kompyuter bilan ishlashdagi xavfli va zararli omillar aniqlandi. Ish xonasi, ish stoli, stul va monitorga qo'yiladigan ergonomik talablar, mikroiklim, yoritish va shovqin me'yorlari hamda mehnat va dam olish rejimi keltirildi. 6×6 m o'lchamli dasturchilar xonasi uchun joylashtirish mumkin bo'lgan ish joylari soni (5 ta), sun'iy yoritish (12 ta 36 W li LED yoritgich, haqiqiy yoritilganlik 421 lk) va zarur havo almashinuvi ($150 \text{ m}^3/\text{soat}$) hisoblandi.

Ikkinchi bo'limda IT-ofisda yong'in chiqish sabablari, yong'in sinflari va o't o'chirish vositalari, evakuatsiyani rejalashtirishning texnik va tashkiliy tadbirlari ko'rib chiqildi hamda namunaviy qavat uchun evakuatsiya reja-sxemasi ishlab chiqildi. 45 kishilik qavat uchun hisobiy evakuatsiya vaqti taxminan 0,83 daqiqani tashkil etishi va eshiklarda tirbandlik yuzaga kelmasligi hisoblab ko'rsatildi.

Yong'in, zilzila va boshqa avariyalarda xodimlarning harakatlar tartibi berildi. Bundan tashqari, "Milliy chat" tizimidan favqulodda vaziyatlarda xodimlarni qo'shimcha xabardor qilish vositasi sifatida foydalanish imkoniyati ko'rsatildi.

XULOSA

Bitiruv malakaviy ishida foydalanuvchi ma'lumotlarini o'z serverida saqlaydigan, Telegramga o'xshash "Milliy chat" veb-messenjeri loyihalandi va ishlab chiqildi. Ishning kirish qismida qo'yilgan vazifalar quyidagicha hal etildi.

- Telegram, WhatsApp, Signal, Viber va WeChat messenjerlari funksional imkoniyatlari, xavfsizlik modeli va ma'lumotlarni saqlash joyi bo'yicha taqqoslandi. Ularning barchasi xorijiy yurisdiksiyada ekani va bu "Shaxsga doir ma'lumotlar to'g'risida"gi Qonunning lokalizatsiya talablari nuqtai nazaridan milliy yechim zarurligini asoslashi ko'rsatildi.
- Real vaqtda aloqa texnologiyalari (short va long polling, Server-Sent Events, WebSocket, SignalR) tahlil qilindi. WebSocket samaradorligini saqlagan holda zaxira transportlarga avtomatik o'tadigan SignalR tanlandi.
- Tizimning aktorlari, 12 ta funksional va 7 guruh nofunksional talablari aniqlandi.
- Server qismi Clean Architecture tamoyillari asosida Domain, Application, Infrastructure va API qatlamlariga ajratildi. PostgreSQL ma'lumotlar bazasi 9 ta sxemadagi 32 ta jadvaldan iborat qilib loyihalandi. Tizim ER, use-case va ketma-ketlik diagrammalari bilan modellashtirildi.
- Parolsiz autentifikatsiya tizimi ishlab chiqildi. U elektron pochtaga yuboriladigan va PBKDF2 bilan xeshlanadigan bir martalik kod, Google hisobi, 15 daqiqalik JWT access token, HMAC xeshi saqlanadigan va har safar almashtiriladigan refresh token hamda qurilmalar bo'yicha sessiyalarni boshqarishni o'z ichiga oladi.
- .NET 8 platformasida 56 ta endpointli REST API va SignalR habi amalga oshirildi. Ular shaxsiy va guruh chatlari, xabarlarni yuborish, tahrirlash, o'chirish, javob berish va qidirish, rasm, video va fayllar almashish, "yozmoqda" va o'qilganlik belgilari, onlayn holat hamda 24 soatlik hikoyalarni ta'minlaydi. Guruhlar uchun egasi, administrator va a'zo rollariga asoslangan ruxsatlar tizimi va xizmat xabarlari yaratildi.

- Angular 22 freymvorkida signals asosidagi holat boshqaruvi, o‘z dizayn tizimi, yorug‘ va qorong‘i mavzular va mobil qurilmalarga moslashuvchan o‘zbek tilidagi interfeys ishlab chiqildi.
- Server ilovasi Docker konteyneri ko‘rinishida bulutli platformaga joylashtirildi. Tizimning to‘g‘riligi 39 ta unit test, API darajasidagi 43 ta integratsion tekshiruv va qo‘lda o‘tkazilgan sinov ssenariylari bilan tasdiqlandi.
- Hayot faoliyati xavfsizligi bo‘limida dasturchining ish joyiga qo‘yiladigan ergonomik talablar tizimlashtirildi, dasturchilar xonasi uchun ish joylari soni, sun‘iy yoritish va havo almashinuvi hisoblandi. IT-ofis uchun evakuatsiya reja-sxemasi ishlab chiqildi, hisobiy evakuatsiya vaqti aniqlandi, yong‘in va boshqa avariyalarda xodimlarning harakatlar tartibi belgilandi.

Ishning amaliy ahamiyati shundaki, “Milliy chat” to‘liq ishlaydigan dasturiy mahsulot bo‘lib, uni ta’lim muassasalari, tashkilotlar yoki davlat idoralari o‘z serverlarida, jumladan respublika hududidagi data-markazlarda joylashtirib, ichki muloqot vositasi sifatida qo‘llashi mumkin. Bunda foydalanuvchi ma’lumotlari ustidan to‘liq nazorat mamlakat ichida qoladi. Qatlamli arxitektura va oldindan loyihalangan ma’lumotlar bazasi sxemasi tizimni bosqichma-bosqich kengaytirish imkonini beradi.

Kelajakda tizimni quyidagi yo‘nalishlarda rivojlantirish rejalashtirilgan: kanallar va botlar platformasi; WebRTC asosidagi ovozli va video qo‘ng‘iroqlar; maxfiy chatlar uchun uchdan-uchgacha shifrlash; ikki bosqichli autentifikatsiya va foydalanuvchilarni bloklash; xabarlarga reaksiyalar, so‘rovnomalar va stikerlar; push-bildirishnomalar; mobil ilovalar (Android, iOS); Redis va bir nechta server nusxasi yordamida gorizontol kengaytirish.

FOYDALANILGAN ADABIYOTLAR

Normativ-huquqiy hujjatlar

1. O‘zbekiston Respublikasining “Shaxsga doir ma’lumotlar to‘g‘risida”gi Qonuni. 2019-yil 2-iyul, O‘RQ-547-son.
2. O‘zbekiston Respublikasining “Axborotlashtirish to‘g‘risida”gi Qonuni. 2003-yil 11-dekabr, 560-II-son.
3. O‘zbekiston Respublikasining Mehnat kodeksi. 2022-yil 28-oktabr, O‘RQ-798-son.
4. O‘zbekiston Respublikasining “Mehnatni muhofaza qilish to‘g‘risida”gi Qonuni. 2016-yil 22-sentabr, O‘RQ-410-son.
5. O‘zbekiston Respublikasining “Yong‘in xavfsizligi to‘g‘risida”gi Qonuni. 2009-yil 30-sentabr, O‘RQ-226-son.
6. O‘zbekiston Respublikasining “Favqulodda vaziyatlarda aholini va hududlarni muhofaza qilish to‘g‘risida”gi Qonuni. 1999-yil 20-avgust, 824-I-son.
7. O‘zbekiston Respublikasi Prezidentining “«Raqamli O‘zbekiston – 2030» strategiyasini tasdiqlash va uni samarali amalga oshirish chora-tadbirlari to‘g‘risida”gi Farmoni. 2020-yil 5-oktabr, PF-6079-son.

Asosiy adabiyotlar

8. Martin R. C. Clean Architecture: A Craftsman’s Guide to Software Structure and Design. — Prentice Hall, 2017. — 432 p.
9. Martin R. C. Clean Code: A Handbook of Agile Software Craftsmanship. — Prentice Hall, 2008. — 464 p.
10. Kleppmann M. Designing Data-Intensive Applications. — O’Reilly Media, 2017. — 616 p.
11. Fowler M. Patterns of Enterprise Application Architecture. — Addison-Wesley, 2002. — 560 p.
12. Evans E. Domain-Driven Design: Tackling Complexity in the Heart of Software. — Addison-Wesley, 2003. — 560 p.

- 13.Lock A. ASP.NET Core in Action. 3rd ed. — Manning Publications, 2023.
- 14.Smith J. P. Entity Framework Core in Action. 2nd ed. — Manning Publications, 2021.
- 15.Price M. J. C# 12 and .NET 8 – Modern Cross-Platform Development Fundamentals. 8th ed. — Packt Publishing, 2023.
- 16.Obe R. O., Hsu L. S. PostgreSQL: Up and Running. 3rd ed. — O'Reilly Media, 2017.
- 17.Stallings W. Cryptography and Network Security: Principles and Practice. 8th ed. — Pearson, 2022.
- 18.Fowler M. UML Distilled: A Brief Guide to the Standard Object Modeling Language. 3rd ed. — Addison-Wesley, 2003.
- 19.Yormatov G'.Yo., Isamuxamedov Yo.U. Hayot faoliyati xavfsizligi. — T.: Aloqachi, 2009.

Standartlar va texnik spetsifikatsiyalar

- 20.Fette I., Melnikov A. RFC 6455: The WebSocket Protocol. — IETF, 2011. — <https://www.rfc-editor.org/rfc/rfc6455>
- 21.Jones M., Bradley J., Sakimura N. RFC 7519: JSON Web Token (JWT). — IETF, 2015. — <https://www.rfc-editor.org/rfc/rfc7519>
- 22.Moriarty K., Kaliski B., Rusch A. RFC 8018: PKCS #5: Password-Based Cryptography Specification Version 2.1. — IETF, 2017.
- 23.Krawczyk H., Bellare M., Canetti R. RFC 2104: HMAC: Keyed-Hashing for Message Authentication. — IETF, 1997.
- 24.OWASP Foundation. OWASP Top 10 — 2021. — <https://owasp.org/Top10/>
- 25.ISO 9241-5:1998. Ergonomic requirements for office work with visual display terminals (VDTs) — Part 5: Workstation layout and postural requirements.
- 26.ISO 7010:2019. Graphical symbols — Safety colours and safety signs — Registered safety signs.

- 27.GOST 12.2.032-78. SSBT. Rabocheye mesto pri vypolnenii rabot sidya. Obshchiye ergonomicheskiye trebovaniya.
- 28.GOST 12.1.005-88. SSBT. Obshchiye sanitarno-gigiyenicheskiye trebovaniya k vozdukhу rabochey zony.
- 29.GOST 12.1.004-91. SSBT. Pozharnaya bezopasnost. Obshchiye trebovaniya.
- 30.KMK 2.01.05-98. Tabiiy va sun'iy yoritish. — T.: O'zbekiston Respublikasi Davarxitektqurilish qo'mitasi, 1998.

Internet manbalari

- 31.Microsoft. ASP.NET Core SignalR documentation. — <https://learn.microsoft.com/aspnet/core/signalr/>
- 32.Microsoft. Entity Framework Core documentation. — <https://learn.microsoft.com/ef/core/>
- 33.Microsoft. ASP.NET Core fundamentals. — <https://learn.microsoft.com/aspnet/core/>
- 34.Angular. Official documentation. — <https://angular.dev>
- 35.The PostgreSQL Global Development Group. PostgreSQL Documentation. — <https://www.postgresql.org/docs/>
- 36.Docker Inc. Docker documentation: Multi-stage builds. — <https://docs.docker.com/build/building/multi-stage/>
- 37.Telegram. MTProto Mobile Protocol. — <https://core.telegram.org/mtproto>
- 38.Signal Messenger. Technical documentation. — <https://signal.org/docs/>
- 39.FluentValidation documentation. — <https://docs.fluentvalidation.net>
- 40.xUnit.net documentation. — <https://xunit.net>

ILOVALAR

1-ilova. SignalR habi (ChatHub)

Fayl: NationalChat/src/API/Hubs/ChatHub.cs

```
using System.Security.Claims;
using Application.Features.Authentication;
using Application.Features.Messages;
using Application.Features.Presence;
using Application.Features.Profiles;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.SignalR;

namespace API.Hubs;

[Authorize]
public sealed class ChatHub(
    IMessageRepository messageRepository,
    IChatRealtimeNotifier realtimeNotifier,
    IPresenceTracker presenceTracker,
    IAuthRepository authRepository,
    IProfileRepository profileRepository,
    TimeProvider timeProvider) : Hub
{
    private static readonly TimeSpan OfflineGracePeriod = TimeSpan.FromSeconds(8);

    public override async Task OnConnectedAsync()
    {
        var userId = GetCurrentUserId();
        await Groups.AddToGroupAsync(Context.ConnectionId,
            ChatHubGroups.User(userId));

        if (presenceTracker.AddConnection(userId, Context.ConnectionId))
        {
            var relatedUserIds = await
                profileRepository.GetRelatedUserIdsAsync(userId, Context.ConnectionAborted);
            await realtimeNotifier.UserPresenceChangedAsync(userId, true, null,
                relatedUserIds, Context.ConnectionAborted);
        }

        await base.OnConnectedAsync();
    }

    public override async Task OnDisconnectedAsync(Exception? exception)
    {
        var userId = GetCurrentUserId();
        var sessionId = GetCurrentSessionId();
        await base.OnDisconnectedAsync(exception);

        if (!presenceTracker.RemoveConnection(userId, Context.ConnectionId))
        {
            return;
        }

        // Grace period absorbs quick reconnects (page refresh, brief network drop)
        without flashing "offline".
        await Task.Delay(OfflineGracePeriod, CancellationToken.None);
        if (presenceTracker.IsOnline(userId))
        {
            return;
        }
    }
}
```

```

        var lastSeenAt = timeProvider.GetUtcNow().UtcDateTime;
        var session = await authRepository.FindSessionAsync(userId, sessionId,
CancellationToken.None);
        if (session is not null)
        {
            session.LastActiveAt = lastSeenAt;
            await authRepository.SaveChangesAsync(CancellationToken.None);
        }

        var recipientUserIds = await profileRepository.GetRelatedUserIdsAsync(userId,
CancellationToken.None);
        await realtimeNotifier.UserPresenceChangedAsync(userId, false, lastSeenAt,
recipientUserIds, CancellationToken.None);
    }

    public async Task JoinChat(int chatId)
    {
        await EnsureChatMembershipAsync(chatId);
        await Groups.AddToGroupAsync(Context.ConnectionId,
ChatHubGroups.Chat(chatId));
    }

    public Task LeaveChat(int chatId) =>
        Groups.RemoveFromGroupAsync(Context.ConnectionId, ChatHubGroups.Chat(chatId));

    public async Task SetTyping(int chatId, bool isTyping)
    {
        await EnsureChatMembershipAsync(chatId);
        await Clients.OthersInGroup(ChatHubGroups.Chat(chatId))
            .SendAsync("TypingChanged", new { chatId, userId = GetCurrentUserId(),
isTyping });
    }

    public async Task MarkMessagesRead(int chatId, IReadOnlyCollection<int>
messageIds)
    {
        await EnsureChatMembershipAsync(chatId);
        var ids = messageIds.Where(id => id > 0).Distinct().Take(100).ToArray();
        if (ids.Length == 0) return;

        var readerUserId = GetCurrentUserId();
        await messageRepository.MarkAsReadAsync(chatId, readerUserId, ids,
timeProvider.GetUtcNow().UtcDateTime, Context.ConnectionAborted);
        await realtimeNotifier.MessagesReadAsync(chatId, readerUserId, ids, await
messageRepository.GetMemberUserIdsAsync(chatId, Context.ConnectionAborted),
Context.ConnectionAborted);
    }

    private async Task EnsureChatMembershipAsync(int chatId)
    {
        if (!await messageRepository.IsChatMemberAsync(chatId, GetCurrentUserId(),
Context.ConnectionAborted))
        {
            throw new HubException("Bu chatga kirish huquqi yo'q.");
        }
    }

    private int GetCurrentUserId() =>
        int.Parse(Context.User?.FindFirstValue(ClaimTypes.NameIdentifier)
            ?? throw new HubException("Foydalanuvchi aniqlanmadi."));

    private int GetCurrentSessionId() =>
        int.Parse(Context.User?.FindFirstValue("sid")
            ?? throw new HubException("Sessiya aniqlanmadi."));
}

```


2-ilova. Guruh chatlari servisi (GroupService)

Fayl: NationalChat/src/Application/Features/Groups/Services/GroupService.cs

```
using Application.Features.Files;
using Application.Features.Files.DataTransferObjects.Requests;
using Application.Features.Groups.DataTransferObjects.Requests;
using Application.Features.Groups.DataTransferObjects.Responses;
using Application.Features.Groups.Factories;
using Application.Features.Groups.Mappers;
using Application.Features.Messages;
using Application.Features.Messages.Factories;
using Application.Features.Messages.Mappers;
using Application.Features.Presence;
using Domain.Entities;
using FluentValidation;

namespace Application.Features.Groups;

public sealed class GroupService(
    IGroupRepository repository,
    IFileService fileService,
    IChatRealtimeNotifier realtimeNotifier,
    IPresenceTracker presenceTracker,
    IValidator<CreateGroupRequest> createValidator,
    IValidator<UpdateGroupRequest> updateValidator,
    IValidator<AddGroupMembersRequest> addMembersValidator,
    IValidator<UpdateGroupMemberRoleRequest> updateRoleValidator,
    TimeProvider timeProvider) : IGroupService
{
    private const string NotFoundError = "Guruh topilmadi.";
    private const string ForbiddenError = "Bu amal uchun ruxsat yo'q.";

    public async Task<GroupDto?> GetAsync(int currentUserId, int chatId,
        CancellationToken cancellationToken = default)
    {
        var group = await repository.GetGroupAsync(chatId, cancellationToken);
        return group is null || FindMember(group, currentUserId) is null ? null :
        ToDto(group, currentUserId);
    }

    public async Task<GroupResult> CreateAsync(int currentUserId, CreateGroupRequest
        request, CancellationToken cancellationToken = default)
    {
        var validation = await createValidator.ValidateAsync(request,
            cancellationToken);
        if (!validation.IsValid) return Fail(validation.Errors[0].ErrorMessage);

        var memberIds = request.MemberIds.Where(id => id !=
            currentUserId).Distinct().ToArray();
        if (memberIds.Length == 0) return Fail("Kamida bitta a'zo tanlanishi kerak.");

        var users = await repository.FindUsersAsync([currentUserId, .. memberIds],
            cancellationToken);
        var creator = users.FirstOrDefault(user => user.Id == currentUserId);
        var members = users.Where(user => user.Id != currentUserId).ToArray();
        if (creator is null) return Fail("Foydalanuvchi aniqlanmadi.");
        if (members.Length != memberIds.Length) return Fail("Ba'zi foydalanuvchilar
            topilmadi.");

        var now = timeProvider.GetUtcNow().UtcDateTime;
```

```

        var group = GroupFactory.Create(request.Title.Trim(),
NormalizeDescription(request.Description), creator, members, now);
        await repository.AddGroupAsync(group, cancellationTokens);
        await repository.SaveChangesAsync(cancellationTokens);

        await PublishServiceMessageAsync(group.Chat.Messages.Single(), group,
cancellationTokens);
        return new(ToDto(group, currentUserId), null);
    }

    public async Task<GroupResult> UpdateAsync(int currentUserId, int chatId,
UpdateGroupRequest request, CancellationToken cancellationTokens = default)
    {
        var validation = await updateValidator.ValidateAsync(request,
cancellationTokens);
        if (!validation.IsValid) return Fail(validation.Errors[0].ErrorMessage);

        var group = await repository.GetGroupAsync(chatId, cancellationTokens);
        var actor = group is null ? null : FindMember(group, currentUserId);
        if (group is null || actor is null) return Fail(NotFoundError);
        if (!CanManageGroup(actor)) return Fail(ForbiddenError);

        var title = request.Title.Trim();
        var titleChanged = title != group.Title;
        group.Title = title;
        group.Description = NormalizeDescription(request.Description);
        var serviceMessage = titleChanged
            ? AddServiceMessage(group, actor.User, MessageServiceAction.TitleChanged,
title)
            : null;
        await repository.SaveChangesAsync(cancellationTokens);

        if (serviceMessage is not null) await
PublishServiceMessageAsync(serviceMessage, group, cancellationTokens);
        await PublishGroupUpdatedAsync(group, cancellationTokens);
        return new(ToDto(group, currentUserId), null);
    }

    public async Task<GroupResult> UpdatePhotoAsync(int currentUserId, int chatId,
StoreImageRequest request, CancellationToken cancellationTokens = default)
    {
        var group = await repository.GetGroupAsync(chatId, cancellationTokens);
        var actor = group is null ? null : FindMember(group, currentUserId);
        if (group is null || actor is null) return Fail(NotFoundError);
        if (!CanManageGroup(actor)) return Fail(ForbiddenError);

        var storedResult = await fileService.StoreImageAsync(request,
cancellationTokens);
        if (storedResult.File is null) return Fail(storedResult.Error ?? "Rasm
yuklanmadi.");
        var stored = storedResult.File;

        var oldFile = group.Photo?.File;
        var file = new Domain.Entities.File { Name = stored.FileName, MimeType =
stored.MimeType, SizeBytes = stored.SizeBytes, StoragePath = stored.StoragePath };
        var photo = new Photo { File = file, Width = stored.Width, Height =
stored.Height };
        await repository.AddPhotoAsync(file, photo, cancellationTokens);
        group.Photo = photo;
        var serviceMessage = AddServiceMessage(group, actor.User,
MessageServiceAction.PhotoChanged, null);
        await repository.SaveChangesAsync(cancellationTokens);

        if (oldFile is not null) await fileService.DeleteAsync(oldFile.StoragePath,
cancellationTokens);
    }

```

```

        await PublishServiceMessageAsync(serviceMessage, group, cancellationToken);
        await PublishGroupUpdatedAsync(group, cancellationToken);
        return new(ToDto(group, currentUserId), null);
    }

    public async Task<GroupResult> AddMembersAsync(int currentUserId, int chatId,
AddGroupMembersRequest request, CancellationToken cancellationToken = default)
    {
        var validation = await addMembersValidator.ValidateAsync(request,
cancellationToken);
        if (!validation.IsValid) return Fail(validation.Errors[0].ErrorMessage);

        var group = await repository.GetGroupAsync(chatId, cancellationToken);
        var actor = group is null ? null : FindMember(group, currentUserId);
        if (group is null || actor is null) return Fail(NotFoundError);
        if (!CanManageGroup(actor)) return Fail(ForbiddenError);

        var existingIds = group.Chat.Members.Select(member =>
member.UserId).ToHashSet();
        var newIds = request.UserIds.Where(id =>
!existingIds.Contains(id)).Distinct().ToArray();
        if (newIds.Length == 0) return Fail("Tanlangan foydalanuvchilar allaqachon
guruhda.");
        if (existingIds.Count + newIds.Length > GroupLimits.MaxMembers) return
Fail($"Guruhda {GroupLimits.MaxMembers} tadan ortiq a'zo bo'lishi mumkin emas.");

        var users = await repository.FindUsersAsync(newIds, cancellationToken);
        if (users.Count != newIds.Length) return Fail("Ba'zi foydalanuvchilar
topilmadi.");

        var now = timeProvider.GetUtcNow().UtcDateTime;
        foreach (var user in users)
        {
            group.Chat.Members.Add(GroupFactory.CreateMember(user,
ChatMemberRole.Member, now));
        }

        var names = string.Join(", ", users.Select(GroupMapper.DisplayName));
        var serviceMessage = AddServiceMessage(group, actor.User,
MessageServiceAction.MembersAdded, names);
        await repository.SaveChangesAsync(cancellationToken);

        await PublishServiceMessageAsync(serviceMessage, group, cancellationToken);
        await PublishGroupUpdatedAsync(group, cancellationToken);
        return new(ToDto(group, currentUserId), null);
    }

    public async Task<GroupResult> RemoveMemberAsync(int currentUserId, int chatId,
int userId, CancellationToken cancellationToken = default)
    {
        var group = await repository.GetGroupAsync(chatId, cancellationToken);
        var actor = group is null ? null : FindMember(group, currentUserId);
        if (group is null || actor is null) return Fail(NotFoundError);

        var target = FindMember(group, userId);
        if (target is null) return Fail("Foydalanuvchi guruh a'zosi emas.");
        if (target.UserId == currentUserId) return Fail("Guruhdan chiqish uchun
\"Guruhdan chiqish\" amalidan foydalaning.");
        if (!CanRemove(actor, target)) return Fail(ForbiddenError);

        RemoveMember(group, target);
        var serviceMessage = AddServiceMessage(group, actor.User,
MessageServiceAction.MemberRemoved, GroupMapper.DisplayName(target.User));
        await repository.SaveChangesAsync(cancellationToken);
    }

```

```

        await realtimeNotifier.ChatDeletedAsync(chatId, [target.UserId],
cancellationToken);
        await PublishServiceMessageAsync(serviceMessage, group, cancellationToken);
        await PublishGroupUpdatedAsync(group, cancellationToken);
        return new(ToDto(group, currentUserId), null);
    }

    public async Task<GroupResult> UpdateMemberRoleAsync(int currentUserId, int
chatId, int userId, UpdateGroupMemberRoleRequest request, CancellationToken
cancellationToken = default)
    {
        var validation = await updateRoleValidator.ValidateAsync(request,
cancellationToken);
        if (!validation.IsValid) return Fail(validation.Errors[0].ErrorMessage);

        var group = await repository.GetGroupAsync(chatId, cancellationToken);
        var actor = group is null ? null : FindMember(group, currentUserId);
        if (group is null || actor is null) return Fail(NotFoundError);
        if (actor.Role != ChatMemberRole.Creator) return Fail("Faqat guruh egasi
adminlarni tayinlay oladi.");

        var target = FindMember(group, userId);
        if (target is null) return Fail("Foydalanuvchi guruh a'zosi emas.");
        if (target.Role == ChatMemberRole.Creator) return Fail("Guruh egasining rolini
o'zgartirib bo'lmaydi.");

        target.Role = request.Role;
        await repository.SaveChangesAsync(cancellationToken);

        await PublishGroupUpdatedAsync(group, cancellationToken);
        return new(ToDto(group, currentUserId), null);
    }

    public async Task<GroupLeaveResult> LeaveAsync(int currentUserId, int chatId,
CancellationToken cancellationToken = default)
    {
        var group = await repository.GetGroupAsync(chatId, cancellationToken);
        var leaver = group is null ? null : FindMember(group, currentUserId);
        if (group is null || leaver is null) return new(false, NotFoundError);

        var now = timeProvider.GetUtcNow().UtcDateTime;
        RemoveMember(group, leaver);

        if (group.Chat.Members.Count == 0)
        {
            group.Chat.DeletedAt = now;
            await repository.SaveChangesAsync(cancellationToken);
            await realtimeNotifier.ChatDeletedAsync(chatId, [currentUserId],
cancellationToken);
            return new(true, null);
        }

        if (leaver.Role == ChatMemberRole.Creator)
        {
            // Ownership passes to the longest-serving admin, or to the longest-
serving member if there are no admins.
            var successor = group.Chat.Members
                .OrderByDescending(member => member.Role == ChatMemberRole.Admin)
                .ThenBy(member => member.JoinedAt)
                .ThenBy(member => member.Id)
                .First();
            successor.Role = ChatMemberRole.Creator;
            group.CreatorId = successor.UserId;
            group.Creator = successor.User;
        }
    }

```

```

    }

    var serviceMessage = AddServiceMessage(group, leaver.User,
        MessageServiceAction.MemberLeft, GroupMapper.DisplayName(leaver.User));
    await repository.SaveChangesAsync(cancellationToken);

    await realtimeNotifier.ChatDeletedAsync(chatId, [currentUserId],
        cancellationToken);
    await PublishServiceMessageAsync(serviceMessage, group, cancellationToken);
    await PublishGroupUpdatedAsync(group, cancellationToken);
    return new(true, null);
}

private static bool CanManageGroup(ChatMember actor) =>
    actor.Role is ChatMemberRole.Admin or ChatMemberRole.Creator;

private static bool CanRemove(ChatMember actor, ChatMember target) => actor.Role
switch
{
    ChatMemberRole.Creator => true,
    ChatMemberRole.Admin => target.Role == ChatMemberRole.Member,
    _ => false
};

private static ChatMember? FindMember(Group group, int userId) =>
    group.Chat.Members.FirstOrDefault(member => member.UserId == userId);

private void RemoveMember(Group group, ChatMember member)
{
    group.Chat.Members.Remove(member);
    repository.RemoveMember(member);
}

private Message AddServiceMessage(Group group, User actor, MessageServiceAction
action, string? textContent)
{
    var message = MessageFactory.CreateService(group.ChatId, actor.Id, action,
        textContent, timeProvider.GetUtcNow().UtcDateTime);
    message.Sender = actor;
    group.Chat.Messages.Add(message);
    return message;
}

private Task PublishServiceMessageAsync(Message message, Group group,
    Cancellation token cancellationToken) =>
    realtimeNotifier.MessageCreatedAsync(MessageMapper.ToDto(message),
        MemberIds(group), cancellationToken);

private Task PublishGroupUpdatedAsync(Group group, Cancellation token
cancellationToken) =>
    realtimeNotifier.GroupUpdatedAsync(group.ChatId, group.Title,
        group.Description, group.PhotoId, group.Chat.Members.Count, MemberIds(group),
        cancellationToken);

private static IReadOnlyCollection<int> MemberIds(Group group) =>
    group.Chat.Members.Select(member => member.UserId).ToArray();

private GroupDto ToDto(Group group, int currentUserId) =>
    GroupMapper.ToDto(group, currentUserId, presenceTracker.IsOnline);

private static string? NormalizeDescription(string? description) =>
    string.IsNullOrEmpty(description) ? null : description.Trim();

```

```
private static GroupResult Fail(string error) => new(null, error);
}
```

3-ilova. Bir martalik kodlarni xeshlash (PBKDF2)

Fayl:

NationalChat/src/Infrastructure/Security/Hashing/Pbkdf2OneTimeCodeHasher.cs

```
using System.Security.Cryptography;
using Application.Features.Authentication;

namespace Infrastructure.Security.Hashing;

public sealed class Pbkdf2OneTimeCodeHasher : IOneTimeCodeHasher
{
    private const int Iterations = 210_000;
    private const int SaltSize = 16;
    private const int HashSize = 32;

    public string Hash(string code)
    {
        var salt = RandomNumberGenerator.GetBytes(SaltSize);
        var hash = Rfc2898DeriveBytes.Pbkdf2(code, salt, Iterations,
        HashAlgorithmName.SHA256, HashSize);
        return
        $"{Iterations}.{Convert.ToBase64String(salt)}.{Convert.ToBase64String(hash)}";
    }

    public bool Verify(string code, string storedHash)
    {
        var parts = storedHash.Split('.', StringSplitOptions.RemoveEmptyEntries);
        if (parts.Length != 3 || !int.TryParse(parts[0], out var iterations))
        {
            return false;
        }

        try
        {
            var salt = Convert.FromBase64String(parts[1]);
            var expectedHash = Convert.FromBase64String(parts[2]);
            var actualHash = Rfc2898DeriveBytes.Pbkdf2(code, salt, iterations,
            HashAlgorithmName.SHA256, expectedHash.Length);
            return CryptographicOperations.FixedTimeEquals(actualHash, expectedHash);
        }
        catch (FormatException)
        {
            return false;
        }
    }
}
```

4-ilova. Klientdagi real vaqt ulanishi (ChatRealtimeService)

Fayl: NationalChatClient/src/app/features/chat/data-access/chat-realtime.service.ts

```
async connect(): Promise<void> {
    if (!isPlatformBrowser(this.platformId) || !this.auth.accessToken()) return;
    if (!this.connection) this.createConnection();
    if (this.connection!.state === HubConnectionState.Connected) return;
```

```

    if (this.connectPromise) return this.connectPromise;

    this.connectionState.set('connecting');
    this.connectPromise = this.connection!.start()
      .then(() => this.connectionState.set('connected'))
      .catch((error: unknown) => {
        this.connectionState.set('disconnected');
        throw error;
      })
      .finally(() => (this.connectPromise = undefined));
    return this.connectPromise;
  }

  async joinChat(chatId: number): Promise<void> {
    const previousChatId = this.joinedChatId;
    this.joinedChatId = chatId;
    await this.connect();
    if (!this.connection || this.connection.state !== HubConnectionState.Connected)
    return;
    if (previousChatId && previousChatId !== chatId)
      await this.connection.invoke('LeaveChat', previousChatId);
    await this.connection.invoke('JoinChat', chatId);
  }

  async leaveChat(chatId: number): Promise<void> {
    if (this.joinedChatId === chatId) this.joinedChatId = null;
    if (!this.connection || this.connection.state !== HubConnectionState.Connected)
    return;
    await this.connection.invoke('LeaveChat', chatId);
  }

  async setTyping(chatId: number, isTyping: boolean): Promise<void> {
    if (!this.connection || this.connection.state !== HubConnectionState.Connected)
    return;
    await this.connection.send('SetTyping', chatId, isTyping);
  }

  async markMessagesRead(chatId: number, messageIds: number[]): Promise<void> {
    if (
      !messageIds.length ||
      !this.connection ||
      this.connection.state !== HubConnectionState.Connected
    )
      return;
    await this.connection.send('MarkMessagesRead', chatId, messageIds);
  }

  async disconnect(): Promise<void> {
    this.joinedChatId = null;
    if (this.connection && this.connection.state !== HubConnectionState.Disconnected)
      await this.connection.stop();
    this.connectionState.set('disconnected');
  }

  private createConnection(): void {
    this.connection = new HubConnectionBuilder()
      .withUrl(apiUrl('/hubs/chat'), { accessTokenFactory: () =>
this.auth.accessToken() ?? '' })
      .withAutomaticReconnect([0, 2_000, 5_000, 10_000])
      .configureLogging(LogLevel.Warning)
      .build();

    this.connection.on('MessageReceived', (message: Message) =>
      this.messageReceivedSubject.next(message),
    );
    this.connection.on('TypingChanged', (event: TypingChangedEvent) =>

```

```

        this.typingChangedSubject.next(event),
    );
    this.connection.on('MessagesRead', (event: MessagesReadEvent) =>
        this.messagesReadSubject.next(event),
    );
    this.connection.on('MessageUpdated', (message: Message) =>
        this.messageUpdatedSubject.next(message));
    this.connection.on('MessageDeleted', (event: { chatId: number; messageId: number
    }) => this.messageDeletedSubject.next(event));
    this.connection.on('ChatCleared', (event: { chatId: number }) =>
        this.chatClearedSubject.next(event));
    this.connection.on('ChatDeleted', (event: { chatId: number }) =>
        this.chatDeletedSubject.next(event));
    this.connection.on('ProfilePhotoUpdated', (event: ProfilePhotoUpdatedEvent) =>
        this.profilePhotoUpdatedSubject.next(event));
    this.connection.on('UserPresenceChanged', (event: UserPresenceChangedEvent) =>
        this.userPresenceChangedSubject.next(event));
    this.connection.on('GroupUpdated', (event: GroupUpdatedEvent) =>
        this.groupUpdatedSubject.next(event));
    this.connection.onreconnecting(() => this.connectionState.set('reconnecting'));
    this.connection.onreconnected(async () => {
        this.connectionState.set('connected');
        if (this.joinedChatId) await this.connection?.invoke('JoinChat',
        this.joinedChatId);
    });
    this.connection.onclose(() => this.connectionState.set('disconnected'));

```

5-ilova. Guruh chatlari uchun unit testlar (parcha)

Fayl: NationalChat/tests/NationalChat.Tests/Groups/GroupServiceTests.cs

```

[Fact]
public async Task
Create_ValidRequest_CreatesGroupWithCreatorRoleAndServiceMessage()
{
    var result = await _service.CreateAsync(_sardor.Id, new
    CreateGroupRequest("315-21 guruh", "TATU", [_jasur.Id, _dilnoza.Id]));

    Assert.Null(result.Error);
    Assert.NotNull(result.Group);
    Assert.Equal(ChatMemberRole.Creator, result.Group!.MyRole);
    Assert.Equal(3, result.Group.Members.Count);

    var chat = Assert.Single(_repository.Groups).Chat;
    Assert.Equal(ChatType.Group, chat.Type);
    var message = Assert.Single(chat.Messages);
    Assert.Equal(MessageServiceAction.GroupCreated, message.ServiceAction);

    await _notifier.Received(1).MessageCreatedAsync(
        Arg.Is<MessageDto>(m => m.ServiceAction ==
        MessageServiceAction.GroupCreated),
        Arg.Is<IReadOnlyCollection<int>>(ids => ids.Count == 3),
        Arg.Any<CancellationToken>());
}

[Fact]
public async Task Create_UnknownMember_IsRejected()
{
    var result = await _service.CreateAsync(_sardor.Id, new
    CreateGroupRequest("Guruh", null, [_jasur.Id, 999]));

    Assert.Null(result.Group);
    Assert.NotNull(result.Error);
    Assert.Empty(_repository.Groups);
}

```



```
}

[Fact]
public async Task Create_EmptyTitle_FailsValidation()
{
    var result = await _service.CreateAsync(_sardor.Id, new CreateGroupRequest("
", null, [_jasur.Id]));

    Assert.Null(result.Group);
    Assert.Empty(_repository.Groups);
}
```